Autonomous Robotics-merged

Indice

- 0. Modelli Cinematici E Di Movimento
  - Problemi Chiave Nella Robotica Autonoma
  - Robot Su Ruote E Il Modello Del Disco
  - Ipotesi Teoriche Di Base
  - Classi Di Modelli Cinematici
  - Modello Cinematico Continuo Del Differenziale
  - Discretizzazione Nel Dominio Digitale
  - Velocity Motion Model (Integrazione Esatta Su Arco)
  - Limiti Dell'Odometria Ad Anello Aperto
- 1. Modelli Di Movimento Stocastici
  - Dalla Cinematica Deterministica Ai Modelli Stocastici
  - Calcolo Delle Distribuzioni Vs Campionamento (Sampling)
  - Propagazione Dell'Incertezza E Linearizzazione Di Taylor
  - Modellazione Dell'Incertezza Nei Modelli Cinematici
  - Il Terzo Parametro Di Rotazione (γ)
  - Velocity Motion Model Stocastico Completo
  - Campionamento Con Modelli Eulero E Runge-Kutta 2
  - Analisi Degli Scatter Plot: L'Effetto Delle Varianze
  - Configurazione Di AMCL In ROS E Modelli Di Movimento
- 2. Sensori per la Robotica Mobile
  - Il Ruolo Cruciale Della Percezione Nella Robotica Mobile
  - Tassonomia E Classificazione Dei Sensori
  - Encoder Ottici E Odometria Di Bordo
  - Unità Di Misura Inerziale (IMU)
  - Sensori Di Distanza A Tempo Di Volo: Laser Scanner E LiDAR
  - Visione Artificiale E Il Modello Prospettico
  - Sistemi Di Riferimento Basati Su Beacon E GNSS/GPS
  - Appendice: Caratteristiche Metrologiche E Sensori Fisici
  - Riepilogo Concettuale E Collegamenti
- 3. Rappresentazione dell'Ambiente Ed Estrazione Di Feature
  - Inquadramento Geometrico E Spazio Delle Configurazioni
  - Tassonomia Delle Rappresentazioni Cartografiche
  - Estrazione Di Linee Da Dati Di Distanza (Line Extraction from Range Data)
  - Modello Geometrico Di Retta E Line Fitting
  - Algoritmi Di Segmentazione
  - Tabella Comparativa Degli Algoritmi Di Segmentazione
- 4. Localizzazione E Filtri Di Bayes
  - Dalla Localizzazione Deterministica Alla Stima Stocastica
  - Rappresentazione Del Belief (Stato Di Conoscenza)
  - Modelli Di Markov Nascosti (HMM) E Proprietà Di Markov
  - Algoritmo Del Filtro Di Bayes Ricorsivo
  - Esempio Didattico Dettagliato: La Porta Di Thrun
  - Dimostrazione Formale Del Filtro Di Bayes per Induzione
  - Limiti dell'Ipotesi Di Markov E Verso I Filtri Gaussiani
- 5. Localizzazione Con Histogram Filter
  - Dal Filtro Continuo Alla Tassellatura Dello Spazio
  - Formulazione Discreta Del Filtro Con Mappa
  - Algoritmo Dell'Histogram Filter
  - Calcolo Della Distribuzione Vs Campionamento Stocastico

    - Progettazione Di Un Histogram Filter 1D
    - Analisi Di Sensibilità Rispetto Alla Risoluzione Spaziale
    - Limiti Computazionali E Transizione Verso Spazi 2D
- 6. Localizzazione Con Particle Filter
    - Dal Limite Dell'Histogram Filter Alla Rappresentazione A Particelle
    - Panoramica Visiva Ed Evoluzione Nel Piano 2D
    - Architettura Del Filtro: Ciclo Di Predizione, Correzione E Ricampionamento
    - Fondamenti Matematici Dell'Importance Sampling
    - Algoritmi Di Ricampionamento (Resampling)
    - Algoritmo Formale Del Particle Filter
    - Progettazione Di Un Particle Filter 1D
    - Criticità Ingegneristiche E Compromessi Real-World
    - Adaptive MCL Tramite KLD-Sampling
- 7. Modelli Di Misura per la Localizzazione
    - Il Ruolo Del Modello Di Misura Nel Ciclo Di Stima Bayesiano
    - Cinematica Dei Sistemi Di Riferimento Del Sensore Laser 2D
    - Modello Basato Su Feature (Feature-Based Measurement Model)
    - Modello a Campo Di Verosimiglianza (Likelihood Field Model)
    - Modello a Raggio Fisico (Beam Model)
- 8. Filtro Di Kalman Lineare
    - Dal Filtro Di Bayes Ai Filtri Gaussiani Analitici
    - Modellazione Dei Sistemi Dinamici Lineari Stocastici
    - Il Ponte Con I Controlli Automatici: L'Osservatore Di Luenberger
    - Il Filtro Di Kalman Lineare Time-Varying
    - Interpretazione Bayesiana Del Filtro Di Kalman
    - Proprietà Di Ottimalità E Considerazioni Sulla Stabilità
- Zz. Argomenti Extra
    - Metodi Di Integrazione ODE (Eulero, RK2, RK4)
    - Derivazione Analitica Del Drift Quadratico Nelle IMU
    - Geometria Della Visione Stereoscopica
    - Modellistica Dello Pseudorange Nei Sistemi GNSS / GPS
    - Parametri Metrologici Avanzati E Fisica Dei Sensori
    - Derivazione Analitica Del Line Fitting (OLS E WLS) in Coordinate Polari
    - Formulazione Matriciale Dell'Histogram Filter Come Catena Di Markov
    - Derivazione Formale Della Traiettoria Completa Nel Particle Filter
    - Implementazione Industriale Di AMCL in ROS: Analisi Del Codice C Sorgente
    - Dettaglio Analitico Dei Modelli Di Misura Beam E Stima MLE Dei Parametri
    - Derivazione Formale Del Filtro Di Kalman Lineare Dalla Teoria Bayesiana

0. Modelli Cinematici E Di Movimento

Iniziamo a studiare i modelli di movimento per la robotica autonoma. In questo corso lavoreremo quasi esclusivamente con robot su ruote, anche se molti degli algoritmi che vedremo si possono applicare anche a droni o robot con zampe.

Un robot su ruote non può muoversi liberamente in qualsiasi direzione: ha vincoli cinematici stringenti. Guidando un'auto sappiamo bene che non possiamo traslare lateralmente istantaneamente, ma con opportune manovre possiamo raggiungere qualsiasi posizione e orientamento nel parcheggio.

Problemi Chiave Nella Robotica Autonoma

Un robot autonomo deve rispondere a tre domande fondamentali:

1. **Dove mi trovo?**: problema della **Localizzazione** e dello **SLAM** (con o senza mappa iniziale, indoor o outdoor).

2. **Come arrivo all'obiettivo?**: problema del **Planning** (pianificazione geometrica di percorsi e traiettorie evitando ostacoli).
3. **Come mi muovo concretamente?**: problema del **Motion Control** (tecniche di controllo in retroazione per inseguire la traiettoria).

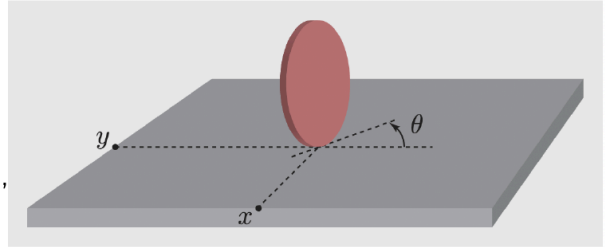
Il percorso logico del corso segue proprio questa sequenza:

$$\text{Localizzazione / SLAM} \implies \text{Planning} \implies \text{Motion Control}$$

I modelli cinematici di movimento sono il punto di partenza: ci servono sia per pianificare come muoverci, sia per stimare la posizione del robot integrando i comandi nel tempo (odometria).

Robot Su Ruote E Il Modello Del Disco

Per comprendere la cinematica di un veicolo su ruote, partiamo dal caso elementare di un disco che rotola su un piano orizzontale mantenendo il proprio piano sagittale (il piano verticale che contiene il disco):



Lo stato del sistema è descritto dalla **posa del robot** $\mathbf{x}$, che rappresenta il vettore nello spazio di configurazione $\mathcal{C}$:

$$\mathbf{x} = \begin{bmatrix} x \\ y \\ \theta \end{bmatrix}$$

Dove (x, y) sono le coordinate del punto di contatto sul piano cartesiano e θ è l'angolo di orientamento (imbardata) rispetto all'asse x.

> **Note**
>
> Dal punto di vista geometrico formale, $\mathcal{C}$ è una varietà differenziabile e non un semplice spazio vettoriale, perché θ è una coordinata angolare periodica ($\theta \in S^1$). Lavorando su traiettorie nel piano 2D possiamo comunque trattarla con la terminologia degli spazi vettoriali; la distinzione diventa cruciale quando si lavora in 3D con orientamenti nello spazio.

Vincolo Non-Olonomo E Segnali Di Ingresso

Il disco è soggetto a un **vincolo non-olonomo**: in assenza di slittamento, la velocità del punto di contatto con il suolo ha componente identicamente nulla nella direzione perpendicolare al piano sagittale. In altre parole, il disco può avanzare o ruotare, ma non può traslare istantaneamente di lato.

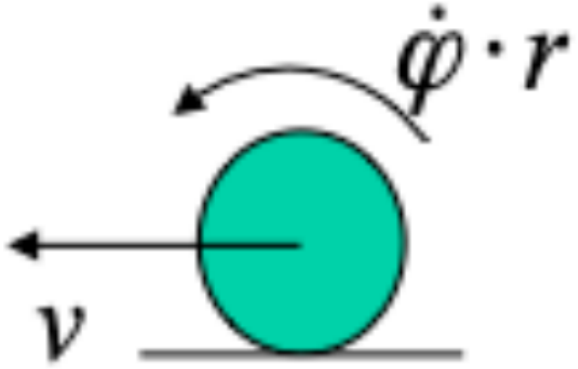
Un modello cinematico mette in relazione la variazione della posa con i segnali di comando in ingresso. Per il disco rotolante gli ingressi di controllo sono:

- v: velocità lineare di traslazione nel piano sagittale (avanti / indietro).
- ω: velocità angolare attorno all'asse verticale (rotazione / imbardata $\dot{\theta}$).

In questa fase non consideriamo forze, masse o coppie motrici: ci muoviamo nel campo della cinematica pura, dove assumiamo di poter imporre direttamente le velocità desiderate.

Ipotesi Teoriche Di Base

Per definire modelli cinematici analitici trattabili formuliamo alcune ipotesi ideali sul contatto ruota-suolo:



- Moto confinato su una superficie perfettamente piana.
- Ruote rigide e indeformabili con contatto puntiforme con il terreno.
- Condizione di puro rotolamento (*pure rolling*): velocità nulla nel punto di contatto istantaneo.
- Assenza di fenomeni di slittamento laterale (*no skidding*).
- Assenza di attrito rispetto alla rotazione attorno al punto di contatto verticale.
- Assi di sterzo perfettamente ortogonali al piano di movimento.
- Ruote vincolate tra loro da un telaio rigido (*chassis*).

> **Important**
>
> Nel mondo reale nessuna di queste ipotesi ideali è verificata alla perfezione: gli pneumatici si deformano, il terreno presenta asperità e durante le sterzate si generano inevitabili slittamenti. Per questo i modelli cinematici da soli non bastano a localizzare il robot ad anello aperto: forniscono una prima stima (odometria) che deve essere continuamente corretta tramite sensori esterni (LiDAR, telecamere, filtri di Kalman/Bayes).

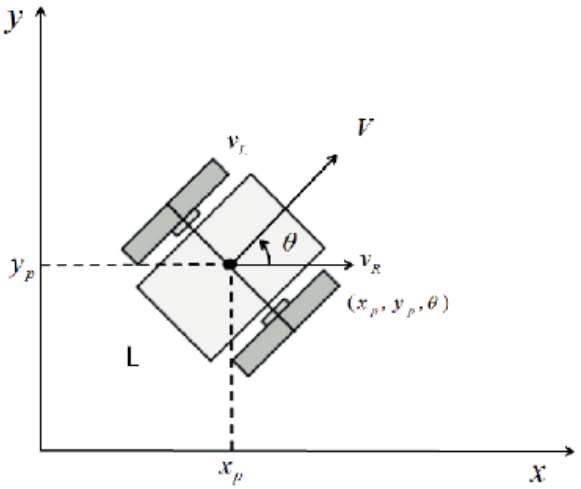
Classi Di Modelli Cinematici

I veicoli terrestri su ruote si raggruppano principalmente in tre famiglie costruttive:

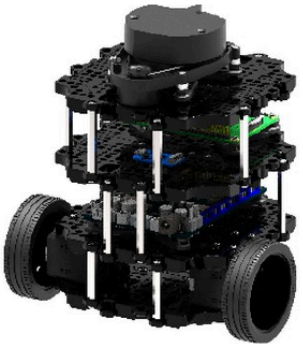
1. Differential Drive (Unicycle)

È l'architettura classica dei robot mobili da laboratorio e dei robot aspirapolvere (es. Roomba): due ruote motrici indipendenti montate su un asse comune e una o due ruote folli (*castor wheels*) passive per garantire stabilità:

- Posa: $\mathbf{x} = [x, y, \theta]^T$
- Segnale di controllo: $\mathbf{u} = [v, \omega]^T$ (velocità lineare e velocità angolare)







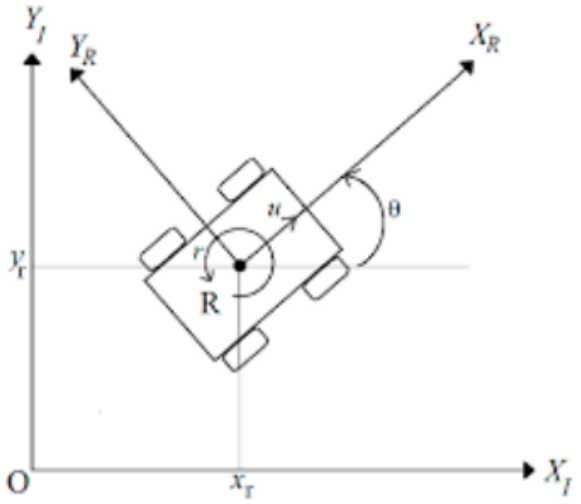




2. Skid-Steering

Comune nei robot cingolati, nelle ruspe compatte (tipo Bobcat) e nei rover da esplorazione: ha ruote o cingoli a orientamento fisso. Per curvare si applicano velocità diverse ai lati destro e sinistro, forzando le ruote a slittare lateralmente sul suolo:

- Posa: $\mathbf{x} = [x, y, \theta]^T$
- Segnale di controllo: $\mathbf{u} = [v, \omega]^T$





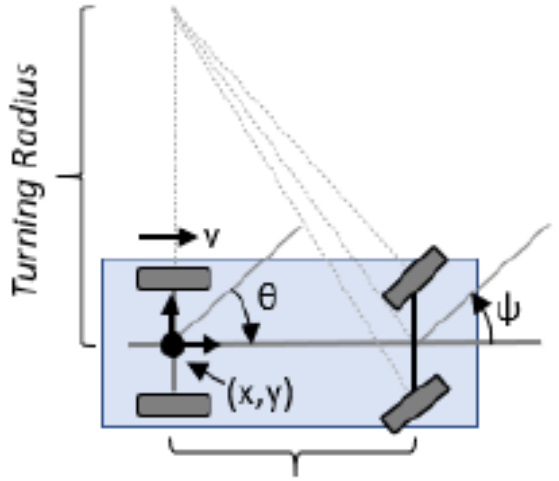


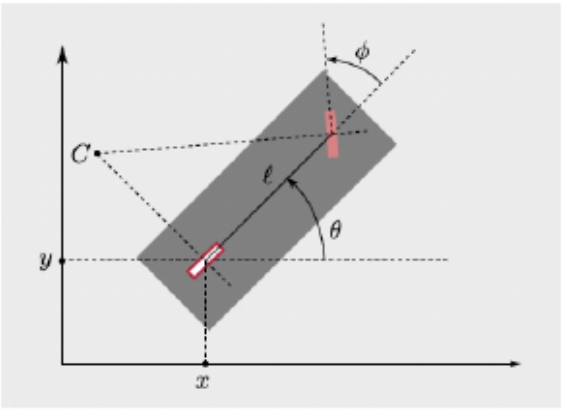


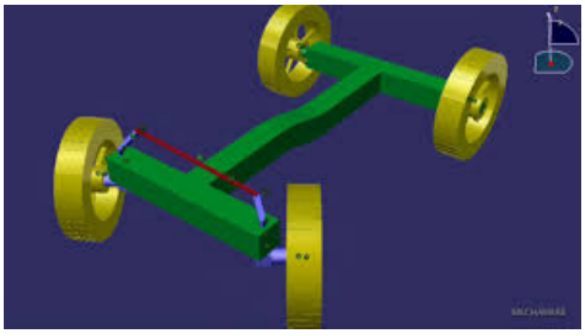
3. Ackermann Steering (Car-Like)

È lo schema cinematico delle automobili e delle biciclette: ruote posteriori di trazione fisse e ruote anteriori sterzanti azionate da un quadrilatero articolato che evita lo strisciamento in curva:

- Posa: $\mathbf{x} = [x, y, \theta, \phi]^T$, dove ϕ indica l'angolo di sterzo delle ruote anteriori.
- Segnale di controllo: $\mathbf{u} = [v, \omega]^T$ (velocità longitudinale e velocità di variazione dello sterzo).





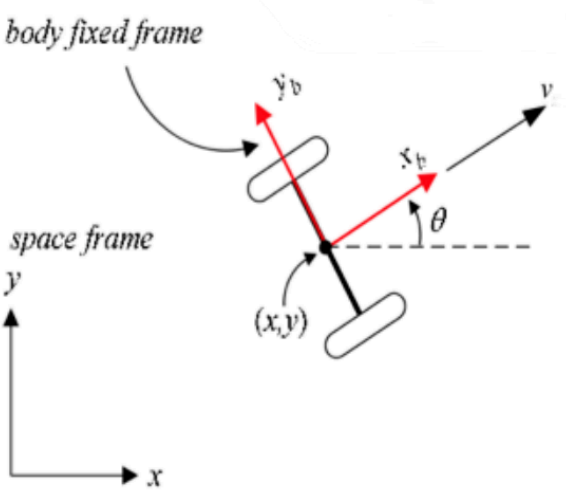




> **Note**
>
> Nel seguito ci concentreremo quasi esclusivamente sul modello a guida differenziale (unicycle), che costituisce lo standard de facto anche nel middleware ROS (dove le velocità vengono scambiate sul topic `/cmd_vel`).

Modello Cinematico Continuo Del Differenziale

Fissiamo una terna di riferimento cartesiana fissa inerziale nel mondo (*space frame*) e una terna solidale al telaio del robot (*body frame*):



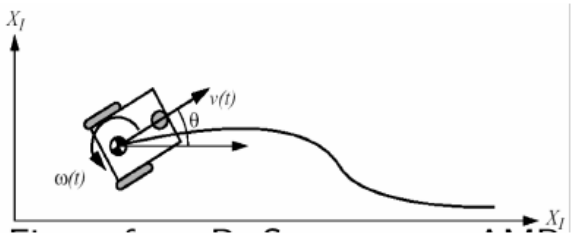
La posa e il comando di controllo sono:

$$\mathbf{x} = \begin{bmatrix} x \\ y \\ \theta \end{bmatrix}, \qquad \mathbf{u} = \begin{bmatrix} v \\ \omega \end{bmatrix}$$

Proiettando la velocità lineare v lungo gli assi cartesiani dello space frame otteniamo il sistema di equazioni differenziali ordinarie (ODE) del primo ordine:

$$\begin{cases} \dot{x} = v\cos\theta \\ \dot{y} = v\sin\theta \\ \dot{\theta} = \omega \end{cases}$$

Conoscendo la posa iniziale $\mathbf{x}_0$ e la storia temporale dei comandi $\mathbf{u}(t) = [v(t), \omega(t)]^T$, integrando queste equazioni nel tempo possiamo ricostruire la traiettoria percorsa dal robot nel piano:

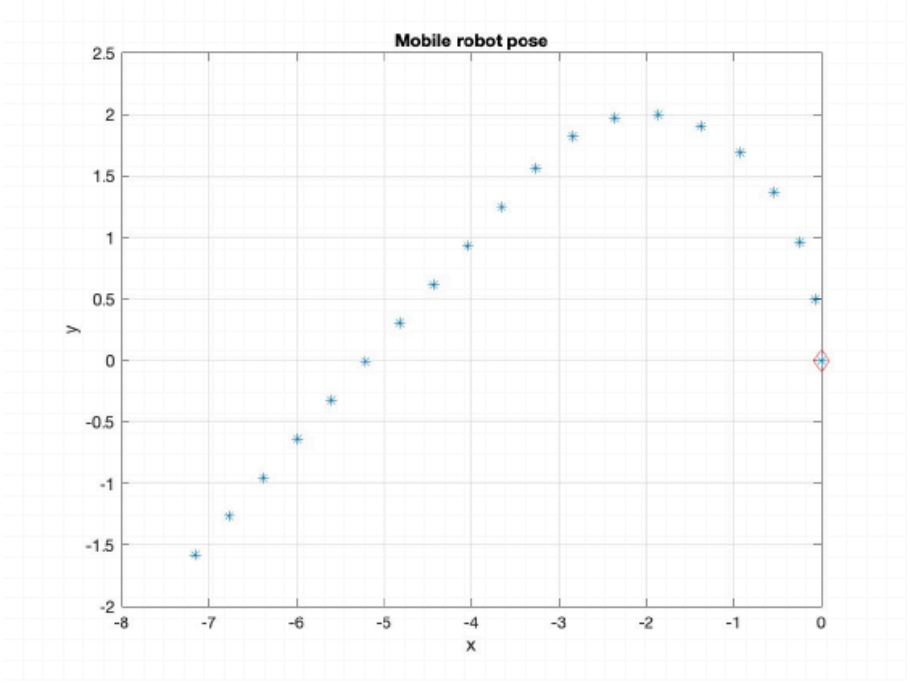


Discretizzazione Nel Dominio Digitale

A bordo del robot le decisioni vengono calcolate da un computer a intervalli temporali discreti T (periodo di campionamento costante).

Tra un istante di campionamento e il successivo i segnali di controllo sono tenuti costanti da un circuito di tenuta ad ordine zero (*zero-order hold*). Vogliamo quindi determinare una formula ricorsiva che aggiorni la posa:

$$\mathbf{x}_{t+1} = f(\mathbf{x}_t, \mathbf{u}_t, T)$$



Metodo Di Eulero Esplicito

Il metodo di Eulero approssima la derivata con il rapporto incrementale in avanti, calcolando la velocità all'inizio dell'intervallo temporale (θ_t costante su tutto il passo):

$$
\begin{aligned}
x_{t+1} &= x_t + v_t T \cos\theta_t \\
y_{t+1} &= y_t + v_t T \sin\theta_t \\
\theta_{t+1} &= \theta_t + \omega_t T
\end{aligned}
$$

Questo metodo assume che il robot si muova lungo la tangente alla curva calcolata all'istante iniziale. È molto rapido, ma se il passo T non è piccolissimo tende a tagliare le curve e ad accumulare rapidamente errore.

```
def euler(p, v, w, T):
    x, y, th = p
    xn = x + v * T * np.cos(th)
    yn = y + v * T * np.sin(th)
    thn = th + w * T
    return np.array([xn, yn, thn])
```

Metodo Di Runge-Kutta Al Secondo Ordine (RK-2)

Per ottenere una traiettoria più fedele possiamo usare il metodo di Runge-Kutta al secondo ordine (metodo del punto medio / midpoint).

Invece di proiettare la velocità sull'orientamento iniziale θ_t, valutiamo la direzione a metà dell'intervallo di tempo, ruotando l'angolo di metà dell'incremento angolare complessivo ($\theta_t + \frac{\omega_t T}{2}$):

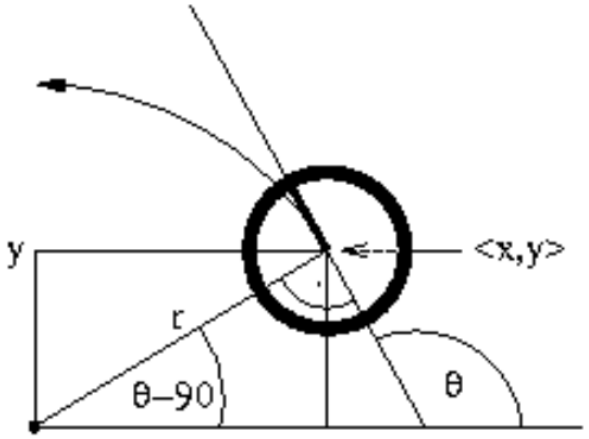
$$
\begin{aligned}
x_{t+1} &= x_t + v_t T \cos\left(\theta_t + \frac{\omega_t T}{2}\right) \\
y_{t+1} &= y_t + v_t T \sin\left(\theta_t + \frac{\omega_t T}{2}\right) \\
\theta_{t+1} &= \theta_t + \omega_t T
\end{aligned}
$$

In questo modo la corda approssima molto meglio l'arco reale di traiettoria percorso dal veicolo:

```
def runge_kutta(p, v, w, T):
    x, y, th = p
    th_mid = th + 0.5 * w * T
    xn = x + v * T * np.cos(th_mid)
    yn = y + v * T * np.sin(th_mid)
    thn = th + w * T
    return np.array([xn, yn, thn])
```

Velocity Motion Model (Integrazione Esatta Su Arco)

Eulero e Runge-Kutta sono schemi numerici approssimati. Se però assumiamo che i comandi di velocità v e ω siano **rigorosamente costanti** durante il periodo di controllo T, il moto del robot differenziale non deve essere approssimato: si muove esattamente lungo un **arco di circonferenza**.



Dal moto circolare uniforme sappiamo che il raggio dell'arco di traiettoria è dato da:

$$
r = \frac{v}{\omega} \quad (\text{poiché } v = \omega r)
$$

Posizione Del Centro Di Rotazione Istantaneo (ICR)

Dato il robot nella posa corrente x, y, θ, il raggio vettore che dal centro di rotazione x_c, y_c punta verso il robot forma un angolo polare pari a $\theta - 90^\circ$ rispetto all'orizzontale:

$$
\begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} x_c \\ y_c \end{bmatrix} + r \begin{bmatrix} \cos(\theta - 90^\circ) \\ \sin(\theta - 90^\circ) \end{bmatrix}
$$

Isolando le coordinate del centro x_c, y_c:

$$x_c = x - r\cos(\theta - 90^\circ) = x - \frac{v}{\omega}\sin\theta$$

$$y_c = y - r\sin(\theta - 90^\circ) = y - \left(-\frac{v}{\omega}\cos\theta\right) = y + \frac{v}{\omega}\cos\theta$$

> **Note**
>
> Ricordando l'identità goniometrica $\sin(\theta - 90^\circ) = -\cos\theta$, il segno negativo della proiezione all'indietro si compensa con il segno meno del coseno, restituendo correttamente $+\frac{v}{\omega}\cos\theta$.

Posa A Fine Intervallo

Durante l'intervallo temporale T, il robot ruota attorno al centro di un angolo $\Delta\theta = \omega T$. L'angolo della congiungente centro-robot diventa quindi $(\theta - 90^\circ) + \omega T = (\theta + \omega T) - 90^\circ$. La posa finale x', y', θ' è:

$$x' = x_c + r\cos((\theta + \omega T) - 90^\circ) = x_c + \frac{v}{\omega}\sin(\theta + \omega T)$$

$$y' = y_c + r\sin((\theta + \omega T) - 90^\circ) = y_c - \frac{v}{\omega}\cos(\theta + \omega T)$$

$$\theta' = \theta + \omega T$$

Sostituendo le coordinate del centro (x_c, y_c) otteniamo il **Velocity Motion Model**:

.

$$\begin{cases} x_t = x_{t-1} - \frac{v_t}{\omega_t}\sin\theta_{t-1} + \frac{v_t}{\omega_t}\sin(\theta_{t-1} + \omega_t T) \\ y_t = y_{t-1} + \frac{v_t}{\omega_t}\cos\theta_{t-1} - \frac{v_t}{\omega_t}\cos(\theta_{t-1} + \omega_t T) \\ \theta_t = \theta_{t-1} + \omega_t T \end{cases}$$

A differenza di Runge-Kutta che proietta in avanti da t a $t+1$, questo modello viene tipicamente formulato a ritroso per calcolare la posa presente all'istante t a partire dalla posa al passo precedente $t-1$ e dai comandi applicati (v_t, ω_t).

> **Caso Di Moto Rettilineo** $(\omega = 0)$
>
> Se il robot va dritto ($\omega = 0$), non possiamo dividere per zero. In tal caso l'arco collassa in un segmento retto percorso a velocità costante:
>
> $$x_t = x_{t-1} + v_t T\cos\theta_{t-1}$$
>
> $$y_t = y_{t-1} + v_t T\sin\theta_{t-1}$$
>
> $$\theta_t = \theta_{t-1}$$

L'implementazione Python gestisce la soglia singolare tramite un controllo su ω:

```
def velocity_motion_model(p, v, w, T, eps=1e-9):
    x, y, th = p
```

```
    if abs(w) < eps:
        xn = x + v * T * np.cos(th)
        yn = y + v * T * np.sin(th)
        thn = th
    else:
        r = v / w
        thn = th + w * T
        xn = x - r * np.sin(th) + r * np.sin(thn)
        yn = y + r * np.cos(th) - r * np.cos(thn)
    return np.array([xn, yn, thn])
```

Limiti Dell'Odometria Ad Anello Aperto

Tutti i modelli visti finora sono **deterministici**: stimano la posa basandosi unicamente su grandezze propriocettive (i comandi inviati ai motori o le letture degli encoder alle ruote).

Possiamo affidarci a questi modelli per risolvere da soli il problema della localizzazione? No, per via di sorgenti di errore inevitabili:

- **Errori sistematici**: disallineamento meccanico delle ruote, raggio effettivo leggermente diverso tra ruota destra e sinistra, incertezza sulla reale carreggiata del telaio.
- **Errori non sistematici (stocastici)**: slittamento su pavimenti lisci o tappeti, dislivelli e ostacoli imprevisti, accumulo di giochi negli ingranaggi.

Integrando nel tempo ad anello aperto, questi piccoli errori si sommano a ogni passo: l'odometria inevitabilmente **deriva** (*drift*), e la posa stimata divergerà da quella reale.

Per questo motivo, la cinematica e l'odometria costituiscono il modello di previsione temporale, ma per mantenere la stima accurata dovremo fondere le misure dei sensori esterocettivi (LiDAR, videocamere, sonar) all'interno di filtri di stima stocastici come i filtri di Bayes, l'Extended Kalman Filter (EKF) e gli algoritmi di SLAM.

1. Modelli Di Movimento Stocastici

Nel capitolo precedente 0. Modelli Cinematici e di Movimento abbiamo ricavato i modelli cinematici assumendo che il moto del robot fosse perfettamente deterministico: date le velocità v e ω, applicando il modello calcoliamo esattamente dove andrà a finire il veicolo.

Nella realtà operativa dei robot fisici questa certezza svanisce:

- Le ruote slittano impercettibilmente sull'asfalto o sui pavimenti lisci.
- Il raggio effettivo delle ruote non è mai identico al millimetro tra destra e sinistra.
- Gli encoder hanno risoluzione finita e i motori non erogano istantaneamente la velocità richiesta.

Se continuiamo a usare un modello deterministico, l'errore di odometria si accumula a ogni passo provocando una rapida deriva della posa stimata. Per gestire questa incertezza dobbiamo passare a una trattazione rigorosamente **stocastica**.

Dalla Cinematica Deterministica Ai Modelli Stocastici

Nei modelli deterministici la transizione della posa da un passo al successivo ha la forma:

$$\mathbf{x}_t = f(\mathbf{x}_{t-1}, \mathbf{u}_t)$$

Nel contesto stocastico incorporiamo esplicitamente l'incertezza introducendo una variabile aleatoria multivariata ϵ_t:

$$\mathbf{x}_t = f(\mathbf{x}_{t-1}, \mathbf{u}_t, \epsilon_t)$$

Il nuovo stato $\mathbf{x}_t$ non è più un valore deterministico singolo, ma una variabile aleatoria distribuita secondo una densità di probabilità condizionata:

$$p(\mathbf{x}_t \mid \mathbf{x}_{t-1}, \mathbf{u}_t)$$

Questa densità prende il nome di **probabilità di transizione di stato** (*state transition probability*) e descrive quanto sia probabile trovarsi nella posa $\mathbf{x}_t$ avendo applicato il comando $\mathbf{u}_t$ a partire dalla posa precedente $\mathbf{x}_{t-1}$.

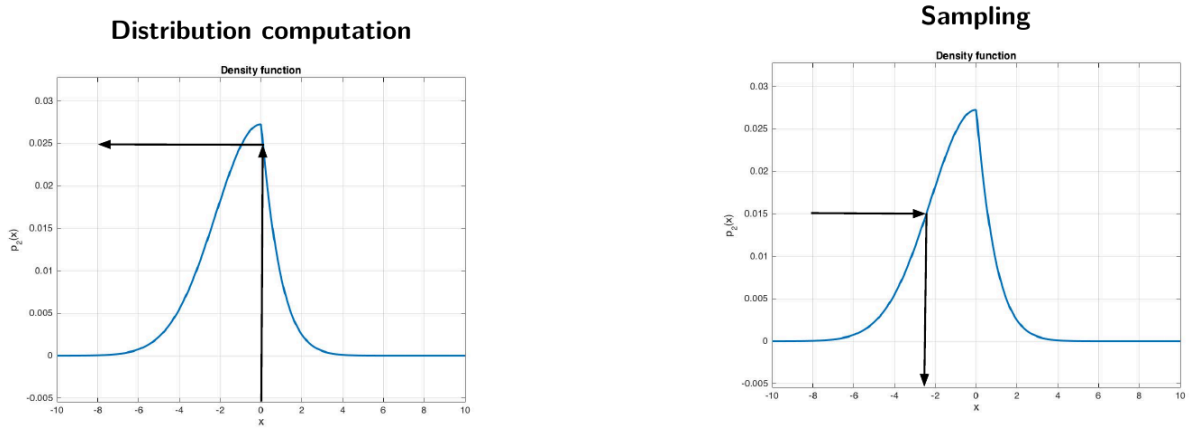
Lo stesso approccio si applica alle misure raccolte dai sensori esterocettivi (laser scanner, sonar, fotocamere): se indichiamo con $\mathbf{z}_t$ la misura rilevata al tempo t, la relazione con la posa reale è descritta dalla densità di verosimiglianza:

$$p(\mathbf{z}_t \mid \mathbf{x}_t)$$

Queste due funzioni di densità costituiscono le colonne portanti della localizzazione probabilistica e dei filtri di Bayes.

Calcolo Delle Distribuzioni Vs Campionamento (Sampling)

Quando manipoliamo densità di probabilità in robotica ci troviamo costantemente davanti a due operazioni concettualmente opposte:



1. Calcolo Della Distribuzione (Distribution Computation)

Dato un valore specifico della variabile aleatoria (una realizzazione x), vogliamo calcolare il corrispondente valore di densità di probabilità:

$$x \implies p(x)$$

Nel grafico a sinistra entriamo con il valore x sull'asse delle ascisse e risaliamo verticalmente fino alla curva per leggere la probabilità sull'asse delle ordinate.

Questa operazione serve quando dobbiamo valutare la plausibilità di una transizione o calcolare la verosimiglianza di una misura (ad esempio nei filtri a griglia discreta o nella fase di correzione dei filtri bayesiani).

2. Campionamento (Sampling)

Data una funzione di densità di probabilità, vogliamo generare numericamente una o più realizzazioni concrete (campioni o particelle) che rispettino quella distribuzione:

$$p(x) \implies x^{(i)} \ p(x)$$

Il grafico a destra rappresenta una classica metafora visiva (introdotta nelle lezioni di Wolfram Burgard e Cyrill Stachniss) che illustra il campionamento come "problema inverso" rispetto alla valutazione della densità: partire dal dominio probabilistico per scendere a un valore di posa x.

Precisazione Matematica Sul Sampling Per Inversione

A livello matematico rigoroso, **non è possibile campionare una densità invertendo direttamente la PDF** $p(x)$: la densità non è monotona, non è biunivoca (un valore di $p(x)$ corrisponde in genere a più valori di x) e può superare il valore 1.
Il campionamento per inversione esatto (*Inverse Transform Sampling*) richiede di invertire la **funzione di ripartizione cumulativa (CDF)**:

$$(x) = P(X \leq x) = \int_{-\infty}^{x} p(t)\, t$$

> Estraendo un numero casuale uniforme u $(0, 1)$, il campione è dato da $x = ^{-1}(u)$. Quando la CDF non è nota o invertibile analiticamente, si ricorre a tecniche come il campionamento per rigetto (*Rejection Sampling*), a metodi Monte Carlo (MCMC, Importance Sampling) o a generatori dedicati per distribuzioni note (come Box-Muller o somme di uniformi per le gaussiane).

Il sampling è l'operazione fondamentale degli algoritmi Monte Carlo e del Particle Filter (AMCL), dove rappresentiamo lo stato incerto del robot mediante una nube di particelle generate campionando ripetutamente dal modello di movimento stocastico.

Esempio Scalare Lineare

Per capire la dualità tra le due operazioni, consideriamo un modello scalare unidimensionale con rumore gaussiano additivo a media nulla:

$$x_t = x_{t-1} + u_x + _{}, \qquad _{} \ (0, ^2_{})$$

1. **Per calcolare la probabilità** di una transizione osservata (x_t, x_{t-1}, u_x), isoliamo il rumore invertendo il modello:

   $$_{} = x_t - (x_{t-1} + u_x)$$

   E valutiamo la densità normale standard:

   $$p(x_t \mid x_{t-1}, u_x) = \frac{1}{\sqrt{2\pi^2_{}}} \exp\left(-\frac{1}{2} \frac{^2_{}}{^2_{}} \right)$$

2. **Per fare sampling**, estraiamo un campione di disturbo $_{}$ dalla distribuzione $(0, ^2_{})$ e calcoliamo direttamente il nuovo stato:

   $$x_t = x_{t-1} + u_x + _{}$$

In sintesi: il modello diretto genera i campioni, mentre il modello inverso (risolto rispetto al rumore) permette di calcolare le probabilità.

Propagazione Dell'Incertezza E Linearizzazione Di Taylor

Quando una variabile aleatoria attraversa una funzione di stato, come cambia la sua dispersione (matrice di covarianza)?

Caso Lineare Esatto

Se la relazione tra le variabili aleatorie multivariate X e $\dot{}$ è puramente lineare:

$$= X$$

La matrice di covarianza C si propaga in forma chiusa **esatta senza alcuna approssimazione**:

$$C = C X^T$$

Se è presente un termine di disturbo additivo indipendente con covarianza C ($= X +$), per l'indipendenza statistica la covarianza totale diventa la somma:

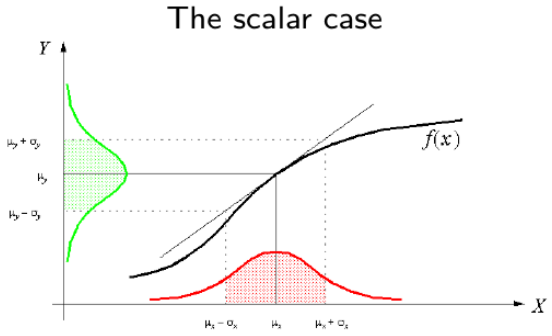
$$C = C X^T + C$$

Caso Non Lineare Generale

Nei robot mobili le relazioni cinematiche contengono funzioni trigonometriche ($\cos\theta, \sin\theta$) e sono intrinsecamente non lineari:

$$= f(X)$$

Non possiamo trasformare la covarianza in modo analitico esatto. Dobbiamo approssimare la funzione non lineare $f(X)$ sviluppandola in serie di Taylor al primo ordine attorno al valore atteso $= E[X]$:



$$f(X) \approx f( ) + X(X -)$$

Dove X è la **matrice Jacobiana** delle derivate parziali calcolata nel punto di lavoro $$.

$$X = \frac{f}{X} \Big|_{X=}$$

Sostituendo l'approssimazione lineare otteniamo la matrice di covarianza propagata:

$$C \approx X C X^T$$

Nel grafico scalare vediamo chiaramente l'effetto geometrico: la distribuzione gaussiana rossa su X viene proiettata sulla retta tangente in per generare la distribuzione gaussiana verde su . Questa linearizzazione locale è il fondamento matematico dell'**Extended Kalman Filter (EKF)**.

Modellazione Dell'Incertezza Nei Modelli Cinematici

Come introduciamo il rumore nei modelli cinematici del robot mobile?

Un errore ingenuo consisterebbe nell'aggiungere rumore bianco direttamente sulle coordinate cartesiane $[x, y, \theta]$. Ma nella realtà fisica gli errori non nascono dal nulla nelle coordinate del mondo: nascono dagli attuatori e dal contatto delle ruote con il suolo.

Se il robot è fermo, la sua posizione non deve accumulare incertezza di movimento! L'approccio corretto consiste nel modellare il disturbo sui **comandi di velocità** inviati ai motori:

$$\begin{pmatrix}\hat{v}\\ \hat{\omega}\end{pmatrix} = \begin{pmatrix}v\\ \omega\end{pmatrix} + \begin{pmatrix}\varepsilon_{\alpha_3}\\ \varepsilon_{\alpha_4}\end{pmatrix}$$

Dove ε_{α_3} sono variabili casuali a media nulla con varianza $\sigma^2_{\alpha_3}$.

Parametrizzazione Delle Varianze

Nel modello probabilistico classico di Thrun, Burgard e Fox la varianza del rumore cresce con l'intensità delle velocità comandate: più il robot va veloce o curva bruscamente, maggiore è l'incertezza generata:

$$\sigma^2_{v} = \alpha_1 v^2 + \alpha_2 \omega^2$$

$$\sigma^2_{\omega} = \alpha_3 v^2 + \alpha_4 \omega^2$$

I parametri $\alpha_1, \alpha_2, \alpha_3, \alpha_4 \geq 0$ sono costanti caratteristiche del robot che ne quantificano l'accuratezza cinematica:

- α_1: incertezza sulla velocità lineare causata dalla traslazione stessa.
- α_2: incertezza sulla velocità lineare causata dalla rotazione.
- α_3: incertezza sulla velocità angolare causata dalla traslazione.
- α_4: incertezza sulla velocità angolare causata dalla rotazione.

In alcune formulazioni alternative le varianze vengono scalate con i valori assoluti ($\sigma^2_v = \alpha_1|v| + \alpha_2|\omega|$), ma la forma quadratica è quella standard.

Il Terzo Parametro Di Rotazione (γ)

Analizzando il modello precedente notiamo un limite geometrico critico: abbiamo 2 parametri di disturbo ($\sigma^2_v, \sigma^2_\omega$) per modellare l'incertezza su 3 variabili di posa (x, y, θ).

Questo vincola l'orientamento θ a dipendere rigidamente dall'arco di cerchio percorso: l'incertezza rimane confinata su una varietà bidimensionale nello spazio 3D delle configurazioni.

Nella realtà delle piattaforme robotiche si verificano scivolamenti indipendenti sull'orientamento finale dovuti a piccole irregolarità del terreno o all'attrito delle ruote folli. Per catturare questa libertà si introduce una terza variabile casuale additiva ε_γ:

$$\hat{\theta} = \theta + \hat{\omega}T + \varepsilon_\gamma T, \qquad \varepsilon_\gamma \sim \mathcal{N}(0, \sigma^2_\gamma)$$

Con varianza dipendente anch'essa dai comandi applicati:

$$\sigma^2_\gamma = \alpha_5 v^2 + \alpha_6 \omega^2$$

Otteniamo così il set completo di 6 parametri costruttivi $(\alpha_1, \alpha_2, \alpha_3, \alpha_4, \alpha_5, \alpha_6)$.

Velocity Motion Model Stocastico Completo

Combinando le velocità rumorose con l'integrazione su arco di cerchio vista nella Lezione 0, otteniamo la formulazione stocastica completa del Velocity Motion Model:

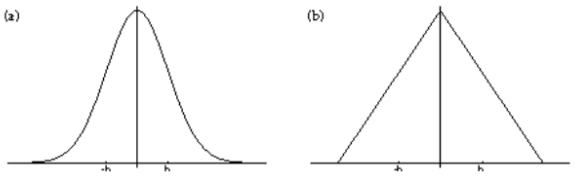
$$\begin{bmatrix} x_t \\ y_t \\ \theta_t \end{bmatrix} = \begin{bmatrix} x_{t-1} \\ y_{t-1} \\ \theta_{t-1} \end{bmatrix} + \begin{bmatrix} -\frac{\hat{v}}{\hat{\omega}}\sin\theta_{t-1} + \frac{\hat{v}}{\hat{\omega}}\sin(\theta_{t-1} + \hat{\omega}T) \\ \frac{\hat{v}}{\hat{\omega}}\cos\theta_{t-1} - \frac{\hat{v}}{\hat{\omega}}\cos(\theta_{t-1} + \hat{\omega}T) \\ \hat{\omega}T + \gamma T \end{bmatrix}$$

I comandi rumorosi generati a ogni iterazione sono:

$$\begin{bmatrix} \hat{v} \\ \hat{\omega} \\ \gamma \end{bmatrix} = \begin{bmatrix} v + \text{sample}(\alpha_1 v^2 + \alpha_2 \omega^2) \\ \omega + \text{sample}(\alpha_3 v^2 + \alpha_4 \omega^2) \\ \text{sample}(\alpha_5 v^2 + \alpha_6 \omega^2) \end{bmatrix}$$

Distribuzioni Di Campionamento Del Rumore

Per estrarre i campioni di rumore sample possiamo impiegare due distribuzioni:



1. **Distribuzione Normale Gaussiana**:

$$\text{prob}(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{1}{2}\frac{x^2}{\sigma^2}\right)$$

   In sistemi embedded può essere approssimata sommando 12 variabili uniformi nel range $[-b, b]$ (grazie al teorema del limite centrale, $\sum_{i=1}^{12} \text{rand}(-b, b)/2$).
2. **Distribuzione Triangolare**:

$$\text{prob}(x) = \max\left(0, \frac{1}{\sqrt{6}b} - \frac{|x|}{6b^2}\right)$$

   Ha il vantaggio computazionale di avere supporto limitato $[-\sqrt{6}b, \sqrt{6}b]$: non genera campioni infinitamente lontani e si campiona velocemente con due sole variabili uniformi:

$$\text{sample} = \frac{\sqrt{6}}{2}(\text{rand}(-b, b) + \text{rand}(-b, b))$$

Implementazione Dell'Algoritmo Di Campionamento

Ecco l'algoritmo completo in Python che calcola un nuovo campione di posa:

```python
import numpy as np

def sample_motion_model_velocity(x_prev, u, alpha, T):
    """Campiona una nuova posa secondo il Velocity Motion Model stocastico.

    x_prev: [x, y, theta] al tempo t-1
    u: [v, w] comandi nominali
    alpha: [a1, a2, a3, a4, a5, a6] parametri di rumore
    T: periodo di campionamento
    """
    x, y, th = x_prev
```

```
    v, w = u
    a1, a2, a3, a4, a5, a6 = alpha

    # Varianze del rumore proporzionali ai comandi
    var_v = a1 * v**2 + a2 * w**2
    var_w = a3 * v**2 + a4 * w**2
    var_g = a5 * v**2 + a6 * w**2

    # Campionamento dei comandi rumorosi (distribuzione normale)
    v_hat = v + np.random.normal(0.0, np.sqrt(max(var_v, 1e-12)))
    w_hat = w + np.random.normal(0.0, np.sqrt(max(var_w, 1e-12)))
    gamma_hat = np.random.normal(0.0, np.sqrt(max(var_g, 1e-12)))

    # Propagazione su arco di circonferenza
    if np.isclose(w_hat, 0.0):
        x_prime = x + v_hat * T * np.cos(th)
        y_prime = y + v_hat * T * np.sin(th)
        th_prime = th + gamma_hat * T
    else:
        r_hat = v_hat / w_hat
        x_prime = x - r_hat * np.sin(th) + r_hat * np.sin(th + w_hat * T)
        y_prime = y + r_hat * np.cos(th) - r_hat * np.cos(th + w_hat * T)
        th_prime = th + w_hat * T + gamma_hat * T

    return np.array([x_prime, y_prime, th_prime])
```

Campionamento Con Modelli Eulero E Runge-Kutta 2

Lo stesso schema di rumore sui comandi $\hat{v}, \hat{\omega},$,si applica direttamente ai modelli discreti approssimati:

Modello Di Eulero Stocastico

I comandi rumorosi vengono applicati lungo la direzione tangente iniziale θ:

$$\begin{cases} x' = x + T\hat{v}\cos\theta \\ y' = y + T\hat{v}\sin\theta \\ \theta' = \theta + T\hat{\omega} + T\, \end{cases}$$

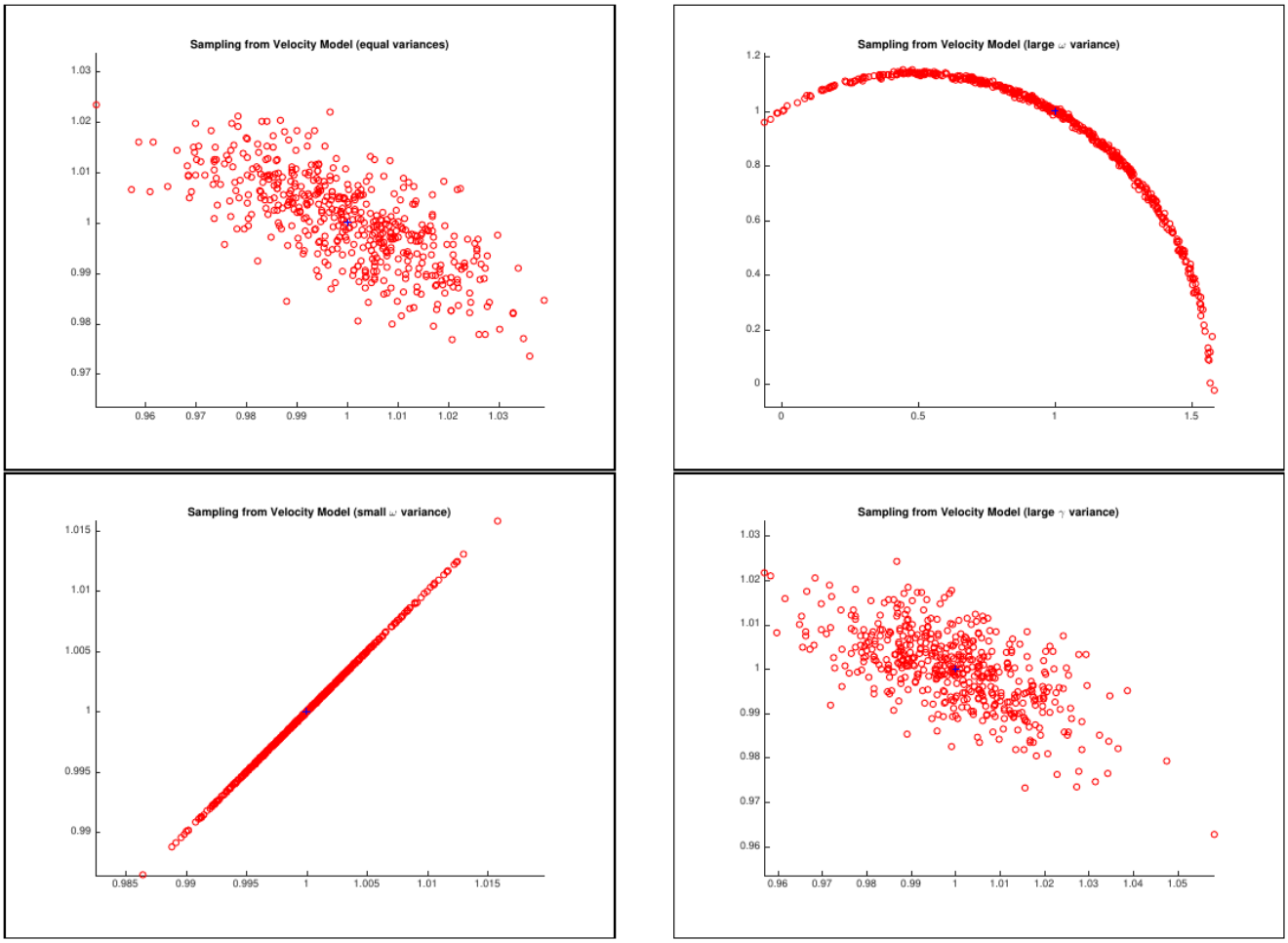
Modello Di Runge-Kutta 2 (Midpoint) Stocastico

La direzione viene valutata a metà del passo temporale ruotando dell'angolo intermedio:

$$\begin{cases} x' = x + T\hat{v}\cos\left(\theta + \dfrac{\hat{\omega}T}{2}\right) \\ y' = y + T\hat{v}\sin\left(\theta + \dfrac{\hat{\omega}T}{2}\right) \\ \theta' = \theta + T\hat{\omega} + T\, \end{cases}$$

Analisi Degli Scatter Plot: L'Effetto Delle Varianze

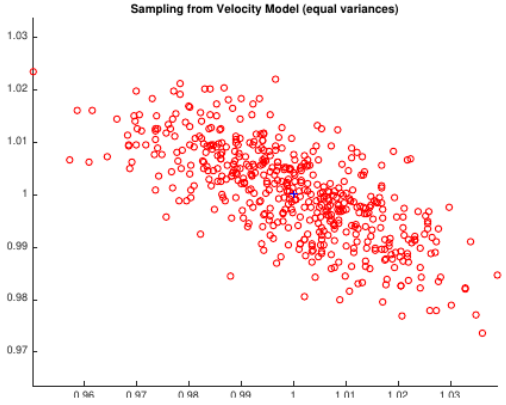
Cosa succede quando generiamo centinaia di campioni con il Velocity Motion Model variando i parametri di rumore?



I quattro grafici mettono a confronto la posa precedente (la crocetta blu al centro in $1, 1$) con la nube di pose al passo successivo (i cerchietti rossi):

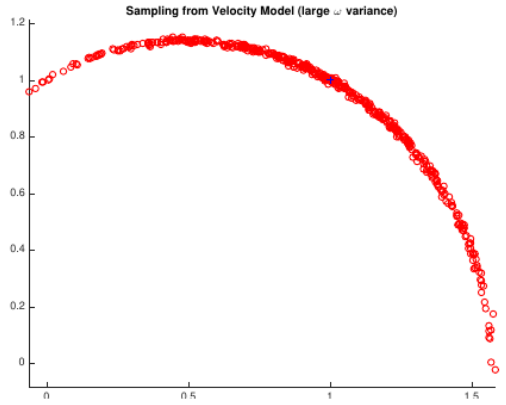
1. Varianze Bilanciate (Top-Left)

Quando le varianze su velocità lineare e angolare sono bilanciate (.comparabili), la nuvola di campioni assume una forma ellissoidale allungata lungo la traiettoria, con dispersione uniforme:



2. Elevata Varianza Angolare ω (Top-Right): La "Banana Shape"

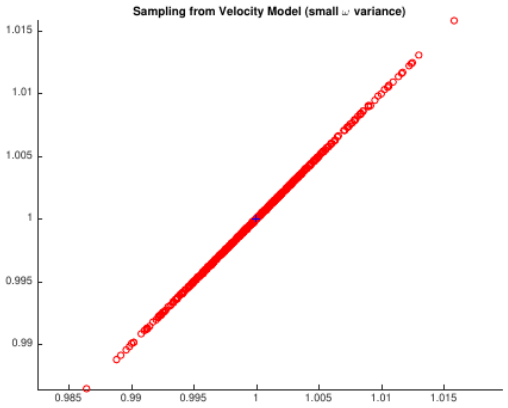
Quando la varianza sulla velocità angolare è elevata rispetto a quella traslazionale, la nube di pose si dispone lungo un arco di cerchio ben visibile, originando la celebre forma arcuata a banana (*banana-shaped distribution*):



Poiché il raggio della traiettoria è $r = \hat{v}/\hat{\omega}$, piccole variazioni di $\hat{\omega}$ provocano grandi cambiamenti nel raggio di curvatura, curvando la distribuzione delle particelle nello spazio cartesiano. Questa non-gaussianità palese è la ragione principale per cui l'Extended Kalman Filter (che presuppone distribuzioni gaussiane) fatica su lunghe manovre curve, mentre il Particle Filter eccelle.

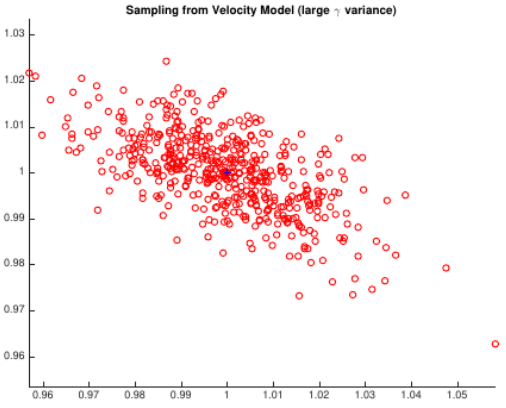
3. Bassa Varianza Angolare ω (Bottom-Left)

Se la varianza su ω è quasi trascurabile rispetto a quella su v, il robot procede quasi in linea retta: i campioni collassano su una retta sottile allineata con l'avanzamento longitudinale:



4. Elevata Varianza Su (Bottom-Right)

Quando incrementiamo il terzo parametro ,,l'orientamento θ subisce oscillazioni casuali supplementari, allargando l'apertura angolare della nube di pose senza alterare la curvatura media della traiettoria:



Configurazione Di AMCL In ROS E Modelli Di Movimento

Nei framework robotici reali come ROS1 e ROS2 (Nav2), il package di riferimento per la localizzazione probabilistica è **AMCL** (*Adaptive Monte Carlo Localization*).

All'interno dei file di configurazione (`amcl.launch` in ROS1 o `amcl.yaml` in ROS2) troviamo i parametri di rumore dell'odometria:

```
# Frammento configurazione Nav2 AMCL (ROS2)
amcl:
  ros__parameters:
    use_sim_time: false
    min_particles: 500
    max_particles: 2000

    # Parametri di rumore del modello di odometria
    alpha1: 0.2  # Errore di rotazione dovuto alla rotazione (rot -> rot)
    alpha2: 0.2  # Errore di rotazione dovuto alla traslazione (trans -> rot)
    alpha3: 0.2  # Errore di traslazione dovuto alla traslazione (trans -> trans)
    alpha4: 0.2  # Errore di traslazione dovuto alla rotazione (rot -> trans)
    alpha5: 0.1  # Rumore di traslazione laterale (solo robot omnidirezionali)
```

Chiarimento Teorico: Velocity Model vs Odometry Model

Spesso si genera confusione confrontando i parametri `alpha1` … `alpha4` di AMCL con le slide del **Velocity Motion Model**. È fondamentale chiarire che nel testo fondamentale *Probabilistic Robotics* (Thrun, Burgard, Fox) coesistono **due modelli di movimento distinti**:

1. **Velocity Motion Model (Capitolo 5.3)**:

- Si basa sui comandi nominali di velocità $\mathbf{u}_t = [v_t, \omega_t]^T$ inviati ai motori.
- Utilizza 6 parametri $(\alpha_1 \dots \alpha_6)$ dove α_1, α_2 pesano la varianza della velocità lineare v, α_3, α_4 la velocità angolare ω, e α_5, α_6 la rotazione additiva γ.

2. **Odometry Motion Model (Capitolo 5.4)**:

- Si basa sulle misure relative fornite dagli encoder delle ruote, scomponendo il moto tra due istanti successivi in tre passi geometrici discreti: prima rotazione δ_{rot1}, avanzamento δ_{trans}, seconda rotazione δ_{rot2}.
- Per questo modello, **Thrun stesso** definisce i 4 parametri esattamente come compaiono in ROS/AMCL:
  - α_1: incertezza di rotazione generata dalla rotazione ($\text{rot} \to \text{rot}$)
  - α_2: incertezza di rotazione generata dalla traslazione ($\text{trans} \to \text{rot}$)
  - α_3: incertezza di traslazione generata dalla traslazione ($\text{trans} \to \text{trans}$)
  - α_4: incertezza di traslazione generata dalla rotazione ($\text{rot} \to \text{trans}$)

Non si tratta quindi di una "inversione" arbitraria operata dagli sviluppatori di ROS, ma dell'adozione dell'Odometry Motion Model (Cap. 5.4).

Nelle slide del corso, il professore adatta euristicamente la notazione di AMCL alle formule continue del Velocity Model:

$$\sigma_{\dot{x}}^2 = \alpha_1 v^2 + \alpha_2 \omega^2, \qquad \sigma_{\dot{y}}^2 = \alpha_3 v^2 + \alpha_4 \omega^2, \qquad \sigma_{\dot{\gamma}}^2 = \alpha_5 \omega^2$$

Tuttavia, esaminando il codice sorgente reale di AMCL (`amcl_diff.cpp` e `amcl_omni.cpp`):

- Per un robot differenziale (`diff` o `diff-corrected`), α_5 **non viene utilizzato affatto**: bastano unicamente i 4 parametri $\alpha_1 \dots \alpha_4$.

- Il parametro `alpha5` viene impiegato esclusivamente per piattaforme omnidirezionali (`omni`), dove modella la componente di rumore sulla traslazione laterale ortogonale (*strafe*).

2. Sensori per la Robotica Mobile

Nei capitoli precedenti (0. Modelli Cinematici e di Movimento e 1. Modelli di Movimento Stocastici) abbiamo affrontato la propagazione dello stato del robot nel tempo partendo dai comandi di controllo e dalla cinematica delle ruote. Abbiamo dimostrato matematicamente come, anche con modelli probabilistici raffinati, la pura integrazione nel tempo (odometria o *dead reckoning*) accumuli inesorabilmente incertezza: la varianza della distribuzione cresce senza limite e il robot "diventa cieco" rispetto alla propria posizione reale nel mondo.

Per spezzare questa deriva divergente e chiudere l'anello di retroazione dobbiamo dotare il veicolo di capacità percettive. Come evidenziato nei testi di riferimento della robotica autonoma (*Siegwart, Nourbakhsh, Scaramuzza - Introduction to Autonomous Mobile Robots*, MIT Press), la percezione sensoriale costituisce la tecnologia abilitante indispensabile per permettere a una piattaforma autonoma di interagire con l'ambiente circostante, estrarre punti di riferimento (*landmarks*), stimare le distanze dagli ostacoli e correggere l'errore di posa.

Il Ruolo Cruciale Della Percezione Nella Robotica Mobile

Un ingegnere robotico non può considerare i sensori come semplici blocchi a scatola chiusa che restituiscono la "verità a terra" (*ground truth*). Ogni dispositivo fisico è governato da leggi ottiche, elettromagnetiche o acustiche che ne determinano limiti intrinseci, zone d'ombra e specifiche modalità di guasto.

I tre interrogativi cardine della navigazione autonoma dipendono interamente dall'apparato sensoriale di bordo:

1. **Where am I? (Localizzazione e SLAM)**: richiede di confrontare i dati sensoriali correnti con una mappa nota o di stimare contemporaneamente mappa e traiettoria.
2. **Where am I going? (Planning)**: impone di ricostruire la geometria dello spazio libero per calcolare percorsi liberi da collisioni.
3. **How do I get there? (Controllo e Reattività)**: necessita di un flusso continuo di informazioni ad alta frequenza per evitare ostacoli dinamici imprevisti.

Nel nostro laboratorio (ISARLab) sono operative piattaforme robotiche eterogenee, ciascuna equipaggiata con una specifica suite sensoriale tarata sul dominio operativo:

Agrobot platform (ISARLab equipment)



- Velodyne Lidar
- RTK, GPS and IMU
- Stereo camera and monocular camera
- Wheel encoders
- Multispectral camera (mostly monitoring)
- Kinematics: Skid steering

La piattaforma **Agrobot** è concepita per la navigazione autonoma in contesti agricoli non strutturati (vigneti e oliveti). Presenta una cinematica a slittamento (*skid steering*), trazione integrale su quattro ruote motrici e una suite di sensori altamente ridondante:

- **LiDAR 3D Velodyne**: per la scansione volumetrica dei filari e la navigazione tra gli alberi.
- **Modulo RTK, GNSS/GPS e IMU**: per la georeferenziazione assoluta con precisione centimetrica in campo aperto.
- **Telecamera stereoscopica e monoculare**: per la stima visiva dell'odometria e il riconoscimento della vegetazione.
- **Encoder ottici sulle ruote**: per l'odometria cinematico-differenziale a breve termine.
- **Camera multispettrale**: dedicata al monitoraggio dello stato di salute delle colture e dell'indice di vigore vegetativo.

TurtleBot (ISARLab equipment)



- Wheel encoders
- Laser
- Stereo Camera
- Jetson TX2 board
- Kinematics: Differential drive

La piattaforma **TurtleBot** è invece il riferimento per la robotica di servizio indoor su pavimenti lisci. Dotata di cinematica a trazione differenziale pura, integra:

- **Encoder ottici di quadratura**: calettati direttamente sull'asse dei due motori di trazione.
- **Laser scanner ToF planare 2D**: per la mappatura orizzontale delle stanze e l'algoritmo AMCL.
- **Stereo camera RGB-D**: per l'identificazione degli ostacoli tridimensionali fuori dal piano laser.
- **Unità di calcolo a bordo (board NVIDIA Jetson TX2)**: deputata all'elaborazione neurale e all'inferenza in tempo reale dei pacchetti sensoriali ROS.

Tassonomia E Classificazione Dei Sensori

I sensori impiegati nella robotica mobile possono essere classificati secondo due criteri ortogonali fondamentali:

1. **Origine dell'informazione**: Sensori Propriocettivi vs Esterocettivi.
2. **Modalità di interazione energetica**: Sensori Passivi vs Attivi.

Definizione: Sensori Propriocettivi (PC)

I sensori **propriocettivi** misurano grandezze fisiche interne allo stato del robot. Non richiedono alcuna interazione con l'ambiente esterno circostante. Rientrano in questa categoria la velocità angolare delle ruote (encoder), la tensione e corrente dei motori, l'accelerazione lineare e la velocità angolare del telaio (accelerometri e giroscopi della IMU).

Definizione: Sensori Esterocettivi (EC)

I sensori **esterocettivi** acquisiscono informazioni provenienti direttamente dall'ambiente operativo esterno al robot. Rientrano in questa classe i sensori di distanza (LiDAR, sonar a ultrasuoni, ToF a infrarossi), i sensori di visione (telecamere monoculari, stereo ed eventi) e i sistemi di posizionamento assoluto (ricevitori GPS/GNSS, antenne di tracciamento beacon RF o ottici).

Per quanto concerne l'interazione energetica:

- **Sensori Passivi**: misurano l'energia ambientale naturalmente presente senza emettere segnali propri. Esempi tipici sono le telecamere standard (che catturano la luce solare o artificiale riflessa), le bussole magnetiche (che rilevano il campo magnetico terrestre) e i microfoni.
- **Sensori Attivi**: emettono energia controllata nell'ambiente (impulsi laser, onde elettromagnetiche radio, impulsi a ultrasuoni) e ne analizzano la porzione riflessa dall'ostacolo. Offrono generalmente un rapporto segnale-rumore (*SNR*) molto più elevato rispetto ai passivi, ma possono soffrire di auto-interferenza tra robot vicini e comportano un assorbimento di potenza significativamente superiore.

La tabella seguente sintetizza la tassonomia dei sensori di comune impiego nella robotica mobile:

Tipologia di misura	Sensore fisico	Propriocettivo (PC) / Esterocettivo (EC)	Principio operativo
Ruote / Motori	Encoder ottici (incrementali / assoluti)	PC	Interruzione o riflessione di fasci luminosi su reticolo
	Resolver, Synchros	PC	Accoppiamento elettromagnetico induttivo
	Potenziometri rotativi	PC	Partitore di tensione resistivo a contatto strisciante
Heading (Orientamento)	Bussola magnetica (fluxgate, Hall, AMR)	EC	Misura vettoriale del campo magnetico terrestre
	Giroscopi (meccanici, MEMS, ottici FOG)	PC	Conservazione del momento angolare / Forza di Coriolis
Accelerazione / Inerzia	Accelerometri (piezo, capacitivi, ToF MEMS)	PC	Deflessione di una massa sismica vincolata
Ground Beacons	Beacon attivi (ottici, RF, ultrasuoni, UWB)	EC	Triangolazione o trilaterazione su riferimenti noti
	GPS / GNSS / Galileo / GLONASS	EC	Trilaterazione satellitare ToF con correzione d'orologio
Active Ranging (Distanza)	Sonar a ultrasuoni	EC	Tempo di volo di pacchetti di onde acustiche
	Laser scanner 2D / LiDAR 3D	EC	Tempo di volo o modulazione di fase di luce laser collimata
	Sensori a luce strutturata / ToF Camera	EC	Pattern a infrarossi proiettato o ToF su sensore a matrice
Visione	Telecamere CCD / CMOS (monoculari)	EC	Proiezione prospettica su matrice fotosensibile
	Telecamere stereoscopiche binoculari	EC	Disparità spaziale su baseline nota con triangolazione

La Sfida Della Percezione Nel Mondo Reale

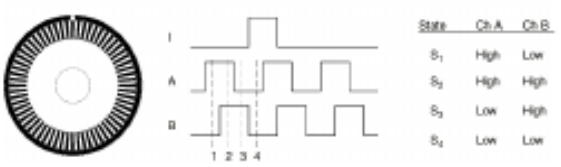
Un robot mobile deve percepire, analizzare e interpretare lo stato del proprio ambiente partendo da misurazioni affette da rumore e ambiguità fisica. L'ambiente fisico reale non si comporta mai come un laboratorio ideale:

- **Riflessioni speculari**: superfici levigate, specchi o vetrate riflettono i fasci laser o ultrasonici allontanandoli dal ricevitore, causando la completa invisibilità dell'ostacolo (*drop-out*) o la stima di distanze enormemente sovrastimate dovute a rimbalzi multipli.

- **Assorbimento ottico e acustico**: superfici nere opache assorbono la quasi totalità della lunghezza d'onda laser emessa (spesso nel vicino infrarosso, ad es. a $\theta 5\ \mathrm{nm}$); materiali fonoassorbenti disperdono gli impulsi ultrasonici.
- **Cross-sensitivity (Sensibilità incrociata)**: un sensore progettato per misurare una grandezza cinematica primaria risulta invariabilmente influenzato da parametri ambientali secondari (ad esempio, la deriva termica nei giroscopi e negli oscillatori al quarzo, la sensibilità delle telecamere ai repentini cambi di illuminazione o all'abbagliamento solare diretto).
- **Struttura del rumore di misura**: gli errori sensoriali non sono puramente gaussiani a media nulla; contengono spesso *outliers* sistematici causati da ostruzioni parziali, riflessioni secondarie o perdita momentanea di tracciamento.

Encoder Ottici E Odometria Di Bordo

L'encoder è un dispositivo elettromeccanico che converte la posizione angolare o il moto rotatorio di un albero in un segnale analogico o digitale, fungendo a tutti gli effetti da **trasduttore di angolo**.



Un encoder rotativo incrementale ottico è costituito da un disco trasparente calettato sull'albero rotante, su cui è serigrafato un reticolo circolare alternato di settori opachi e trasparenti ad alta densità. Una sorgente luminosa (LED infrarosso) illumina il reticolo, mentre sul lato opposto una coppia di fotorilevatori (fotodiodi o fototransistor) genera una transizione di tensione logica ogni volta che un settore opaco interrompe il fascio.



Canali In Quadratura E Determinazione Del Verso

Se l'encoder possedesse un singolo canale di fotorilevazione, il conteggio degli impulsi consentirebbe di stimare l'entità della rotazione, ma risulterebbe del tutto cieco rispetto al **verso di rotazione** (orario o antiorario).

Per risolvere questa ambiguità si impiegano due canali distinti, denominati **Canale A** e **Canale B**, geometricamente sfalsati nello spazio di un quarto di periodo, corrispondente a uno sfasamento elettrico di θ° (quadratura di fase).

Come illustrato nello schema dei segnali, durante la rotazione si generano due onde quadre sfasate nel tempo:

- Quando il disco ruota in un verso, il fronte di salita del canale A precede il fronte di salita del canale B.
- Quando il disco inverte la direzione di marcia, il canale B precede il canale A.

Ciascun periodo dell'onda quadra viene suddiviso in 4 stati discreti (S_1, S_2, S_3, S_4):

Stato logico	Canale A	Canale B	Transizione oraria	Transizione antioraria
S_1	High	Low	da S_4 a S_1	da S_2 a S_1
S_2	High	High	da S_1 a S_2	da S_3 a S_2
S_3	Low	High	da S_2 a S_3	da S_4 a S_3
S_4	Low	Low	da S_3 a S_4	da S_1 a S_4

Moltiplicazione della Risoluzione ($4\times$)

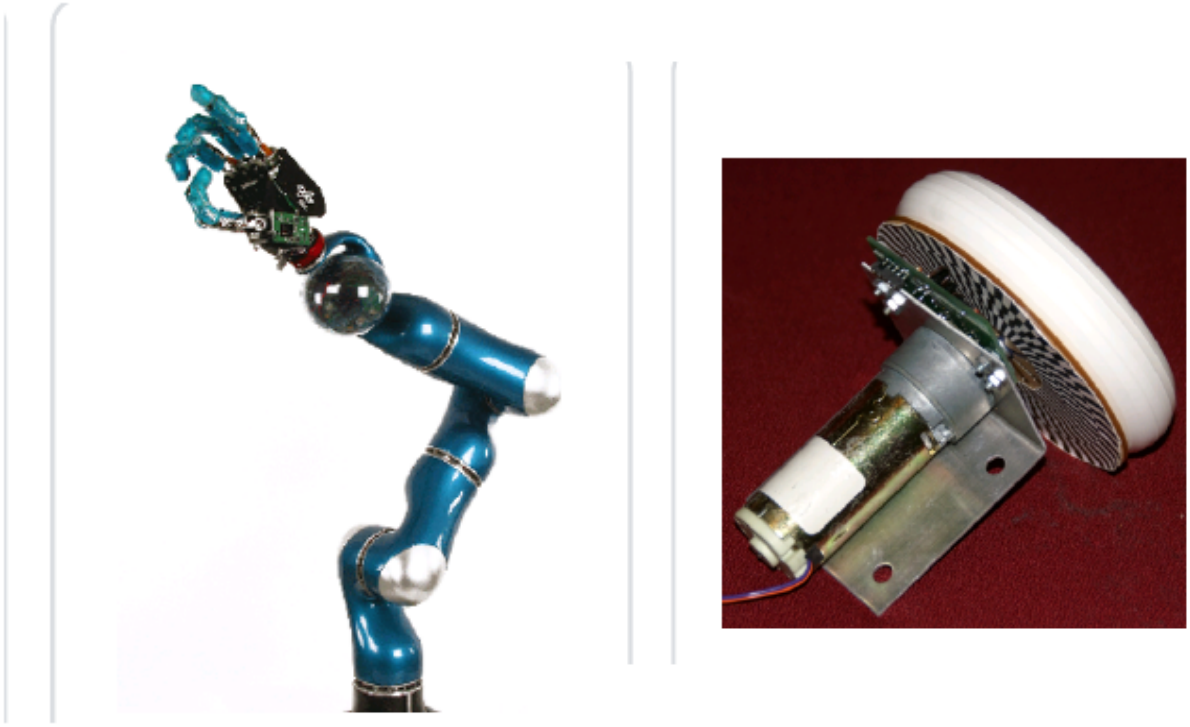
Campionando entrambi i fronti (di salita e di discesa) di entrambi i canali A e B, la logica di conteggio digitale (spesso integrata in moduli dedicati FPGA o microcontrollori) genera **4 conteggi per ciascuna fenditura fisica** del reticolo. Un disco ottico commerciale da 500 CPR (*counts per revolution* o fenditure per giro) eroga quindi una risoluzione effettiva di:

$$N = 500 \times 4 = 2000 \text{ impulsi/giro}$$

corrispondente a una risoluzione angolare di:

$$\Delta\theta = \frac{2\pi}{2000} \approx 0.00314 \text{ rad} \approx 0.18^\circ$$

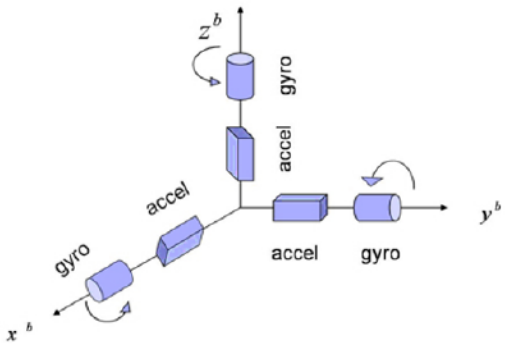
Gli encoder incrementali integrano inoltre un terzo canale, denominato **Canale di Indice** o **Index (**) (indicato con la traccia nello schema). Questo canale possiede un'unica fenditura radiale e genera un solo impulso isolato a ogni giro completo (360°). Serve come punto di zero assoluto (*homing*) per azzerare i registri di conteggio e identificare la posizione iniziale assoluta del meccanismo.



Gli encoder trovano applicazione sistematica sia nei giunti rotoidali dei manipolatori robotici per la cinematica diretta, sia calettati sull'albero dei motori a corrente continua nelle ruote di trazione dei robot mobili per calcolare l'odometria differenziale.

Unità Di Misura Inerziale (IMU)

L'**Inertial Measurement Unit (IMU)** è un apparato propriocettivo a stato solido che sfrutta la combinazione di accelerometri e giroscopi per stimare la velocità angolare, l'accelerazione lineare e, mediante integrazione temporale, l'orientamento, la velocità traslazionale e la posizione relativa del veicolo mobile.



Inertial Measurement Unit
3 accelerometers, 3 gyroscopes

Una configurazione a 6 gradi di libertà (6-DoF) standard allinea 3 accelerometri lineari ortogonali lungo gli assi del sistema di riferimento solidale al corpo del robot (*body frame* b, individuato dalla terna x^b, y^b, z^b) e 3 giroscopi angolari disposti attorno agli stessi assi:

- **Giroscopi**: misurano i tassi di rotazione angolare istantanei $\underset{\sim}{\omega} = [\omega_x, \omega_y, \omega_z]^T$ (rollio, beccheggio, imbardata).
- **Accelerometri**: misurano l'accelerazione specifica (la somma dell'accelerazione cinematica lineare $\mathbf{a}^b$ e della reazione alla gravità terrestre $\mathbf{g}$).

La Compensazione Del Vettore Di Gravità E Il Problema Del Drift

Per ricavare la reale accelerazione di movimento del veicolo mobile rispetto alla terna fissa inerziale nel mondo, il vettore di gravità $\mathbf{g}$ deve essere opportunamente proiettato nella terna corpo e sottratto dalle letture degli accelerometri:

$$\mathbf{a}^b_{\text{moto}} = \mathbf{a}^b_{\text{misurata}} - \underset{w}{\overset{b}{R}}\mathbf{g}^w$$

dove $\underset{w}{\overset{b}{R}}$ è la matrice di rotazione che descrive l'orientamento del robot rispetto al mondo, ricavata integrando le velocità angolari fornite dai giroscopi.

> ⚠ **Il Problema Del Drift E L'Errore Quadratico ($O(t^2)$)**
>
> Come evidenziato nelle slide della lezione, le IMU presentano una forte vulnerabilità al drift:
>
> 1. Un errore di stima nell'orientamento del giroscopio compromette l'orientamento rispetto alla verticale, portando a una sottrazione errata del vettore di gravità $\mathbf{g}$.
> 2. Poiché i dati dell'accelerometro devono essere **integrati due volte** per ottenere la posizione, qualsiasi componente residua costante o spuria di accelerazione genera un **errore quadratico sulla posizione** ($O(t^2)$).
>
> Dopo un periodo prolungato di funzionamento tutte le IMU derivano: per annullare l'errore è indispensabile accoppiarle con riferimenti esterni (GPS, laser o telecamere).

> 🖉 **Approfondimento Teorico**

Per la derivazione analitica formale dell'accoppiamento gravità-orientamento (che genera una deriva fino a $O(t^3)$), l'esempio numerico quantitativo (18metri di errore dopo $60\ \text{s}$) e il principio fisico dell'effetto Sagnac nei giroscopi a fibra ottica FOG, si veda zz. Argomenti extra: Derivazione Analitica Del Drift Quadratico Nelle IMU.



Nelle applicazioni critiche (aerospaziale, veicoli a guida autonoma) si impiegano sensori ad altissima precisione come i giroscopi a fibra ottica (**FOG**, *Fiber Optic Gyro*, quale il modulo KVH 1750 illustrato), capaci di ridurre la deriva a frazioni minime di grado all'ora.

Sensori Di Distanza A Tempo Di Volo: Laser Scanner E LiDAR

I sensori attivi di distanza a fascio ottico basano il proprio funzionamento sul principio del **Tempo di Volo** (**ToF**, *Time-of-Flight*). Un emettitore a diodo laser emette un impulso luminoso collimato a una frequenza prefissata; l'impulso viaggia nell'aria alla velocità della luce $c \approx 3 \times 10^{8}\ \text{m/s}$, colpisce un ostacolo riflettente e torna verso il fotorilevatore a valanga (APD) del sensore.

La distanza radiale d dall'ostacolo viene ricavata misurando l'intervallo temporale di andata e ritorno Δt:

$$d = \frac{c \cdot \Delta t}{2}$$

Il fattore 2 a denominatore tiene conto del tragitto bidirezionale (andata e ritorno).

Dal Laser Scanner Planare 2D Al LiDAR 3D

- **Laser scanner 2D**: il raggio laser è deflesso su un unico piano orizzontale tramite uno specchio rotante motorizzato. Il sensore produce una scansione polare bidimensionale (θ) che interseca gli ostacoli lungo una singola "fetta" orizzontale dell'ambiente.
- **LiDAR 3D**: sfrutta un fascio multiplo di laser sovrapposti o un prisma rotante bidirezionale per coprire un intero **angolo solido** nello spazio tridimensionale, generando una densa **nuvola di punti** (*point cloud*) che descrive l'ambiente in coordinate volumetriche (X, Y, Z).

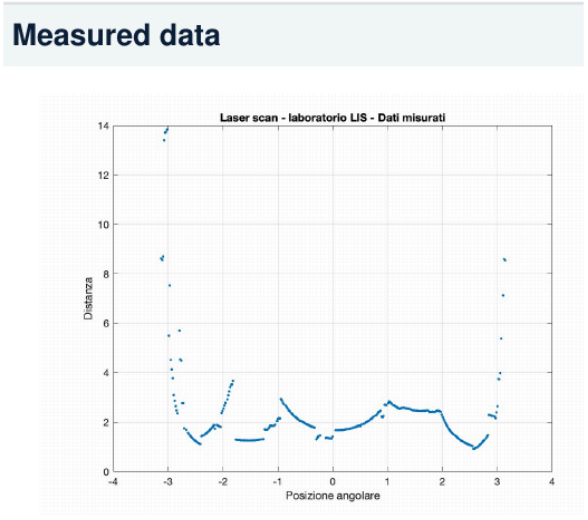


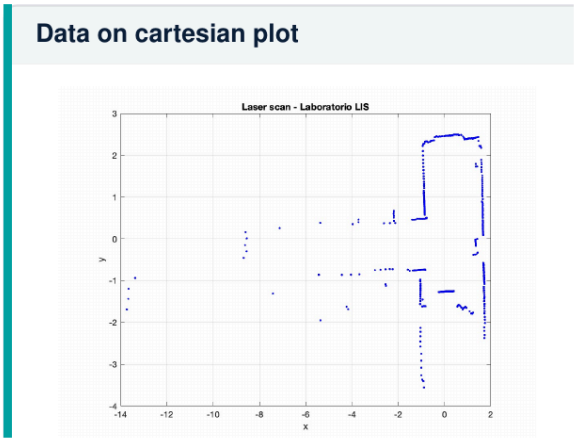
Un esempio industriale diffuso è il sensore **Velodyne Puck VLP-16**, equipaggiato a bordo del nostro robot Agrobot. Le specifiche operative del costruttore evidenziano le prestazioni tipiche di un sensore 3D compatto:

Parametro operativo	Valore di targa	Note ingegneristiche
Portata di misura (*Measurement range*)	Fino a 100 metri	Limitato dall'albedo della superficie e dalla luce solare
Accuratezza (*Accuracy*)	3 cm (tipica)	Errore sistematico residuo su bersaglio standard
Campo visivo verticale (*Vertical FOV*)	30°	Escursione angolare simmetrica da -15° a $+15^\circ$
Risoluzione angolare verticale	2°	Distanziamento fisso tra i 16 fasci laser verticali
Campo visivo orizzontale (*Azimuth FOV*)	360°	Rotazione continua su albero a cuscinetti
Risoluzione angolare orizzontale	$0.1^\circ - 0.4^\circ$	Inversamente proporzionale alla frequenza di scansione
Frequenza di rotazione (*Rotation rate*)	$5 - 20$ Hz	Frequenza di campionamento delle nuvole di punti (300.000 pt/s)

Elaborazione Dati Laser 2D: Dal Dominio Polare Al Piano Cartesiano

Per comprendere a fondo il funzionamento di un laser scanner 2D, analizziamo i dati reali acquisiti all'interno del laboratorio LIS (*Laboratorio di Sistemi Intelligenti*):





I dati grezzi forniti dal driver ROS del laser scanner non sono coppie di coordinate (x, y), ma consistono in un vettore unidimensionale di distanze ordinate:

$$\text{range} = [8621,\ 8559869,\ 13.39\ldots,\ 7.127,\ 8600,\ 8545]$$

I metadati associati al pacchetto definiscono la geometria della scansione angolare:

```
angle_min = -3.1241;        % Angolo iniziale in radianti (~ -pi)
angle_max = 3.1415;         % Angolo finale in radianti (~ +pi)
angle_increment = 0.0058172; % Risoluzione angolare (~ 0.33 gradi)
range_min = 0.02;           % Distanza minima affidabile (2 cm)
range_max = 60.0;           % Portata massima di fondo scala (60 m)
```

Nel grafico polare grezzo a sinistra le misure compaiono come una curva continua di distanza in funzione dell'angolo: per interpretare questa curva e individuare la stanza, dobbiamo trasformare ciascuna lettura polare $(_i, \theta_i)$ nella terna cartesiana del robot.

Risoluzione Geometrica: Conversione Polare-Cartesiana Per Il Laser 2D

L'angolo del raggio i-esimo viene determinato per indicizzazione lineare:

$$\theta_i = \theta_{\min} + i \cdot \Delta\theta$$

La proiezione nel piano cartesiano 2D solidale al sensore è ottenuta applicando la **trigonometria piana elementare**:

$$x_i = _i \cos(\theta_i)$$

$$y_i = _i \sin(\theta_i)$$

Nel caso in cui il sensore laser sia posizionato sul robot con un offset roto-traslazionale (x_L, y_L, θ_L) rispetto al centro di massa del veicolo, le coordinate del punto nella terna robot sono calcolate tramite la trasformazione rigida 2:

$$\begin{bmatrix} x_i \\ y_i \\ 1 \end{bmatrix} = \begin{bmatrix} \cos\theta_L & -\sin\theta_L & x_L \\ \sin\theta_L & \cos\theta_L & y_L \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_i \\ y_i \\ 1 \end{bmatrix}$$

Il grafico cartesiano a destra mostra il risultato di questa trasformazione: la complessa curva polare si ricompone istantaneamente nelle pareti rettilinee, negli spigoli, nelle porte e negli ostacoli fisici del laboratorio LIS.

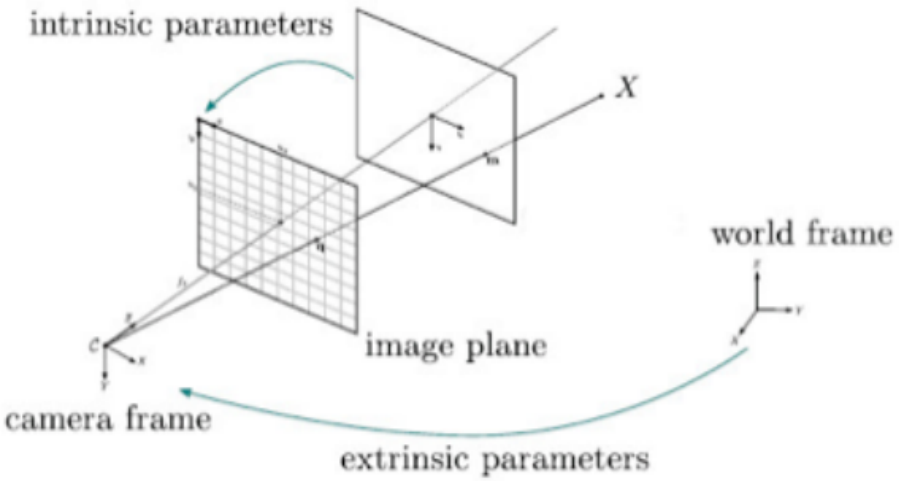
Visione Artificiale E Il Modello Prospettico

Se per il laser scanner 2D la proiezione è governata dalla trigonometria della distanza radiale ToF, il principio geometrico alla base delle **telecamere** è radicalmente diverso: la formazione dell'immagine segue la **proiezione prospettica** su un piano focale.

Come accennato nella panoramica iniziale, una telecamera stereoscopica utilizza due punti di vista differenti per stimare la profondità dell'ambiente tramite disparità orizzontale, in modo analogo alla vista binoculare umana. Al contrario, una telecamera **monoculare** (a singolo obiettivo) perde l'informazione diretta di profondità lungo il raggio proiettivo.

Approfondimento Teorico

Per la derivazione geometrica completa della triangolazione stereoscopica ($= \frac{f \cdot b}{}$) e l'analisi dell'incertezza quadratica di profondità in funzione della baseline b, si veda zz. Argomenti extra: Geometria Della Visione Stereoscopica.



Il Modello Pinhole Ideale

Nel modello a foro stenopeico (*pinhole camera model*), un punto tridimensionale dello spazio descritto nella terna della telecamera da $P_c = [x, y, z]^T$ viene proiettato sul piano immagine posto a distanza focale f lungo l'asse ottico z:

$$\begin{cases} u = f\dfrac{x}{z} \\ v = f\dfrac{y}{z} \end{cases}$$

Esprimendo le coordinate immagine in forma omogenea $p = [u, v, 1]^T$ con fattore di scala proiettivo $\lambda = z$, la relazione assume una rappresentazione lineare compatta:

$$p = \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \begin{bmatrix} f & 0 & 0 & 0 \\ 0 & f & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

Pixelization E Modello Prospettico Reale

Nei sensori digitali reali (CCD o CMOS), l'immagine è discretizzata in una matrice di fotodiodi (pixel). Emergono pertanto una serie di non-idealità costruttive fisiche:

1. **Spostamento del centro ottico** (u_0, v_0): l'origine del piano immagine discreto non si trova nel punto di intersezione con l'asse ottico (*principal point*), ma nell'angolo in alto a sinistra della matrice dei pixel.
2. **Dimensioni finite e pixel non quadrati (k_u, k_v)**: i fattori di conversione da metri a pixel lungo gli assi orizzontale e verticale possono differire ($k_u \neq k_v$), definendo le lunghezze focali espresse in pixel:

$$\alpha = fk_u, \quad \beta = fk_v$$

3. **Assi non perfettamente ortogonali (fk_c)**: i fotodiodi sulla griglia di silicio possono presentare un leggero angolo di skew, quantificato dal coefficiente $\gamma = fk_c$.

Introducendo queste grandezze, definiamo la **Matrice dei Parametri Intrinseci** K (di dimensione 3×3, detta anche matrice di calibrazione della telecamera):

$$K = \begin{bmatrix} \alpha & \gamma & u_0 \\ 0 & \beta & v_0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} fk_u & fk_c & u_0 \\ 0 & fk_v & v_0 \\ 0 & 0 & 1 \end{bmatrix}$$

In forma omogenea estesa $[\cdot|\cdot]$ di dimensione 3×4:

$$p = KP_c = \begin{bmatrix} fk_u & fk_c & u_0 & 0 \\ 0 & fk_v & v_0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

Trasformazione Dalle Coordinate Mondo Al Piano Immagine

I punti caratteristici dell'ambiente (*landmarks* o elementi della mappa) sono tipicamente espressi in un sistema di coordinate mondo fisso $P_w = [X_w, Y_w, Z_w]^T$. Per collegare il mondo all'immagine dobbiamo considerare la roto-traslazione rigida che descrive la posa della telecamera:

$$P_c = RP_w + t$$

dove:

- $R \in SO(3)$ è la matrice ortonormale di rotazione 3×3.
- $t \in \mathbb{R}^3$ è il vettore di traslazione 3×1.

La matrice a blocchi $[R|t]$ costituisce la **Matrice dei Parametri Estrinseci** (3×4).

Componendo parametri intrinseci ed estrinseci otteniamo il modello prospettico completo in coordinate omogenee:

$$p = K[R|t]P_w$$

In forma analitica non lineare, la proiezione su pixel (u, v) è descritta dalla funzione di misura h:

$$p = h(\mathbf{x}, P_w, k)$$

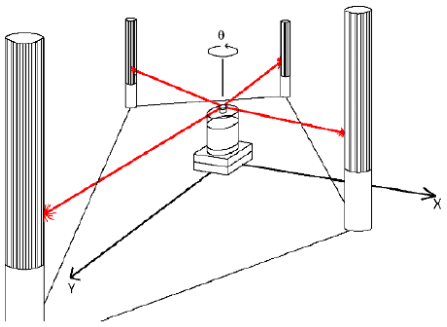
dove $\mathbf{x}$ rappresenta la posa corrente del robot/telecamera (da cui dipendono R e t), P_w è la posizione tridimensionale del landmark nella mappa e k è il vettore dei parametri intrinseci della telecamera ricavati tramite le procedure di calibrazione (ad es. con scacchiera di calibrazione secondo il metodo di Zhang).

> **Natura Proiettiva Della Telecamera Monoculare (Bearing-Only Sensor)**
>
> Nel caso monoculare, a causa della divisione per la coordinata proiettiva z, la scala assoluta dell'ambiente va persa:
>
> $$p = \frac{1}{z}[R|t]P_w$$
>
> Un oggetto piccolo vicino alla lente proietta esattamente lo stesso pattern di pixel di un oggetto grande situato a grande distanza. La telecamera monoculare agisce quindi come un puro **sensore di orientamento/direzione** (*bearing sensor*): per recuperare la scala metrica assoluta è indispensabile fondere la visione con l'odometria, con una IMU (Visual-Inertial Odometry) o ricorrere alla visione stereoscopica.

Sistemi Di Riferimento Basati Su Beacon E GNSS/GPS

La navigazione tramite punti cospicui noti (*beacons*) accompagna la storia dell'umanità sin dai primi viaggi marittimi ed esplorativi:

- **Beacon naturali**: astri (stella polare, sole), cime montuose o profili costieri.
- **Beacon artificiali**: fari marittimi lungo le coste, boe luminose e radiofari di navigazione aerea.



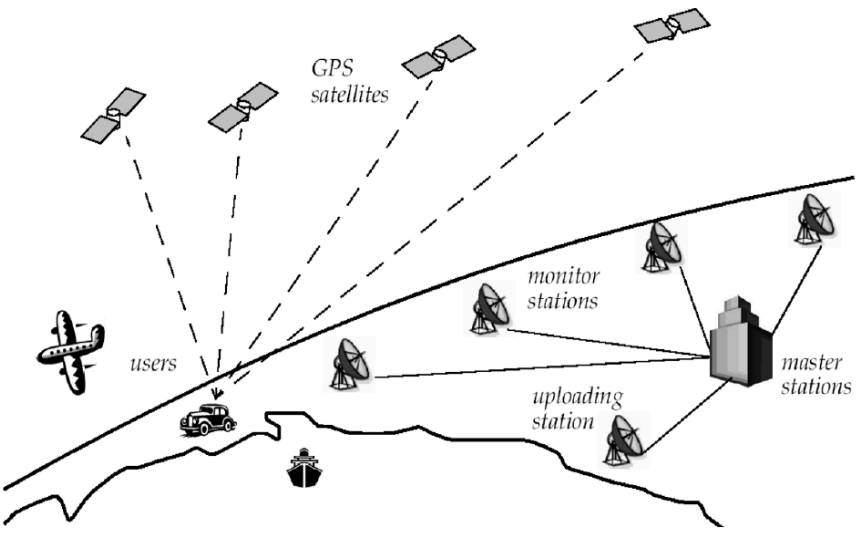
Nella robotica indoor sono stati sviluppati sistemi basati su colonne o fari riflettenti passivi posizionati sulle pareti o sul soffitto, interrogati da un emettitore rotante a bordo del robot. Misurando gli angoli azimutali $\theta_1, \theta_2, \ldots$ di ritorno verso i beacon di coordinate note, il robot risolve un problema di **triangolazione trigonometrica**.

> ⚠ **Limiti Strutturali Dei Beacon Indoor Artificiali**
>
> L'impiego di beacon dedicati per il posizionamento indoor presenta forti svantaggi applicativi:
>
> 1. **Costo di infrastruttura elevato**: richiede l'installazione, la taratura metrologica e la manutenzione di fari distribuiti in ogni ambiente in cui il robot deve operare.
> 2. **Rigidità e assenza di flessibilità**: qualsiasi modifica al layout architettonico dell'edificio o lo spostamento di un mobile che ostruisce la linea di vista diretta (*Line-of-Sight*) compromette l'intero sistema di localizzazione.

Per questo motivo, la ricerca moderna in robotica mobile ha progressivamente abbandonato i beacon fisici indoor a favore delle tecniche di **SLAM** (*Simultaneous Localization and Mapping*), in cui il robot mappa autonomamente le caratteristiche naturali dell'ambiente (*natural landmarks*).

Il Sistema Di Posizionamento Globale GNSS / GPS

In ambienti esterni (*outdoor*), il sistema di riferimento a beacon per eccellenza è la costellazione satellitare **GNSS** (*Global Navigation Satellite System*), di cui il **GPS** (sviluppato inizialmente dal Dipartimento della Difesa USA col nome NAVSTAR e reso accessibile per scopi civili) è l'implementazione storica di riferimento.



L'architettura del sistema GNSS si articola su tre segmenti operativi:

1. **Segmento Spaziale (*Space Segment*)**: costellazione di 24-32 satelliti orbitanti a quota MEO (*Medium Earth Orbit*, circa $20.180\ \text{km}$ di altitudine), distribuiti su 6 piani orbitali inclinati di 55° rispetto all'equatore. Ciascun satellite compie un'orbita completa in circa 12 ore siderali, garantendo che da qualunque punto aperto della superficie terrestre siano costantemente visibili almeno 6-8 satelliti.
2. **Segmento di Controllo (*Control Segment*)**: stazioni terrestri distribuite a livello globale suddivise in:
   - *Monitor Stations*: tracciano le orbite dei satelliti e monitorano la deriva degli orologi atomici di bordo.
   - *Master Station*: calcola le effemeridi orbitali precise e i parametri di correzione oraria.
   - *Uploading Stations*: trasmettono le effemeridi aggiornate ai satelliti per il broadcast agli utenti.
3. **Segmento Utente (*User Segment*)**: i ricevitori a terra, su imbarcazioni, aerei o robot mobili.

Il principio di localizzazione si basa sulla **trilaterazione a tempo di volo**: ciascun satellite trasmette in continuo un segnale radio contenente le proprie coordinate orbitali e il timestamp di trasmissione generato da orologi atomici al cesio o rubidio.

Il ricevitore a terra calcola la distanza (*pseudorange*) rispetto a ciascun satellite visibile. Poiché l'orologio locale del ricevitore presenta un offset temporale incognito $t_{\text{ricevitore}}$, le incognite da stimare sono **quattro** (le coordinate 3D x, y, z e il bias dell'orologio locale), rendendo indispensabile ricevere il segnale da un minimo di **4 satelliti contemporaneamente visibili**.

> **Approfondimento Teorico**
>
> Per la formulazione analitica dettagliata dell'equazione dello pseudorange a quattro incognite e il principio di funzionamento dell'RTK (Real Time Kinematic) basato sul tracciamento di fase portante a livello centimetrico, si veda zz. Argomenti extra: Modellistica Dello Pseudorange GNSS/GPS.

> **Limiti E Soluzioni: Da Standalone a RTK**
>
> Il GPS standard standalone presenta un'accuratezza tipica di $3 - 5\ \text{metri}$. Nelle gallerie, sotto folte chiome arboree e soprattutto negli ambienti indoor il segnale GPS subisce un'attenuazione totale, risultando inutilizzabile.
>
> Per ovviare all'incertezza metrica in campo aperto (come nel nostro Agrobot in vigneto), si impiega la tecnica **RTK (Real Time Kinematic)** con stazione base a terra, portando l'accuratezza di posizionamento alla soglia di $1\ \text{cm}$.

Appendice: Caratteristiche Metrologiche E Sensori Fisici

Per selezionare e modellare correttamente i sensori all'interno degli algoritmi di fusione dati e filtraggio probabilistico, è fondamentale padroneggiare il vocabolario metrologico standard dei sensori (come codificato in *Siegwart et al.*).

Proprietà Metrologiche Fondamentali

1. **Range (Campo di misura)**: valori minimo e massimo della grandezza fisica di ingresso che il sensore è in grado di rilevare affidabilmente ($x_{\min}$ e $x_{\max}$).
2. **Dynamic Range (Range dinamico)**: rapporto tra il limite superiore e il limite inferiore del range di misura. Viene comunemente espresso in decibel [dB]
   - Per grandezze di potenza:

$$\text{Dynamic Range} = 10 \log_{10} \left(\frac{P_{\max}}{P_{\min}} \right) \quad [\text{dB}]$$

   - Per grandezze di tensione o campo:

$$\text{Dynamic Range} = 20 \log_{10} \left(\frac{x_{\max}}{x_{\min}} \right) \quad [\text{dB}]$$

3. **Risoluzione (*Resolution*)**: la minima variazione dell'ingresso misurabile che produce una variazione rilevabile nell'uscita del sensore. Nei sensori digitali è vincolata dal numero di bit del convertitore analogico-digitale (ADC): per un convertitore a b bit su fondo scala FS, la risoluzione teorica vale $\Delta = \frac{FS}{2^b}$.
4. **Linearità (*Linearity*)**: la conformità tra la curva di risposta reale del sensore e una retta ideale $y = K \cdot x$. La non-linearità è misurata come scostamento percentuale massimo rispetto al fondo scala.
5. **Banda passante e Tempo di risposta**: velocità con cui il sensore reagisce a variazioni a gradino dell'ingresso. Limita la massima frequenza a cui il sensore può essere campionato senza introdurre sfasamenti o attenuazioni.
6. **Sensibilità (*Sensitivity*)**: rapporto tra la variazione dell'uscita del sensore e la corrispondente variazione della grandezza di ingresso:

$$S = \frac{\Delta y}{\Delta x}$$

7. **Sensibilità incrociata (*Cross-Sensitivity*)**: sensibilità del sensore a parametri ambientali non desiderati (ad esempio, la variazione dell'uscita di un accelerometro indotta da uno shock termico anziché da una reale accelerazione).
8. **Errore sistematico vs Errore stocastico**:
   - **Errore sistematico (deterministico)**: provocato da fattori modellabili e riproducibili (errori di scala, offset di zero, disallineamento geometrico). Può essere eliminato o fortemente mitigato mediante calibrazione.
   - **Errore casuale (stocastico)**: fluttuazioni imprevedibili generate da rumore termico elettronico, quantizzazione e disturbi ambientali. Non può essere eliminato deterministicamente, ma deve essere descritto mediante modelli statistici (distribuzioni gaussiane con matrice di covarianza Σ).
9. **Accuratezza vs Precisione**:
   - **Accuratezza (*Accuracy*)**: grado di concordanza tra il valore medio misurato e il valore vero standard della grandezza:

$$\text{Accuratezza} = 1 - \frac{|m - v|}{v}$$

   - **Precisione (*Precision*)**: grado di ripetibilità delle misure indipendentemente dal valore vero; è inversamente proporzionale alla deviazione standard σ:

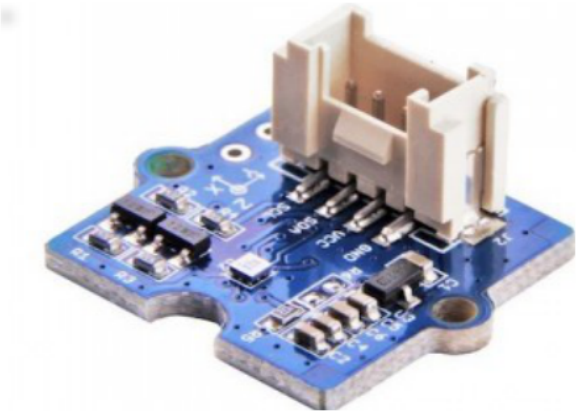
$$\text{Precisione} = \frac{\text{range}}{\sigma}$$

> **Approfondimento Teorico**
>
> Per la quantificazione analitica del dynamic range in decibel di potenza/tensione e per la modellazione del rumore di quantizzazione nei convertitori analogico-digitali (ADC a b bit con varianza $\sigma_q^2 = \frac{(\Delta)^2}{12}$), si veda zz. Argomenti extra: Parametri Metrologici Avanzati E Fisica Dei Sensori.

Principi Fisici Dei Sensori Dedicati

Bussola Magnetica (Magnetometro)

Il principio della bussola magnetica risale al 2000 a.C. in Cina, con la sospensione di magnetite naturale su fili di seta. Nei moderni robot mobili la bussola è implementata tramite **magnetometri a stato solido** (a effetto Hall, magnetoresistivi anisotropi AMR o fluxgate):



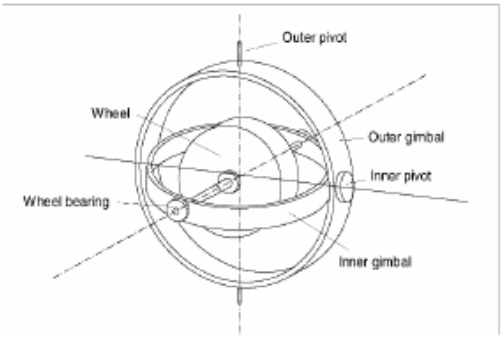
I magnetometri misurano le componenti vettoriali del campo magnetico locale. Poiché il campo magnetico terrestre ha un'intensità debole (circa $30 - 50$), le letture sono soggette a distorsioni dovute a componenti metalliche o campi parassiti a bordo del veicolo (distorsioni *hard-iron* e *soft-iron*), che richiedono un'apposita procedura di calibrazione.

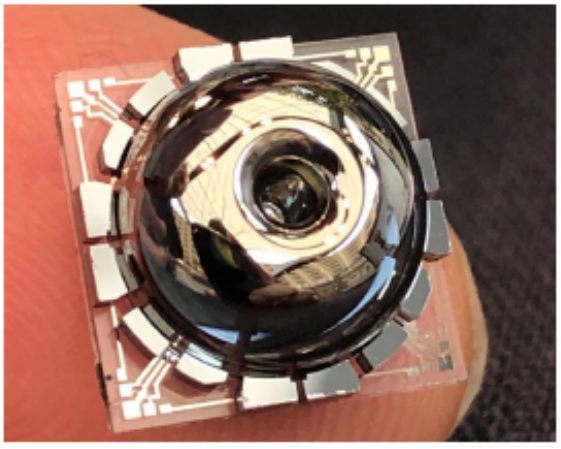
Approfondimento Teorico

Per la modellizzazione matematica delle distorsioni hard-iron (offset costante) e soft-iron (deformazione ellittica) e per l'algoritmo di calibrazione ad ellisse, si veda zz. Argomenti extra: Distorsioni Hard-Iron E Soft-Iron Nelle Bussole Magnetiche.

Giroscopi

I giroscopi sono sensori di orientamento che preservano o misurano la rotazione angolare in relazione a una terna di riferimento fissa. Si distinguono due categorie tecnologiche fondamentali:





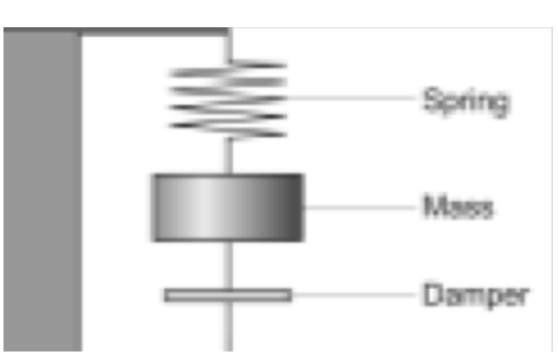
- **Giroscopio meccanico a sospensione cardanica**: sfrutta la conservazione del momento angolare di un rotore massiccio rotante montato su giunti cardanici (*gimbals*), mantenendo l'orientamento assoluto nello spazio inerziale.
- **Micro-giroscopi MEMS risonanti**: integrati su silicio (quali le strutture a campana semisferica risonante), sfruttano la deflessione generata dalla **forza di Coriolis** su una massa microscopica mantenuta in oscillazione per misurare la velocità angolare istantanea.

Approfondimento Teorico

Per la derivazione analitica della forza di Coriolis ($\mathbf{F}_c = -2m(\boldsymbol{\omega} \times \mathbf{v})$) nelle strutture oscillanti dei micro-giroscopi MEMS, si veda zz. Argomenti extra: Meccanica Dei Micro-Giroscopi MEMS E Forza Di Coriolis.

Accelerometri

L'accelerometro misura tutte le forze esterne che agiscono su di esso, inclusa la gravità. Dal punto di vista fisico, un accelerometro si comporta come un sistema **molla-massa-smorzatore**:



Una massa sismica di prova (*proof mass*) m è vincolata a un telaio rigido tramite una molla di costante elastica k e uno smorzatore con coefficiente di attrito c.

L'equazione dinamica che governa il sistema è:

$$_{\text{applied}} = _{\text{inerzia}} + _{\text{smorzamento}} + _{\text{molla}} = m\ddot{x} + c\dot{x} + kx$$

A regime permanente (*steady state*), con derivate relative nulle ($\dot{x} = 0, \ddot{x} = 0$), l'accelerazione $\ddot{x}$ impressa al corpo risulta direttamente proporzionale alla deflessione x della massa sismica:

$$\ddot{x} = \frac{k}{m}x$$

Riepilogo Concettuale E Collegamenti

In questo capitolo abbiamo completato il panorama percettivo della robotica mobile:

- L'odometria da encoder e i modelli cinematici puri sono insufficienti per operare sul lungo termine a causa della divergenza dell'errore di integrazione.
- La IMU fornisce accelerazioni e velocità angolari ad altissima frequenza, ma soffre di deriva quadratica $O(t^2)$ sulla posizione dovuta all'integrazione duplice e all'accoppiamento con la gravità.
- Il Laser scanner ToF 2D misura distanze planari trasformate in coordinate cartesiane tramite trigonometria elementare, mentre il LiDAR 3D (come il Velodyne Puck) genera nuvole di punti volumetriche.
- La visione artificiale opera invece tramite proiezione prospettica su piano immagine (modello pinhole reale con parametri intrinseci ed estrinseci). La telecamera monoculare è un sensore di puro orientamento (*bearing*) che richiede visione stereo o integrazione con altri sensori per recuperare la scala metrica assoluta.
- Nei prossimi capitoli uniremo i modelli di movimento probabilistici con i modelli di misura di questi sensori all'interno degli algoritmi di filtraggio bayesiano (Filtro di Kalman e Particle Filter) per risolvere il problema fondamentale della localizzazione autonoma.

3. Rappresentazione dell'Ambiente Ed Estrazione Di Feature

La capacità di operare autonomamente nello spazio richiede che un robot mobile possieda una rappresentazione strutturata del mondo circostante (**mappa**) e sia in grado di estrarre dai segnali sensoriali grezzi delle entità geometriche o semantiche compatte, stabili e informative (**feature**).

Questo capitolo affronta in modo organico la transizione dai dati grezzi dei sensori di distanza (*range data*, tipicamente da scansioni LiDAR 2D) alla costruzione di modelli ambientali, analizzando:

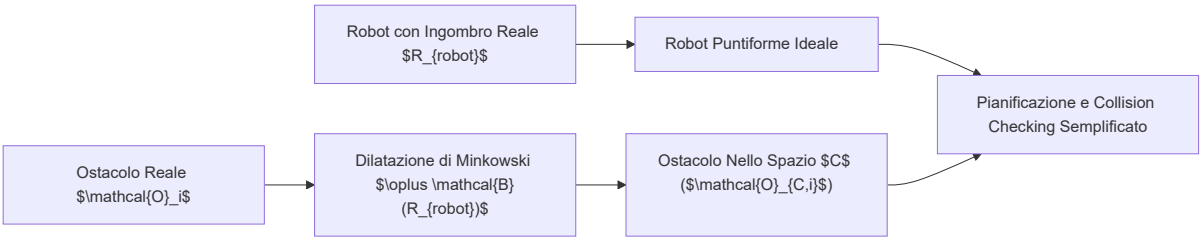
1. L'inquadramento cinematico e geometrico della percezione ambientale e il concetto di spazio delle configurazioni;
2. La tassonomia formale delle rappresentazioni cartografiche (continue, basate su feature, decomposizioni discrete e grafi topologici);
3. La formulazione matematica del **line fitting** in coordinate polari e forma normale di Hesse;
4. Gli algoritmi fondamentali di segmentazione geometrica: **Split-and-Merge**, **Trasformata di Hough** e **RANSAC**.

Inquadramento Geometrico E Spazio Delle Configurazioni

Per localizzarsi, pianificare traiettorie ed evitare collisioni, il robot deve relazionare le proprie misure sensoriali con un modello geometrico dell'ambiente. A livello teorico e di pianificazione di base, il robot viene spesso trattato come un **punto materiale** ideale privo di ingombro nello spazio bidimensionale:

$$\mathbf{p} = \begin{bmatrix} x \\ y \end{bmatrix} \in \mathbb{R}^2$$

Nella realtà operativa, tuttavia, ogni veicolo terrestre possiede una sagoma geometrica finita, definita da un poligono o da un raggio di ingombro r_b.



Dallo Spazio Fisico allo Spazio delle Configurazioni (C-Space)

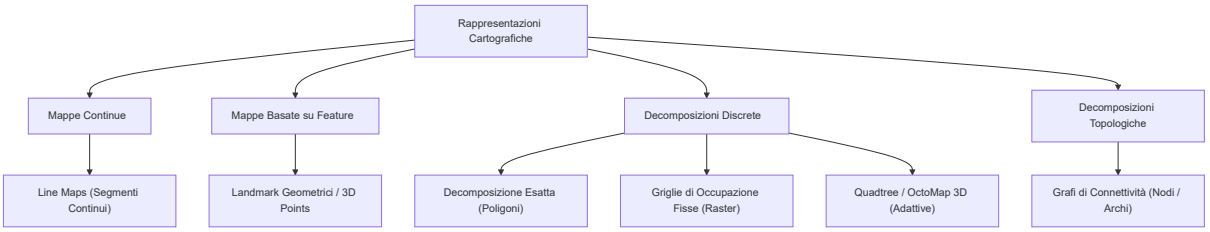
Per preservare la semplicità degli algoritmi di navigazione e collision checking considerando il robot puntiforme, si applica la **somma di Minkowski** ($\oplus$) tra la geometria di ciascun ostacolo $\mathcal{O}_i$ e la geometria del robot $\mathcal{A}$ invertita rispetto all'origine:

$$\mathcal{O}_{C,i} = \mathcal{O}_i \oplus (-\mathcal{A})$$

Nel caso conservativo in cui il robot sia inscritto in un cerchio di raggio r_b, l'operazione coincide con una **dilatazione isotropa degli ostacoli** di una quantità pari al raggio r_b. In questo modo, qualsiasi traiettoria percorsa dal robot puntiforme che non intersechi gli ostacoli dilatati garantisce l'assoluta assenza di collisioni nel mondo reale.

Tassonomia Delle Rappresentazioni Cartografiche

La scelta del formalismo con cui rappresentare l'ambiente dipende strettamente dal compito assegnato, dalle capacità computazionali di bordo e dai sensori disponibili. La letteratura di riferimento suddivide le mappe in quattro macro-categorie:



Mappe Continue a Linee (Line-Based Continuous Maps)

Nelle mappe continue, gli elementi strutturali dell'ambiente (muri, pareti, divisori interni) vengono descritti mediante insiemi di primitive geometriche definite su coordinate reali $\mathbb{R}^2$:

$$\mathcal{M}_{line} = \mathbf{l}_1, \mathbf{l}_2, \ldots, \mathbf{l}_N,$$

Ciascun segmento di retta $\mathbf{l}_j$ è caratterizzato dagli estremi cartesiani $((x_{s,j}, y_{s,j}), (x_{e,j}, y_{e,j}))$ oppure dalla forma normale polare con intervallo di estensione.

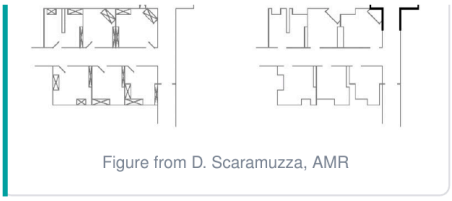


Figure from D. Scaramuzza, AMR

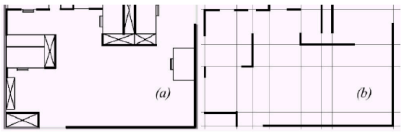


Figure from D. Scaramuzza, AMR

Le proprietà salienti di questa rappresentazione includono:

- **Risoluzione infinita**: non vi è alcuna quantizzazione spaziale indotta da una griglia discreta;
- **Efficienza di memoria**: l'occupazione di memoria non scala con la superficie totale dell'ambiente, bensì è strettamente proporzionale alla **densità degli ostacoli** (numero di muri N);
- **Assunzione di Mondo Chiuso (*Closed-World Assumption*)**: le linee tracciano perimetri continui che racchiudono l'area esplorata; ciò che giace all'esterno del poligono di confini è implicitamente inaccessibile.

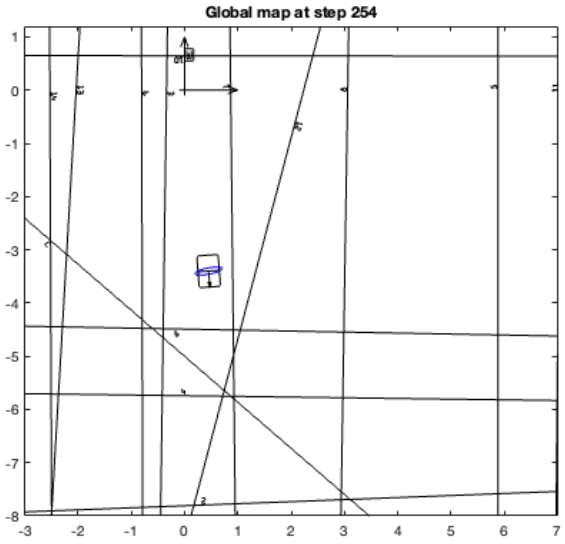
> ⚠ **Limiti delle Mappe Continue a Linee**
>
> Sebbene siano eccellenti per la localizzazione precisa di robot dotati di LiDAR planari in ambienti strutturati (uffici, corridoi), le *line maps* risultano inadatte per la navigazione in ambienti non strutturati, vegetazione o terreni aperti, e non consentono di rappresentare ostacoli di forma irregolare o parzialmente trasparenti.

Mappe Basate Su Feature (Landmark-Based Maps)

Invece di descrivere l'intero contorno fisico dell'ambiente, le mappe basate su feature memorizzano unicamente la posizione e i parametri descrittivi di punti salienti o elementi distintivi (*landmarks*):

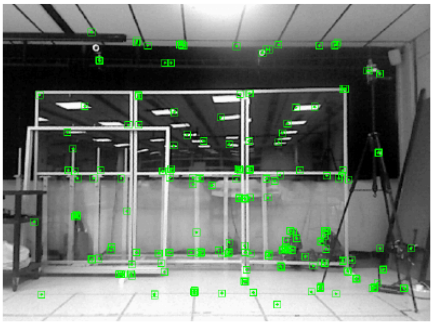
$$\mathcal{M}_{feat} = \mathbf{f}_1, \mathbf{f}_2, \ldots, \mathbf{f}_M, \quad \mathbf{f}_j = \begin{bmatrix} \mathbf{p}_j \\ \mathbf{d}_j \end{bmatrix}$$

dove $\mathbf{p}_j$ rappresenta la coordinata spaziale (2D o 3D) e $\mathbf{d}_j$ è un descrittore associato (geometrico o fotometrico).



Tra i sistemi più rilevanti basati su questo paradigma troviamo i framework di Visual SLAM come **ORB-SLAM**:

1. **Mappa Sparsa di Keypoint 3D**: l'ambiente è memorizzato come una costellazione di punti tridimensionali triangolati dai fotogrammi chiave;
2. **Grafo delle Pose (Pose Graph / Co-visibility Graph)**: i nodi rappresentano le pose stimate della telecamera nel tempo, mentre gli archi modellano i vincoli di co-visibilità di punti comuni e le trasformazioni relative.



ORB-SLAM maps

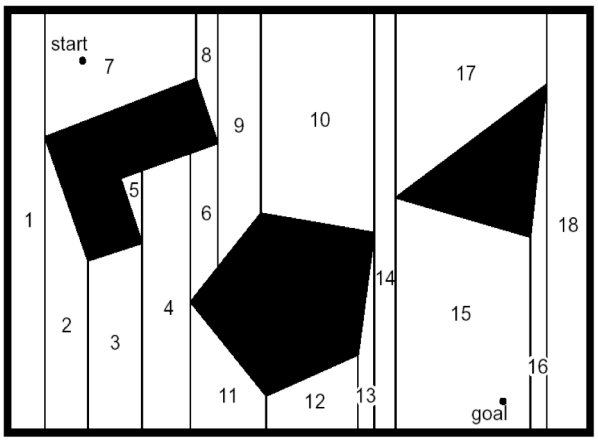


Decomposizioni Discrete: Esatta, Fissa E Adattiva

Quando l'obiettivo primario è il motion planning e la verifica istantanea delle collisioni, l'ambiente continuo viene partizionato in regioni elementari (*celle*).

Decomposizione Esatta (Exact Cell Decomposition)

L'ambiente libero viene suddiviso geometricamente in un insieme di celle poligonali convesse contigue $\mathcal{C}_1, \ldots, \mathcal{C}_K$, tracciando segmenti verticali o estensioni dei vertici degli ostacoli (*trapezoidal decomposition*).



- All'interno di ciascuna cella convessa, due punti qualsiasi possono essere congiunti da un segmento rettilineo che non tocca alcun ostacolo;
- Il motion planning si riduce alla ricerca su grafo tra celle adiacenti (attraversando i punti medi dei segmenti di confine) partendo dalla cella contenente lo `Start` fino a raggiungere quella del `Goal`.

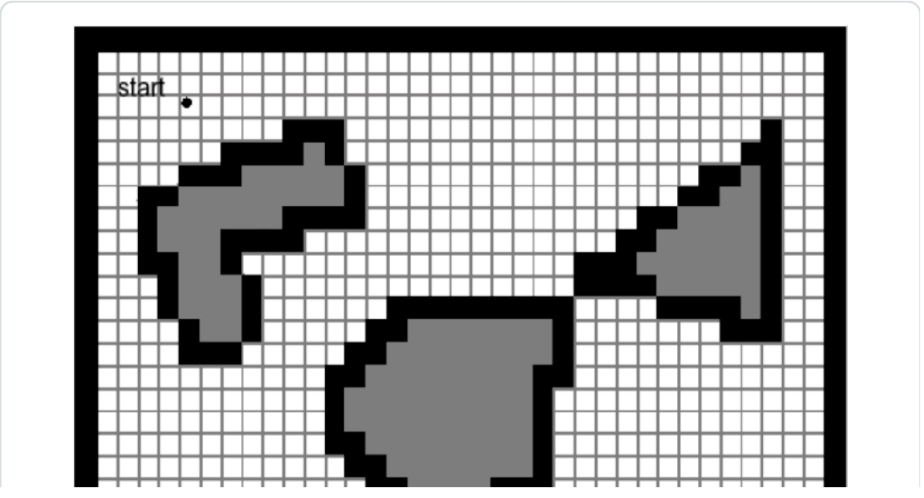
Decomposizione a Griglia Fissa (Occupancy Grid Maps)

Proposta originariamente da Moravec ed Elfes, la decomposizione a griglia fissa tessella lo spazio bidimensionale in una matrice regolare di celle quadrate di lato fisso Δ (tipicamente da $1\ \text{cm}$ a $10\ \text{cm}$):

$$\mathbf{m} = \{m_i\}, \quad m_i \in [0, 1]$$

Ogni cella m_i conserva la probabilità stocastica di occupazione $P(m_i = 1)$:

- $P(m_i) \approx 1$ (nero): cella occupata da un ostacolo;
- $P(m_i) \approx 0$ (bianco): spazio libero esplorato;
- $P(m_i) = 0.5$ (grigio): cella inesplorata o stato sconosciuto.

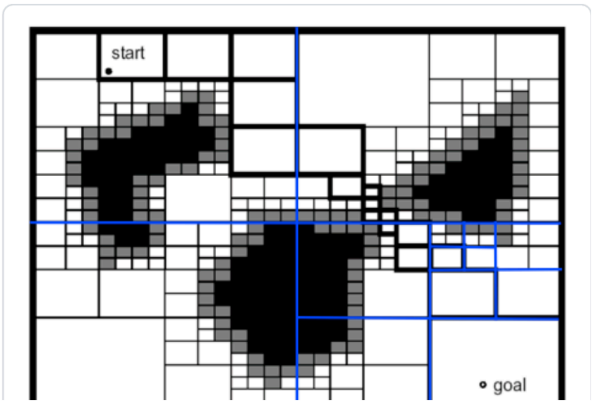


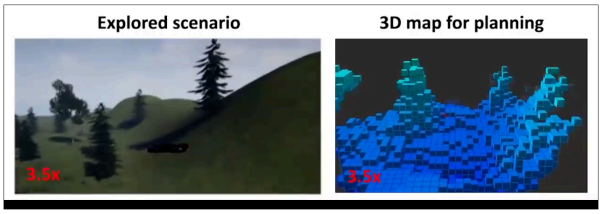
- **Vantaggi**: modellazione naturale dell'incertezza dei sensori, aggiornamento ricorsivo bayesiano immediato, indipendenza dalla forma geometrica degli ostacoli;
- **Svantaggi**: l'occupazione di memoria scala con l'area complessiva dell'ambiente ($O(L^2)$ in 2D e $O(L^3)$ in 3D), indipendentemente dal fatto che l'ambiente sia prevalentemente vuoto.

Decomposizione Adattiva: Quadtree E OctoMap

Per mitigare l'esplosione dimensionale delle griglie fisse preservando l'accuratezza sui bordi degli ostacoli, si ricorre alla suddivisione gerarchica ad albero:

- In 2D si utilizza un **Quadtree**: se una regione non è omogeneamente libera o occupata, viene divisa ricorsivamente in 4 quadranti (NW, NE, SW, SE); le aree uniformi rimangono memorizzate come blocchi grandi;
- In 3D si estende il concetto a un **Octree**, dando origine a sistemi standard come **OctoMap**.





Caratteristiche di OctoMap in 3D

OctoMap permette la modellazione tridimensionale a voxel multirisoluzione, tracciando esplicitamente sia lo spazio occupato che lo spazio libero (essenziale per droni aerei e bracci robotici mobili). L'occupazione di memoria è compressa grazie al raggruppamento dei nodi foglia omogenei (*node pruning*).

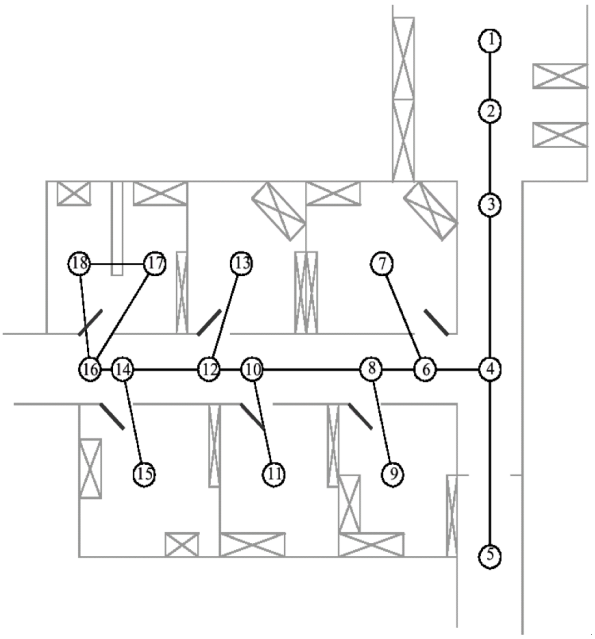
Mappe Topologiche (Topological Maps)

Una mappa topologica astrae completamente la metrica geometrica, descrivendo l'ambiente come un grafo non orientato:

$$\mathcal{G} = (V, E)$$

dove:

- I nodi V rappresentano luoghi fisici o aree salienti (es. *Stanza A*, *Incrocio Corridoio*, *Atrio*);
- Gli archi E modellano la connettività e la navigabilità diretta tra i luoghi (porte, corridoi percorribili).



Ruolo delle Mappe Topologiche nel Corso

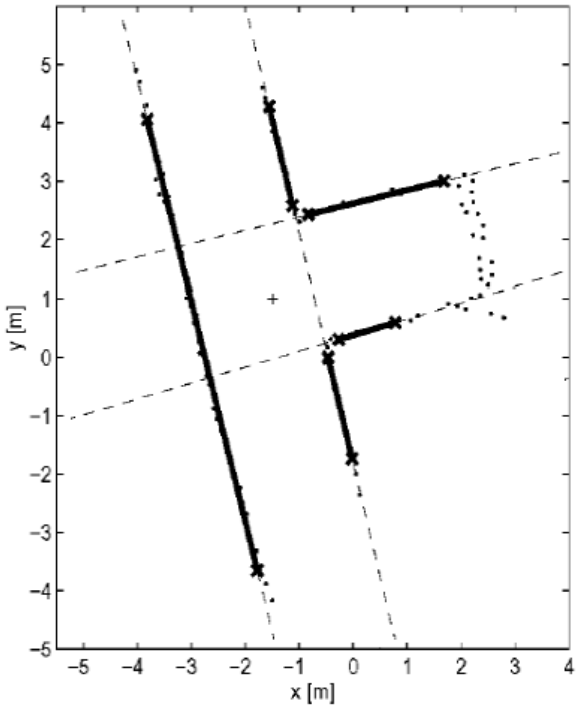
Come chiarito a lezione, **le mappe topologiche non vengono utilizzate per la localizzazione fine metrica del robot**. Sebbene risultino estremamente potenti per la pianificazione globale di percorsi ad alto livello (*symbolic path planning*), esse non contengono informazioni metriche di dettaglio: un robot non può stimare con precisione millimetrica la propria terna (x, y, θ) basandosi esclusivamente su nodi e archi topologici astratti.

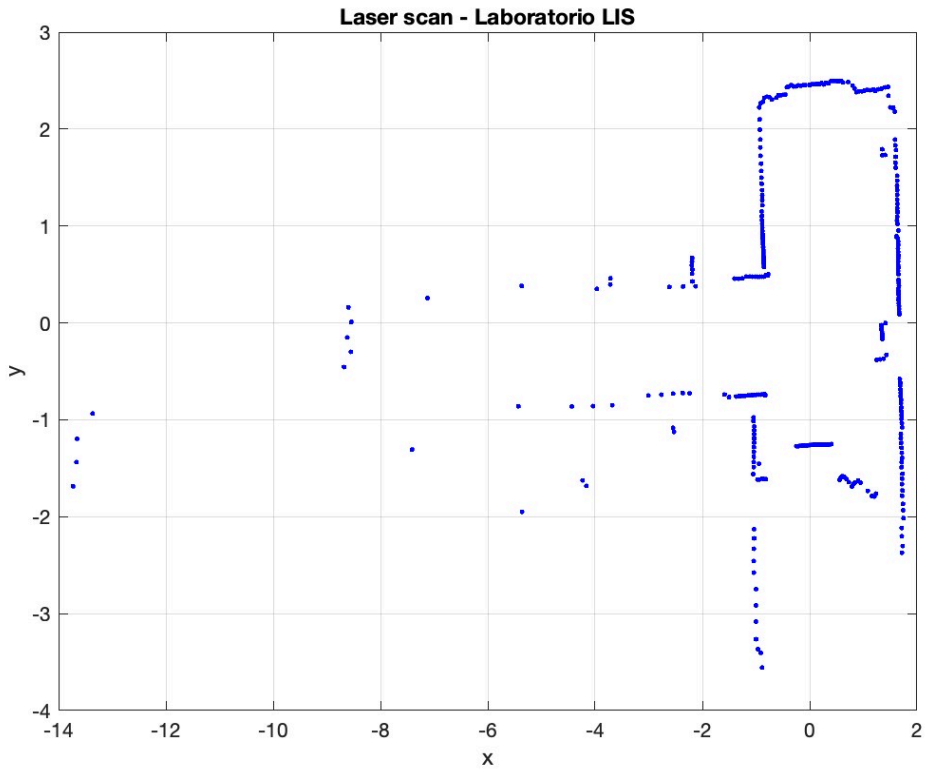
Estrazione Di Linee Da Dati Di Distanza (Line Extraction from Range Data)

La maggior parte dei robot mobili per interni monta sensori di distanza laser (LiDAR planari) che forniscono sequenze di misure polari grezze:

$$\mathcal{P}_w = (\rho_i, \theta_i)_{i=1}^{N}$$

Il problema dell'estrazione di feature consiste nel convertire questa nuvola disordinata di campioni rumorosi in un insieme compatto di primitive geometriche (segmenti di retta).





Il problema si articola formalmente in due sotto-problemi fondamentali:

1. **Data Segmentation**: stabilire quante linee sono presenti e quali punti appartengono a ciascuna linea;
2. **Line Fitting**: dato un sottoinsieme di punti associati a una linea, calcolare i parametri geometrici ottimi della retta minimizzando l'errore di stima.

Modello Geometrico Di Retta E Line Fitting

Perché la Forma Normale Di Hesse in Coordinate Polari?

Nel piano cartesiano standard, una retta viene comunemente espressa come:

$$y = mx + q$$

Questa parametrizzazione presenta una criticità fatale per i robot mobili: quando una retta o una parete è parallela all'asse y (verticale), il coefficiente angolare diverge ($m \to \infty$), provocando instabilità numerica e singolarità negli algoritmi di ottimizzazione.

Per evitare qualsiasi singolarità e mantenere la robustezza rispetto all'origine del robot, si adotta la **forma normale di Hesse** (parametrizzazione polare della retta):

$$x \cos\alpha + y \sin\alpha - r = 0$$

⎽ine model in polar coordinates

$$x\cos(\alpha) + y\sin(\alpha) - r = 0$$

Ξach data point is given by the pair $x_i = (\rho_i, \theta_i)$

\oisy measurements

I parametri geometrici hanno un significato fisico inequivocabile:

- ≥ 0: distanza ortogonale minima dall'origine degli assi (la posizione del sensore) alla retta;
- $\in [-\pi, \pi]$: angolo della normale orientata alla retta condotta dall'origine.

Dato un punto generico $P_i = (x_i, y_i)$, la distanza ortogonale con segno (residuo) tra il punto e la retta è:

$$= x_i \cos + y_i \sin ,$$

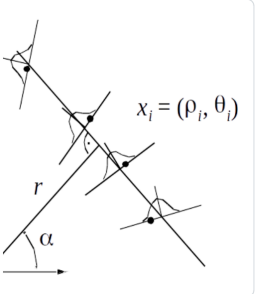
Incertezza Nei Dati Grezzi Del Sensore

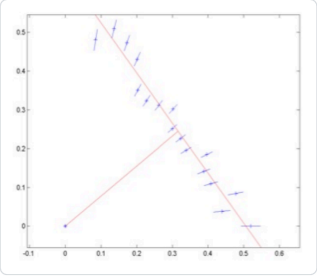
Ciascun raggio laser misura una distanza a un angolo θ_i. La conversione in coordinate cartesiane vale:

$$x_i = \cos\theta_i$$
$$y_i = \sin\theta_i$$

A causa del rumore di misura del sensore, le coordinate polari sono variabili aleatorie con matrice di covarianza:

$$C_i^{\theta} = \begin{vmatrix} & 0 \\ 0 & \end{vmatrix}$$





Ipotesi di Indipendenza e Matrice a Blocchi Diagonali

Nel contesto stocastico, la matrice di covarianza congiunta dell'intera scansione di N punti viene modellata come una matrice diagonale a blocchi:

$$C_{} = \begin{bmatrix} C_1^{\theta} & \cdots & \\ & C_2^{\theta} & \cdots & \\ \cdot & \cdot & & \cdot \\ & & \cdots & C_N^{\theta} \end{bmatrix}$$

Come evidenziato a lezione, l'assunzione che gli errori sui singoli raggi laser siano reciprocamente indipendenti è un'approssimazione (nella realtà esistono correlazioni temporali e spaziali dovute alla risposta del fotodiodo o al movimento del robot durante la scansione). Tuttavia, questa ipotesi è sufficientemente accurata da poter essere applicata con successo senza appesantire il carico computazionale.

Minimi Quadrati Ordinari (OLS) E Minimi Quadrati Pesati (WLS)

Dato un insieme di N punti $(x_i, y_i)_{i=1}^{N}$ appartenenti alla medesima retta, l'obiettivo è determinare i parametri (ρ, θ) che minimizzano la somma dei quadrati delle distanze ortogonali:

$$S(\rho, \theta) = \sum_{i=1}^{N} r_i^2 = \sum_{i=1}^{N} (x_i \cos\theta + y_i \sin\theta - \rho)^2$$

Annullando la derivata parziale rispetto a ρ:

$$\frac{\partial S}{\partial \rho} = -2 \sum_{i=1}^{N} (x_i \cos\theta + y_i \sin\theta - \rho) = 0 \implies \rho = \bar{x}\cos\theta + \bar{y}\sin\theta$$

dove $(\bar{x}, \bar{y}) = \left(\frac{1}{N}\sum x_i, \frac{1}{N}\sum y_i\right)$ è il baricentro dei punti. La retta ottimale passa sempre per il baricentro dell'insieme dei campioni!

Sostituendo ρ in S e minimizzando rispetto ad θ si perviene alla soluzione in forma chiusa:

$$\tan(2\theta) = \frac{-2\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (y_i - \bar{y})^2 - \sum (x_i - \bar{x})^2}$$

Nel caso in cui i singoli punti posseggano varianze differenti σ_i^2, si formula il problema ai **minimi quadrati pesati (WLS)**:

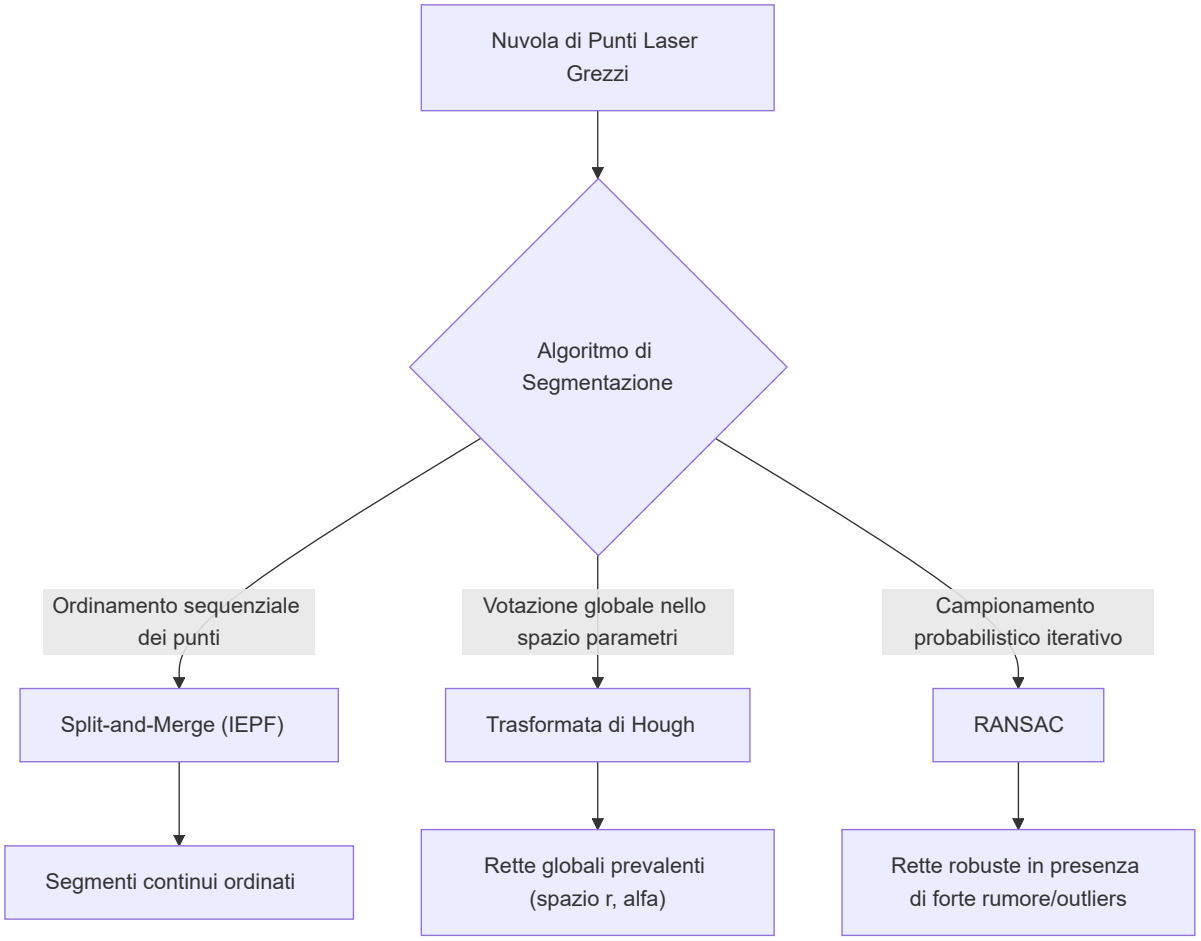
$$S_w(\rho, \theta) = \sum_{i=1}^{N} w_i (x_i \cos\theta + y_i \sin\theta - \rho)^2, \quad w_i = \frac{1}{\sigma_i^2}$$

Approfondimento Teorico

Per la derivazione analitica passo per passo della soluzione per θ, la disambiguazione del quadrante e la propagazione dell'incertezza con il Jacobiano $\partial(\rho, \theta)/\partial(x, y)$ sulla covarianza stimata C_{xy} si veda zz. Argomenti extra: Derivazione Analitica del Line Fitting.

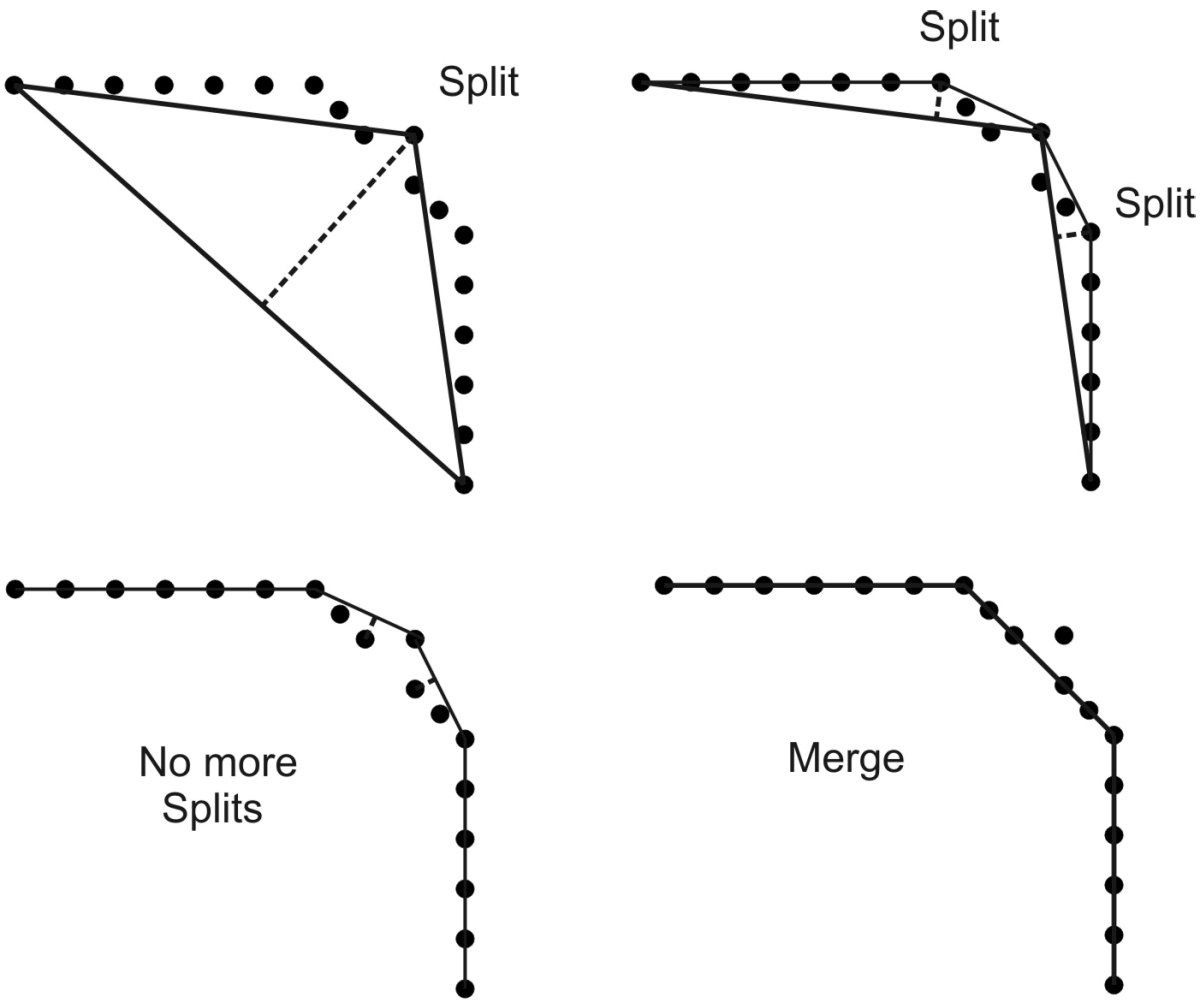
Algoritmi Di Segmentazione

La segmentazione consiste nel partizionare l'insieme totale di punti misurati in sottoinsiemi disgiunti, ciascuno associato a una specifica linea fisica presente nell'ambiente. Vengono analizzati i tre algoritmi fondamentali.



Split-and-Merge (Iterative-End-Point-Fit)

L'algoritmo **Split-and-Merge** (nella sua variante canonica *Iterative-End-Point-Fit*, IEPF) sfrutta la natura intrinsecamente ordinata della scansione laser (i punti sono acquisiti sequenzialmente per angoli θ crescenti).



Procedura Operativa Passo-Passo

1. **Linea iniziale**: Si traccia un segmento di retta provvisorio congiungente il primo punto P_1 e l'ultimo punto P_N della sequenza;
2. **Ricerca della massima distanza**: Per ciascun punto intermedio P_i, si calcola la distanza ortogonale d_i dal segmento; sia $d_{\max} = \max_i d_i$ la distanza massima corrispondente al punto P_k;
3. **Fase di Split**:
   - Se $d_{\max} > \varepsilon$ (soglia di tolleranza prestabilita), la sequenza di punti viene **spezzata in due sottoinsiemi**: da P_1 a P_k e da P_k a P_N;
   - Si applica ricorsivamente la procedura su ciascuno dei due sotto-segmenti;
4. **Arresto dello Split**: Quando in un sotto-segmento nessun punto dista più di ε dalla retta congiungente gli estremi, la suddivisione ricorsiva per quel segmento termina;
5. **Fase di Merge**: I segmenti contigui ottenuti vengono confrontati: se i parametri delle rette associate risultano quasi collineari e la distanza tra i punti di confine è inferiore a una soglia metrica, i segmenti vengono **fusi** (*merged*) in un'unica linea e si esegue un fit globale su tutti i punti congiunti.

> **Chiarimento Concettuale: Split-and-Merge e l'Associazione dei Punti**
>
> Contrariamente a quanto a volte ritenuto, **l'algoritmo Split-and-Merge associa esplicitamente ciascun punto alla rispettiva retta**. Poiché opera per partizioni successive della sequenza ordinata di indici, al termine dell'algoritmo ogni segmento di retta possiede la lista esatta e ordinata dei punti che lo compongono.

- **Vantaggi**: Estremamente veloce ($O(N \log N)$ nel caso medio), deterministico, eccellente per contorni continui;
- **Svantaggi**: Dipende fortemente dall'ordinamento angolare dei punti e può fallire in presenza di punti sparsi isolati o salti di continuità non filtrati.

Trasformata Di Hough

La **Trasformata di Hough** è una tecnica di trasformazione per accumulazione globale introdotta per rilevare forme geometriche parametrizzabili anche in presenza di lacune o forte rumore.

Principio Della Dualità Punto-Linea

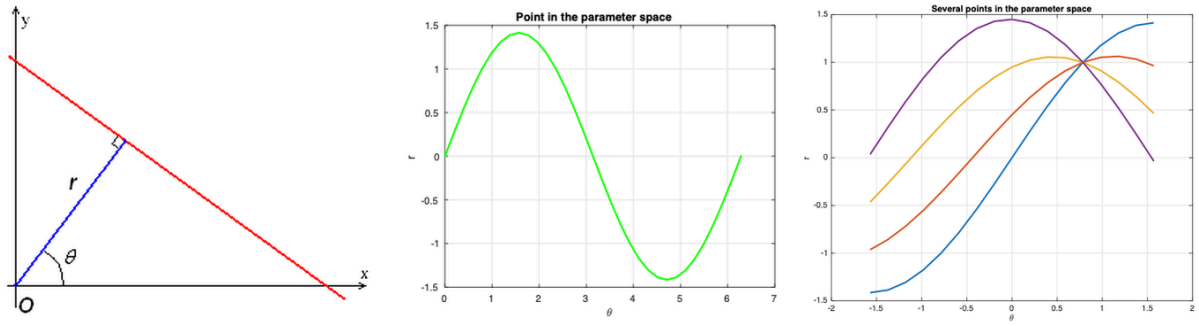
Nello spazio cartesiano geometrico (x, y), una retta è descritta dalla forma normale di Hesse:

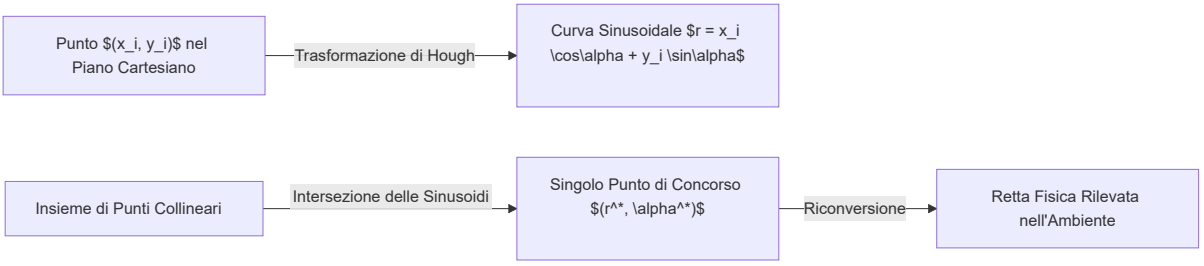
$$x \cos \theta + y \sin \theta = r$$

Se consideriamo un **singolo punto fissato** $P_i = (x_i, y_i)$, quali sono tutte le infinite rette che possono passare per quel punto? Variando l'orientamento $\theta \in [-\pi, \pi]$, la distanza r necessaria affinché la retta passi per P_i è:

$$r(\theta) = x_i \cos \theta + y_i \sin \theta$$

Nello spazio dei parametri (r, θ), questa equazione rappresenta una **curva sinusoidale**!





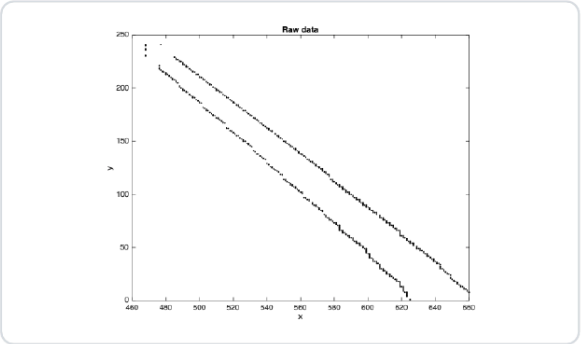
Risoluzione del Dubbio Concettuale sulla Trasformata di Hough

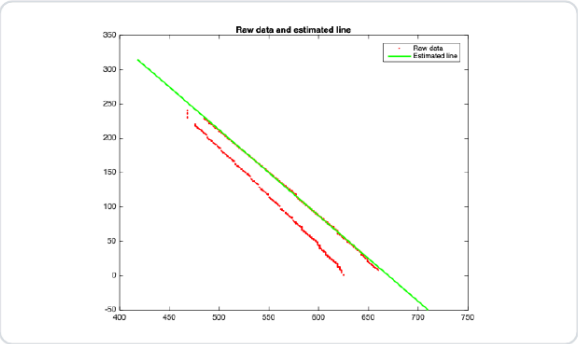
L'idea alla base di Hough è elegante e potente:

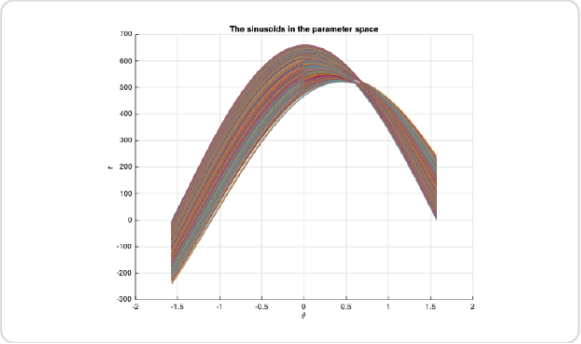
1. Un **punto** nello spazio geometrico diventa una **sinusoide** nello spazio dei parametri (r, θ);
2. Se più punti $P_1, P_2, \ldots, P_k$ appartengono alla **stessa retta fisica**, le rispettive curve sinusoidali nello spazio dei parametri **si intersecheranno tutte esattamente nello stesso punto** (r, θ);
3. Il problema di identificare una retta nello spazio cartesiano si trasforma quindi nel problema di individuare il punto di massima intersezione tra curve nello spazio dei parametri!

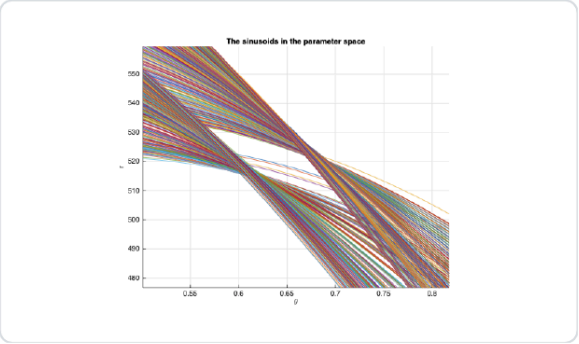
L'Accumulatore Discreto E Il Meccanismo Di Voto

Poiché i computer lavorano su insiemi discreti, lo spazio dei parametri (r, θ) viene quantizzato in una matrice bidimensionale detta **accumulatore di Hough** $A[r, \theta]$:










```
Algoritmo: Trasformata di Hough per Rette
--------------------------------------------------------------------
Dati: Insieme P di punti misurati (x_i, y_i)
Risultato: Parametri delle rette dominanti (r, alpha)

1. Inizializzazione:
   - Quantizza lo spazio (alpha, r) in una griglia H di dimensioni (N_r x N_alpha)
   - Azzera tutti i valori: H[r_q, alpha_q] = 0 per ogni cella

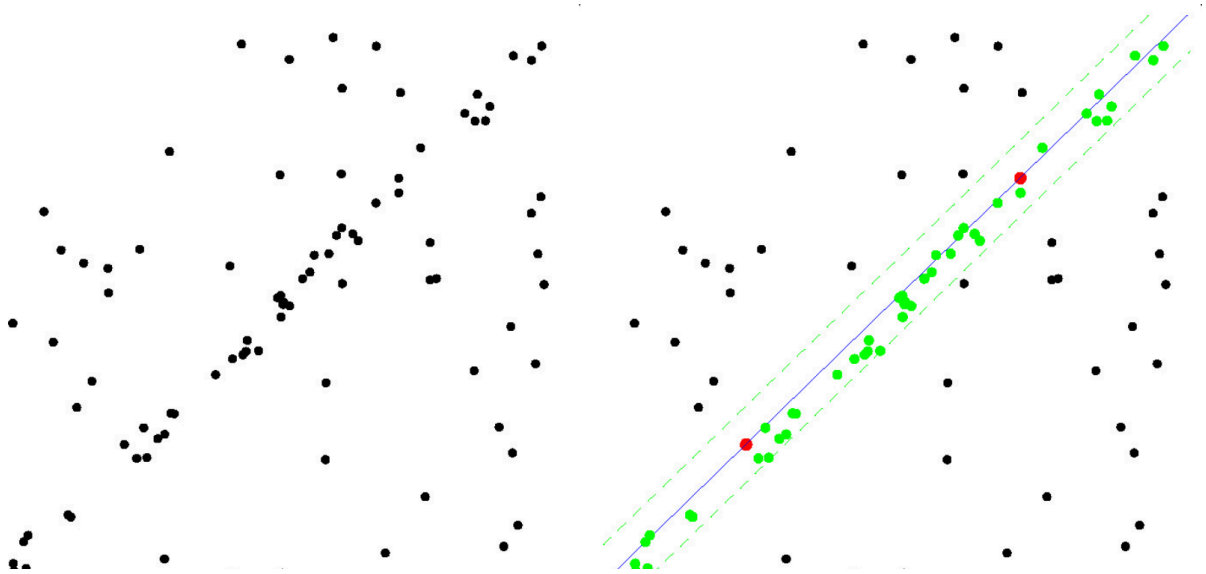
2. Votazione (Voting):
   - Per ciascun punto (x_i, y_i) in P:
       - Per ciascun valore discretizzato alpha_q in A_q:
           - Calcola r_q = x_i * cos(alpha_q) + y_i * sin(alpha_q)
           - Trova l'indice di quantizzazione più vicino per r_q
           - Incrementa il contatore: H[r_q, alpha_q] = H[r_q, alpha_q] + 1

3. Estrazione dei Massimi:
   - Trova i picchi locali (massimi) nella matrice accumulatore H
   - Le celle che superano una soglia di voto T corrispondono alle rette stimate!
```

> **⚠ Limitazione dell'Accumulatore: Associazione dei Punti**
>
> A differenza di Split-and-Merge, la procedura di voto standard nell'accumulatore individua i parametri (r, θ) della retta globale ma **non tiene memoria diretta di quali specifici punti abbiano votato per quella cella**.
> Per recuperare i segmenti fisici ed evitare di unire pareti distanti allineate casualmente:
>
> 1. Si individuano i parametri (r, θ) del picco nell'accumulatore;
> 2. Si esegue una scansione retroattiva dei dati per trovare i punti la cui distanza ortogonale da quella retta sia minore di una tolleranza ε;
> 3. Si raggruppano i punti in segmenti finiti e si rimuovono dalla lista dei punti per consentire il rilevamento delle rette successive.

RANSAC (RANdom SAmple Consensus)

RANSAC è un algoritmo non deterministico iterativo concepito specificamente per stimare i parametri di un modello matematico in presenza di un'elevata percentuale di **outliers** (punti spuri, riflessioni anomale, rumore pesante o ostacoli dinamici non appartenenti al modello).



Funzionamento dell'Algoritmo

Per stimare una retta nel piano cartesiano sono necessari al minimo $s = 2$ punti distinti.

```
Algoritmo: RANSAC per Line Fitting
----------------------------------------------------------------------
Dati: Insieme P di N punti misurati, soglia di distanza epsilon,
      numero minimo di inliers M, numero massimo di iterazioni K
Risultato: Retta ottima e insieme di inliers

Ripeti K volte:
  1. Campiona casualmente un sottoinsieme minimo di 2 punti da P
  2. Calcola la retta candidata passante per i 2 punti (modello provvisorio)
  3. Calcola la distanza ortogonale d_i di TUTTI gli altri punti da questa retta
  4. Costruisci il Consensus Set (Inliers):
     Inliers = { P_i in P : d_i < epsilon }
  5. Se |Inliers| > |Miglior_Consenso|:
     Miglior_Consenso = Inliers
     Miglior_Modello = parametri della retta corrente

Fine ciclo:
  - Prendi il Miglior_Consenso (se supera la soglia M)
  - Ricalcola la retta definitiva eseguendo un Total Least Squares (OLS/WLS)
    su TUTTI i punti appartenenti al Miglior_Consenso!
  - I punti fuori dal consenso vengono scartati come outliers.
```

Determinazione Del Numero Di Iterazioni K

Quante iterazioni casuali K sono necessarie per essere ragionevolmente sicuri di aver estratto almeno una volta una coppia di punti contenente esclusivamente inliers validi?

Sia w la frazione di inliers nell'intero dataset ($w = \frac{N_{inliers}}{N_{tot}}$).
La probabilità che $s = 2$ punti estratti a caso siano entrambi inliers è:

$$P(\text{2 inliers}) = w^s = w^2$$

La probabilità che almeno uno dei due sia un outlier (campione corrotto) vale:

$$P(\text{campione errato}) = 1 - w^s$$

La probabilità che in tutte le K iterazioni indipendenti si estragga sempre un campione corrotto è:

$$P(\text{fallimento in } K \text{ passi}) = (1 - w^s)^K$$

Imponendo che la probabilità di successo sia almeno p (con confidenza tipica $p = 0.99$):

$$1 - (1 - w^s)^K \geq p \implies (1 - w^s)^K \leq 1 - p$$

Applicando il logaritmo naturale a entrambi i membri:

$$K \geq \frac{\ln(1 - p)}{\ln(1 - w^s)}$$

Esempio Numerico di RANSAC

Se l'ambiente presenta il 50% di rumore e ostacoli spuri ($w = 0.5$) e vogliamo una certezza del 99% ($p = 0.99$) con un modello lineare a $s = 2$ punti:

$$K \geq \frac{\ln(1 - 0.99)}{\ln(1 - 0.5^2)} = \frac{\ln(0.01)}{\ln(0.75)} \approx \frac{-4.605}{-0.287} \approx 16.01 \implies K = 17 \text{ iterazioni}$$

Bastano appena 17 iterazioni per avere il 99% di probabilità matematica di estrarre una coppia di punti valida e ripulire completamente gli outliers!

Tabella Comparativa Degli Algoritmi Di Segmentazione

Caratteristica	Split-and-Merge (IEPF)	Trasformata di Hough	RANSAC
Input Richiesto	Punti con ordinamento angolare sequenziale	Punti sparsi qualsiasi	Punti sparsi qualsiasi
Complessità Temporale	$O(N \log N)$ (molto veloce)	$O(N \cdot N_\theta)$ (dipende dalla risoluzione)	$O(K \cdot N)$ (K limitato)
Tolleranza Outliers	Bassa (sensibile a salti di scansione)	Media (i falsi picchi creano artefatti)	**Altissima** (robusto fino al 50%-70% di rumore)
Associazione dei Punti	**Nativa ed esplicita** per ogni segmento	Indiretta (richiede post-processing)	**Esplicita** (tramite *consensus set*)
Natura dell'Algoritmo	Deterministica	Deterministica (su griglia quantizzata)	Probabilistica / Stocastica
Ambito di Elezione	Scansioni LiDAR 2D pulite per muri continui	Riconoscimento linee globali e forme note	Dati 3D rumorosi, stereo vision, feature visuali

4. Localizzazione E Filtri Di Bayes

La localizzazione rappresenta una delle competenze primarie per la mobilità autonoma: consiste nel determinare in tempo reale la posa di un robot (posizione sul piano e orientamento) all'interno di una mappa nota dell'ambiente, fondendo le informazioni provenienti dai sensori di bordo.

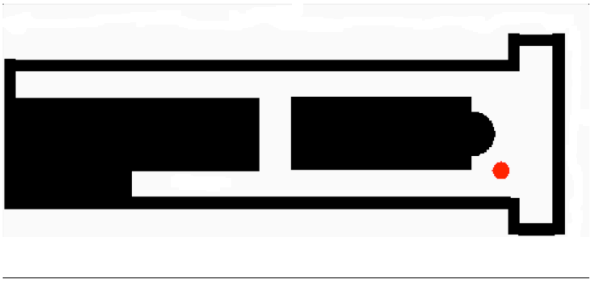
A causa delle imperfezioni dei dispositivi fisici (rumore di misura, aliasing sensoriale, slittamenti cinematici), la localizzazione non può essere trattata con approcci deterministici, bensì come un **problema di stima stocastica ricorsiva**.

Questo capitolo sviluppa in modo rigoroso il framework probabilistico fondamentale della robotica mobile:

1. La formulazione stocastica della localizzazione e le sorgenti di incertezza;
2. La rappresentazione formale del **belief** e la tassonomia dei suoi modelli continui e discreti;
3. I **Modelli di Markov Nascosti (HMM)** e le proprietà di indipendenza condizionale;
4. L'algoritmo concettuale del **Filtro di Bayes Ricorsivo** (ciclo di Predizione e Correzione);
5. L'analisi quantitativa dell'esempio didattico della porta (*Door State Estimation*);
6. La dimostrazione formale della ricorsione bayesiana per induzione;
7. I limiti fisici dell'assunzione markoviana e l'introduzione alle approssimazioni gaussiane.

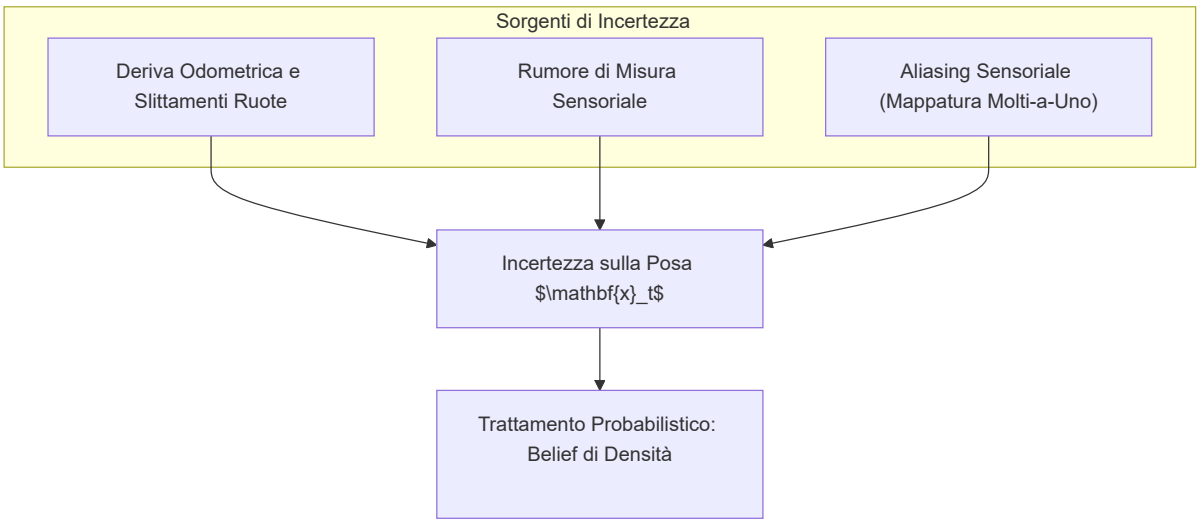
Dalla Localizzazione Deterministica Alla Stima Stocastica

Nel caso ideale teorico, la traiettoria di un veicolo terrestre potrebbe essere calcolata semplicemente integrando i comandi di velocità delle ruote o le letture degli encoder (odometria pura).



Nel mondo reale, l'evoluzione dello stato è soggetta a tre sorgenti ineliminabili di incertezza:

1. **Rumore nei dati propriocettivi (Attuazione e Odometria)**:
   - Gli encoder delle ruote e le piattaforme IMU misurano il movimento relativo rispetto al telaio, ma sono affetti da derive temporali non limitate (*unbounded drift*);
   - A causa di micro-slittamenti tra pneumatico e terreno, attriti e giochi meccanici, l'effetto cinematico reale generato da un comando di velocità $\mathbf{u}_t = (v_t, \omega_t)^T$ è sempre diverso da quello teorico pianificato;
2. **Rumore nei dati esteriorocettivi (Percezione dell'Ambiente)**:
   - LiDAR, telecamere, sonar e ricevitori GNSS catturano la relazione geometrica tra robot e ostacoli circostanti, ma sono degradati da rumore di quantizzazione, riflessioni anomale e condizioni operative variabili;
3. **Aliasing Sensoriale (*Perceptual Aliasing*)**:
   - È la condizione di **mappatura molti-a-uno**: ambienti geometricamente regolari o simmetrici (es. una sequenza di porte identiche lungo un corridoio rettilineo) generano letture sensoriali indistinguibili in punti spaziali diversi;
   - Una singola misura istantanea non è sufficiente a discriminare la posa corretta tra più ipotesi ambigue.



Rappresentazione Del Belief (Stato Di Conoscenza)

Sia $\mathbf{x}_t \in \Omega$ la posa reale del robot all'istante temporale t. Nel caso comune di navigazione su un pavimento orizzontale, la posa è un vettore a tre dimensioni:

$$\mathbf{x}_t = \begin{bmatrix} x_t \\ y_t \\ \theta_t \end{bmatrix} \in \mathbb{R}^2 \times [-\pi, \pi) \ \Omega$$

Definiamo le storie temporali dei dati disponibili al controllore:

- $\mathbf{u}_{1:t} = \mathbf{u}_1, \mathbf{u}_2, \ldots, \mathbf{u}_t$: sequenza dei comandi di controllo propriocettivi fino al tempo t;
- $\mathbf{z}_{1:t} = \mathbf{z}_1, \mathbf{z}_2, \ldots, \mathbf{z}_t$: sequenza delle misure esteriorocettive fino al tempo t.

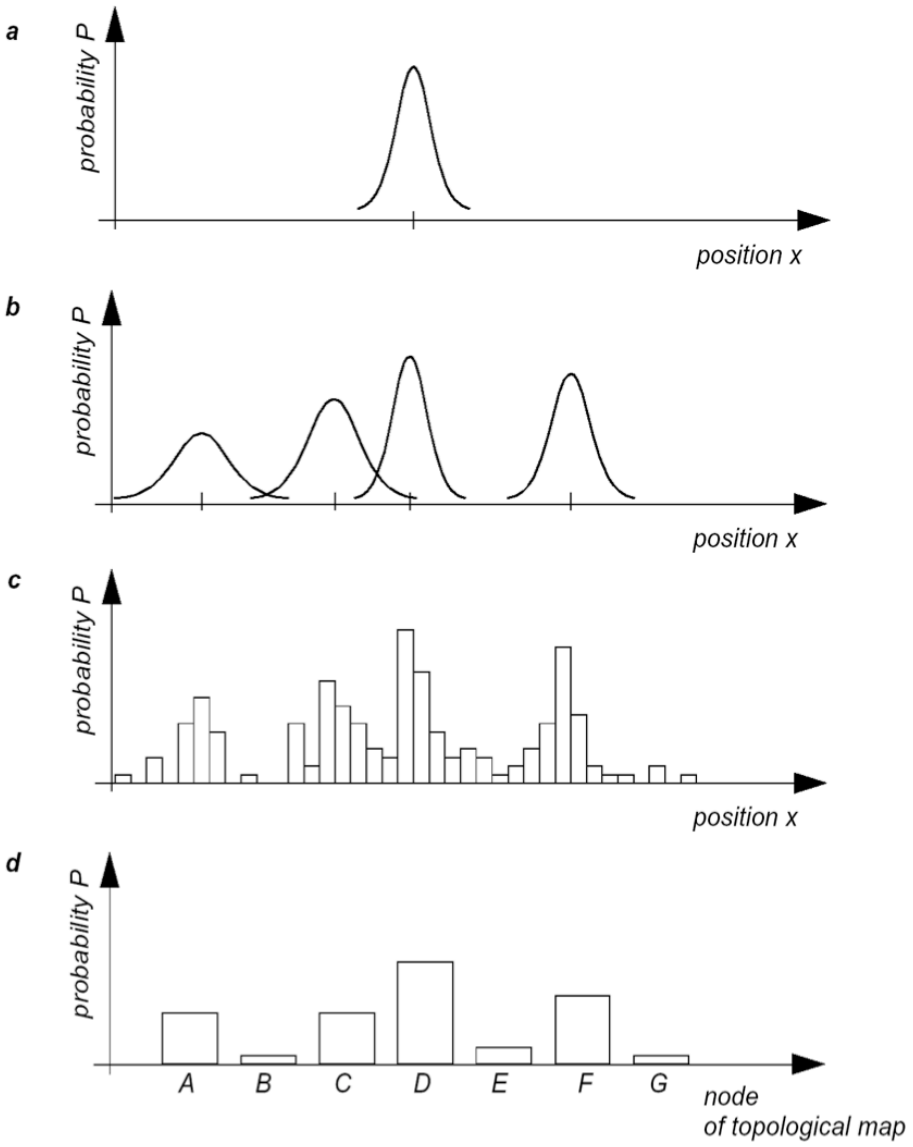
> **Definizione Formale di Belief a Posteriori**
>
> Il **belief** (stato di convinzione o conoscenza interna) del robot sulla propria posa è la funzione di densità di probabilità condizionata (*pdf*) a tutta l'informazione storica disponibile:
>
> $$bel(\mathbf{x}_t) := p(\mathbf{x}_t \mid \mathbf{z}_{1:t}, \mathbf{u}_{1:t})$$

Prima di acquisire la misura all'istante t, il robot può formulare una predizione basata unicamente sull'azione di movimento appena completata. Tale densità prende il nome di **belief a priori** (o predizione):

$$\overline{bel}(\mathbf{x}_t) := p(\mathbf{x}_t \mid \mathbf{z}_{1:t-1}, \mathbf{u}_{1:t})$$

Tassonomia Delle Rappresentazioni Del Belief

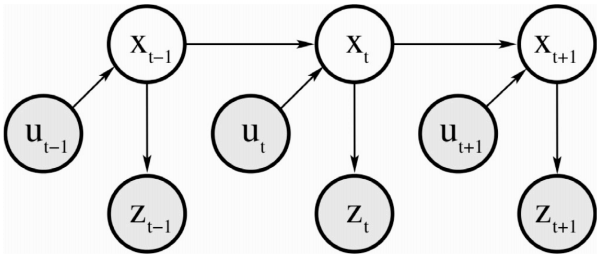
Il formalismo matematico scelto per rappresentare la funzione $bel(\mathbf{x}_t)$ caratterizza la famiglia del filtro di stima. Come classificato nella letteratura di riferimento (Siegwart e Scaramuzza), si distinguono quattro modelli:



1. **Ipotesi Singola Continua (Gaussiana Unimodale - Pannello a)**:
   - Il belief è descritto da un'unica funzione di Gauss (μ, σ);
   - Tipica dei **Filtri di Kalman lineari ed estesi (KF, EKF)**;
   - *Vantaggi*: eccezionale efficienza computazionale (si aggiornano solo vettore media e matrice di covarianza);
   - *Limiti*: non gestisce ambiguità globali né distribuzioni multimodali (*unimodal assumption*).
2. **Ipotesi Multiple Continue (Distribuzione Multimodale - Pannello b)**:
   - Modellata come una combinazione lineare di più gaussiane (*Gaussian Mixture Models*, GMM);
   - Mantiene in vita diverse tracce parallele di localizzazione (*Multi-Hypothesis Tracking*);
   - *Limiti*: complessità computazionale elevata nel gestire la scissione e fusione delle componenti gaussiane.
3. **Griglia Discreta / Istogramma a Tratti (Pannello c)**:
   - L'ambiente continuo viene suddiviso in una maglia di celle discrete costanti a tratti;
   - Tipica degli **Histogram Filters** e della localizzazione su griglie di occupazione;
   - *Vantaggi*: forma libera arbitraria, gestisce la localizzazione globale da zero e il problema del robot rapito (*kidnapped robot*);
   - *Limiti*: esplosione esponenziale della memoria e del tempo di calcolo all'aumentare delle dimensioni dello stato ($O(N^3)$ per pose piane).
4. **Rappresentazione Topologica Discreta (Pannello d)**:
   - Distribuzione di probabilità definita su un insieme finito di nodi simbolici di un grafo topologico (es. Stanza A, Stanza B);
   - Leggera e veloce per pianificazione ad alto livello, ma totalmente priva di risoluzione metrica continua.

Modelli Di Markov Nascosti (HMM) E Proprietà Di Markov

L'interazione temporale tra il robot e l'ambiente viene descritta come un **Modello di Markov Nascosto (Hidden Markov Model, HMM)**, schematizzabile mediante una rete bayesiana dinamica (*Dynamic Bayesian Network, DBN*):



La rete evidenzia i tre flussi informativi del sistema:

- Gli stati $\mathbf{x}_t$ sono nodi nascosti (*latenti*), non direttamente misurabili;
- I comandi $\mathbf{u}_t$ agiscono come ingressi deterministici che guidano la transizione dello stato;
- Le osservazioni $\mathbf{z}_t$ sono nodi osservabili generati dallo stato del robot all'istante di campionamento.

Le Due Ipotesi Fondamentali Del Modello

1. Completezza Dello Stato E Transizione Markoviana

Uno stato si definisce **completo** se racchiude integralmente la memoria passata del sistema, rendendo la storia antecedente irrilevante per la predizione dell'evoluzione successiva.

Nel passaggio temporale discreto tra il passo $t-1$ e il passo t, ciò implica che:
conoscendo lo stato al passo precedente $\mathbf{x}_{t-1}$ e il comando di controllo $\mathbf{u}_t$, lo stato corrente $\mathbf{x}_t$ è condizionatamente indipendente da tutti gli stati antecedenti $\mathbf{x}_{0:t-2}$, dalle misure pregresse $\mathbf{z}_{1:t-1}$ e dai comandi passati $\mathbf{u}_{1:t-1}$:

$$p(\mathbf{x}_t \mid \mathbf{x}_{0:t-1}, \mathbf{z}_{1:t-1}, \mathbf{u}_{1:t}) = p(\mathbf{x}_t \mid \mathbf{x}_{t-1}, \mathbf{u}_t)$$

$p(\mathbf{x}_t \mid \mathbf{x}_{t-1}, \mathbf{u}_t)$ rappresenta la **probabilità di transizione di stato** (il modello cinematico stocastico di movimento).

> **Formulazione Proiettata al Passo Futuro $t+1$**
>
> In modo del tutto equivalente, se si assume come istante di riferimento la posa corrente $\mathbf{x}_t$, la completezza dello stato garantisce che la distribuzione della posa al passo futuro $\mathbf{x}_{t+1}$ dipenda unicamente da $\mathbf{x}_t$ e dal comando successivo $\mathbf{u}_{t+1}$:
>
> $$p(\mathbf{x}_{t+1} \mid \mathbf{x}_{0:t}, \mathbf{z}_{1:t}, \mathbf{u}_{1:t+1}) = p(\mathbf{x}_{t+1} \mid \mathbf{x}_t, \mathbf{u}_{t+1})$$

2. Indipendenza Condizionale Delle Misure

La lettura sensoriale $\mathbf{z}_t$ dipende esclusivamente dallo stato fisico istantaneo $\mathbf{x}_t$ occupato dal robot nell'istante di campionamento:

$$p(\mathbf{z}_t \mid \mathbf{x}_{1:t}, \mathbf{z}_{1:t-1}, \mathbf{u}_{1:t}) = p(\mathbf{z}_t \mid \mathbf{x}_t)$$

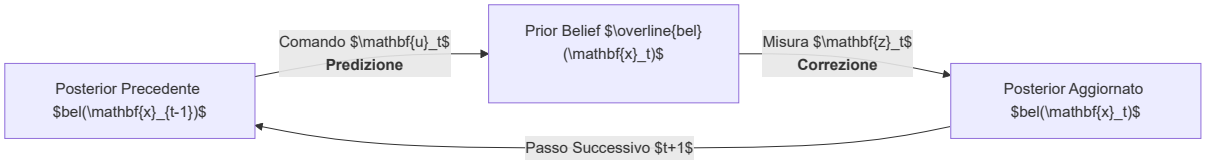
$p(\mathbf{z}_t \mid \mathbf{x}_t)$ rappresenta la **probabilità di misura** (la funzione di verosimiglianza o *likelihood* del sensore).

> **⚠ Attenzione: Misure e Dipendenza Temporale**
>
> È fondamentale chiarire un equivoco concettuale ricorrente: la verosimiglianza della misura $\mathbf{z}_t$ dipende unicamente dallo stato **corrente** $\mathbf{x}_t$ e **NON dallo stato precedente** $\mathbf{x}_{t-1}$. Conoscere dove si trovava il robot un secondo prima non altera la

fisica del raggio laser o dell'immagine acquisita nella posa presente.

Algoritmo Del Filtro Di Bayes Ricorsivo

Il Filtro di Bayes è un paradigma algoritmico ricorsivo che calcola la densità di probabilità $bel(\mathbf{x}_t)$ alternando ciclicamente due fasi complementari.



Struttura dell'Algoritmo

```
Algoritmo: Filtro di Bayes Canonico
-----------------------------------------------------------------------
Dati:   bel(x_{t-1}), comando di controllo u_t, misura z_t
Output: bel(x_t)

1. per ogni posa x_t in Omega fare:
2.     \bar{bel}(x_t) = \int_Omega p(x_t | x_{t-1}, u_t) * bel(x_{t-1}) dx_{t-1}   [PREDIZIONE]
3.     bel(x_t)       = \eta * p(z_t | x_t) * \bar{bel}(x_t)                        [CORREZIONE]
4. fine
5. ritorna bel(x_t)
```

Dettaglio Matematico Dei Due Passaggi

1. Fase Di Predizione (Prediction Step)

Applica il **teorema della probabilità totale** (equazione di Chapman-Kolmogorov):

$$\overline{bel}(\mathbf{x}_t) = \int_{\Omega} p(\mathbf{x}_t \mid \mathbf{x}_{t-1}, \mathbf{u}_t)\, bel(\mathbf{x}_{t-1})\, \mathbf{x}_{t-1}$$

- Il robot integra il comando di controllo $\mathbf{u}_t$ convoluto con la conoscenza a priori dello stato precedente;
- Poiché l'attuazione fisica è imperfetta, questa operazione **aumenta l'entropia e la dispersione** della distribuzione: la varianza cresce e il picco di probabilità si allarga.

2. Fase Di Correzione (Measurement Update)

Applica la **regola di Bayes** incorporando l'evidenza empirica della lettura sensoriale $\mathbf{z}_t$:

$$bel(\mathbf{x}_t) = \eta\, p(\mathbf{z}_t \mid \mathbf{x}_t)\, \overline{bel}(\mathbf{x}_t)$$

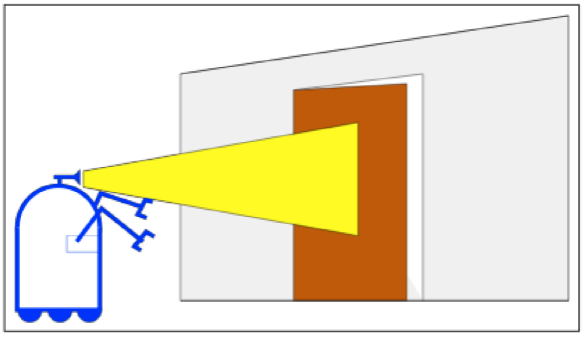
- La verosimiglianza $p(\mathbf{z}_t \mid \mathbf{x}_t)$ modula punto per punto la predizione cinematica, premiando le pose compatibili con il sensore e penalizzando quelle incoerenti;
- Questa fase **riduce l'incertezza**: la varianza diminuisce e il profilo di probabilità si restringe attorno alla stima corretta;
- η è la costante di normalizzazione necessaria a garantire che l'integrale dell'intera pdf rimanga unitario:

$$\eta = \left(\int_{\Omega} p(\mathbf{z}_t \mid \mathbf{x}_t)\, \overline{bel}(\mathbf{x}_t)\, \mathbf{x}_t \right)^{-1}$$

> ⚠ **Perché il Filtro Continuo è Puramente Concettuale?**
>
> Nel caso di uno spazio continuo come il pavimento di una stanza ($\Omega = \mathbb{R}^2 \times [-\pi, \pi)$), la variabile $\mathbf{x}_t$ può assumere infiniti valori reali.
> Il ciclo `per ogni posa x_t in Omega` richiederebbe di valutare **infiniti integrali tridimensionali a ogni ciclo temporale**: **questo algoritmo non è direttamente implementabile su alcun calcolatore digitale**.
> Inoltre, a meno che $p(\mathbf{x}_t \mid \mathbf{x}_{t-1}, \mathbf{u}_t)$ e $bel(\mathbf{x}_{t-1})$ non appartengano a famiglie matematiche integrabili in forma chiusa algebrica (es. polinomi o gaussiane lineari), l'integrale di Chapman-Kolmogorov non ammette primitiva analitica. Tutte le tecniche di robotica pratica consistono in **metodi di discretizzazione finita**:
>
> - Discretizzazione a celle cubiche fisse → **Histogram Filter**;
> - Discretizzazione a campioni stocastici pesati → **Filtri a Particelle (MCL / AMCL)**;
> - Restrizione parametrica a media e covarianza → **Filtro di Kalman (EKF, UKF)**.

Esempio Didattico Dettagliato: La Porta Di Thrun

Per comprendere la meccanica computazionale del filtro senza le complicazioni dello spazio continuo tridimensionale, si analizza il celebre problema minimale formulato da Sebastian Thrun: un robot mobile deve determinare lo stato di una porta e attraversarla.



Definizione Del Modello Discreto

Le variabili del sistema assumono valori in domini binari discreti:

- **Stato del mondo**: $X_t \in \{\text{is_open}, \text{is_closed}\}$;
- **Misure sensoriali**: $Z_t \in \{\text{sense_open}, \text{sense_closed}\}$;
- **Azioni di controllo**: $U_t \in \{\text{do_nothing}, \text{push}\}$

Conoscenza Iniziale (Stato Di Massima Ignoranza)

All'istante iniziale $t = 0$, il robot non dispone di alcuna misura pregressa:

$$bel(X_0 = \text{is_open}) = 0.5, \quad bel(X_0 = \text{is_closed}) = 0.5$$

Modello Di Misura (Stochastic Sensor Model)

Il sensore ottico/laser del robot è rumoroso e soggetto a falsi positivi e falsi negativi:

$$\begin{aligned} p(Z_t = \text{sense_open} \mid X_t = \text{is_open}) &= 0.6, & p(Z_t = \text{sense_closed} \mid X_t = \text{is_open}) &= 0.4 \\ p(Z_t = \text{sense_open} \mid X_t = \text{is_closed}) &= 0.2, & p(Z_t = \text{sense_closed} \mid X_t = \text{is_closed}) &= 0.8 \end{aligned}$$

> **Interpretazione della Verosimiglianza**
>
> Si noti che se la porta è chiusa, il sensore ha comunque un 20% di probabilità di rilevare erroneamente `sense_open`. La misura z_t è un dato noto, ma **non garantisce con certezza lo stato fisico sottostante**.

Modello Di Transizione Di Stato (State Transition Probability)

- Se il robot esegue l'azione **`do_nothing`**, si assume che l'ambiente sia statico e che la porta non possa cambiare stato da sola:

$$p(X_t = X_{t-1} \mid u_t = \text{do_nothing}) = 1.0$$

- Se il robot esegue l'azione **`push`**:
  - Se la porta era già aperta, rimane aperta con certezza:

$$p(\text{open} \mid \text{push}, \text{open}) = 1.0$$

  - Se la porta era chiusa, l'attuatore ha successo nell'80% dei casi e fallisce nel 20%:

$$p(\text{open} \mid \text{push}, \text{closed}) = 0.8 \quad p(\text{closed} \mid \text{push}, \text{closed}) = 0.2$$

Passo Temporale $t = 1$: Azione Nulla E Misura `sense_open`

All'istante $t = 1$, il robot non si muove ($u_1 = \text{do_nothing}$) e acquisisce la lettura $z_1 = \text{sense_open}$.

1. Calcolo Del Prior $\overline{bel}(X_1)$ (Predizione)

Nel dominio discreto, l'integrale di Chapman-Kolmogorov diventa una sommatoria finita su tutti i possibili stati antecedenti x_0:

$$\overline{bel}(x_1) = \sum_{x_0} p(x_1 \mid x_0, u_1)\, bel(x_0)$$

Poiché $u_1 = \text{do_nothing}$, la matrice di transizione è l'identità:

$$\begin{aligned}
\overline{bel}(X_1 = \text{open}) &= p(\text{open} \mid \text{open}, \text{none})\, bel(X_0 = \text{open}) + p(\text{open} \mid \text{closed}, \text{none})\, bel(X_0 = \text{closed}) \\
&= 1.0 \cdot 0.5 + 0.0 \cdot 0.5 = 0.5 \\
\overline{bel}(X_1 = \text{closed}) &= p(\text{closed} \mid \text{open}, \text{none})\, bel(X_0 = \text{open}) + p(\text{closed} \mid \text{closed}, \text{none})\, bel(X_0 = \text{closed}) \\
&= 0.0 \cdot 0.5 + 1.0 \cdot 0.5 = 0.5
\end{aligned}$$

Non avendo compiuto azioni e assumendo l'ambiente statico, la predizione a priori coincide esattamente con lo stato iniziale.

2. Calcolo Del Posterior $bel(X_1)$ (Correzione)

Moltiplichiamo il prior per la verosimiglianza di aver rilevato `sense_open`:

$$\begin{aligned}
bel(X_1 = \text{open}) &= \eta\, p(\text{sense_open} \mid \text{open})\, \overline{bel}(X_1 = \text{open}) = \eta \cdot 0.6 \cdot 0.5 = \eta \cdot 0.3 \\
bel(X_1 = \text{closed}) &= \eta\, p(\text{sense_open} \mid \text{closed})\, \overline{bel}(X_1 = \text{closed}) = \eta \cdot 0.2 \cdot 0.5 = \eta \cdot 0.1
\end{aligned}$$

Imponiamo che la somma delle probabilità sia unitaria:

$$\eta\,(0.3 + 0.1) = 1 \implies \eta = \frac{1}{0.4} = 2.5$$

Il belief a posteriori vale:

$$\begin{cases} bel(X_1 = \text{is_open}) = 2.5 \cdot 0.3 = 0.75 \\ bel(X_1 = \text{is_closed}) = 2.5 \cdot 0.1 = 0.25 \end{cases}$$

La prima osservazione sensoriale ha fatto salire la probabilità di porta aperta dal 50% al 75%.

Passo Temporale $t = 2$: Azione `push` E Seconda Misura `sense_open`

All'istante $t = 2$, il robot spinge la porta ($u_2 = \text{push}$) e riceve una seconda lettura favorevole $z_2 = \text{sense_open}$.

1. Calcolo Del Prior $\overline{bel}(X_2)$ (Predizione Con Atto Fisico)

Calcoliamo l'effetto cinematico dell'azione `push` applicata al belief precedente $bel(X_1)$:

$$\begin{aligned} \overline{bel}(X_2 = \text{open}) &= p(\text{open} \mid \text{push}, \text{open})\, bel(X_1 = \text{open}) + p(\text{open} \mid \text{push}, \text{closed})\, bel(X_1 = \text{closed}) \\ &= 1.0 \cdot 0.75 + 0.8\, 0.25 = 0.75 + 0.20 = 0.95 \end{aligned}$$

$$\begin{aligned} \overline{bel}(X_2 = \text{closed}) &= p(\text{closed} \mid \text{push}, \text{open})\, bel(X_1 = \text{open}) + p(\text{closed} \mid \text{push}, \text{closed})\, bel(X_1 = \text{closed}) \\ &= 0.0 \cdot 0.75 + 0.2 \cdot 0.25 = 0.0 + 0.05 = 0.05 \end{aligned}$$

La sola spinta meccanica ha elevato la confidenza a priori al 95%.

2. Calcolo Del Posterior $bel(X_2)$ (Correzione Di Misura)

Combiniamo il prior con la verosimiglianza di $z_2 = \text{sense_open}$:

$$\begin{aligned} bel(X_2 = \text{open}) &= \eta \cdot 0.6 \cdot 0.95 = \eta \cdot 0.57 \\ bel(X_2 = \text{closed}) &= \eta \cdot 0.2 \cdot 0.05 = \eta \cdot 0.01 \end{aligned}$$

Calcoliamo il fattore normalizzante:

$$\eta\,(0.57 + 0.01) = 1 \implies \eta = \frac{1}{0.58} \approx 1.7241$$

Il belief finale a posteriori risulta:

$$\begin{cases} bel(X_2 = \text{is_open}) = 1.7241 \cdot 0.57 \approx 0.983 \ (98.3\%) \\ bel(X_2 = \text{is_closed}) = 1.7241 \cdot 0.01 \approx 0.017 \ (1.7\%) \end{cases}$$

Riflessione Ingegneristica: Il Criterio Six-Sigma (6σ)

Con una confidenza del 98.3%, il robot potrebbe ritenere sicuro attraversare il varco. Tuttavia, la probabilità residua di errore è:

$$p_e = 0.017 \approx 1.7\%$$

Ciò significa che il robot andrà a sbattere contro la porta chiusa quasi **2 volte ogni 100 tentativi**. In contesti industriali critici o nella guida autonoma, tale affidabilità è assolutamente inaccettabile.

Perché si Chiama "Sei-Sigma"?

Il paradigma industriale **Six-Sigma** impone che il tasso di difetti o fallimenti non superi **3.4 parti per milione di opportunità** (DPMO, *Defects Per Million Opportunities*).
Il nome deriva dalla statistica della distribuzione normale standard: l'area sottesa dalle code oltre 6 deviazioni standard (6σ) dal valore medio racchiude il $99.9\ldots\%$ della probabilità, lasciando un tasso di fallimento marginale pari a 3.4×10^{-6}.

Dimostrazione Formale Del Filtro Di Bayes per Induzione

La validità del ciclo iterativo si dimostra per induzione matematica su t, verificando che la ricorsione preservi la definizione rigorosa di probabilità condizionata.

1. Dimostrazione Del Passo Di Correzione (Measurement Update)

Per definizione di probabilità condizionata:

$$bel(\mathbf{x}_t) = p(\mathbf{x}_t \mid \mathbf{z}_{1:t}, \mathbf{u}_{1:t}) = p(\mathbf{x}_t \mid \mathbf{z}_t, \mathbf{z}_{1:t-1}, \mathbf{u}_{1:t})$$

Applicando la regola di Bayes considerando $\mathbf{z}_t$ come evento condizionante:

$$p(\mathbf{x}_t \mid \mathbf{z}_t, \mathbf{z}_{1:t-1}, \mathbf{u}_{1:t}) = \frac{p(\mathbf{z}_t \mid \mathbf{x}_t, \mathbf{z}_{1:t-1}, \mathbf{u}_{1:t})\, p(\mathbf{x}_t \mid \mathbf{z}_{1:t-1}, \mathbf{u}_{1:t})}{p(\mathbf{z}_t \mid \mathbf{z}_{1:t-1}, \mathbf{u}_{1:t})}$$

Poiché il denominatore è una costante rispetto alla variabile di stima $\mathbf{x}_t$, lo raggruppiamo nel fattore di normalizzazione η:

$$bel(\mathbf{x}_t) = \eta\, p(\mathbf{z}_t \mid \mathbf{x}_t, \mathbf{z}_{1:t-1}, \mathbf{u}_{1:t})\, p(\mathbf{x}_t \mid \mathbf{z}_{1:t-1}, \mathbf{u}_{1:t})$$

Sfruttiamo l'**ipotesi markoviana di completezza dello stato**: se la posa corrente $\mathbf{x}_t$ è nota, la misura attuale $\mathbf{z}_t$ è condizionatamente indipendente da tutte le misure passate $\mathbf{z}_{1:t-1}$ e dai comandi $\mathbf{u}_{1:t}$:

$$p(\mathbf{z}_t \mid \mathbf{x}_t, \mathbf{z}_{1:t-1}, \mathbf{u}_{1:t}) = p(\mathbf{z}_t \mid \mathbf{x}_t)$$

Riconoscendo che il secondo fattore corrisponde esattamente al belief a priori:

$$p(\mathbf{x}_t \mid \mathbf{z}_{1:t-1}, \mathbf{u}_{1:t}) = \overline{bel}(\mathbf{x}_t)$$

Otteniamo la tesi cercata:

$$bel(\mathbf{x}_t) = \eta\, p(\mathbf{z}_t \mid \mathbf{x}_t)\, \overline{bel}(\mathbf{x}_t) \quad \blacksquare$$

---

2. Dimostrazione Del Passo Di Predizione (Prediction Step)

Consideriamo il belief a priori $\overline{bel}(\mathbf{x}_t)$ ed espandiamolo marginalizzando rispetto a tutte le possibili pose antecedenti $\mathbf{x}_{t-1}$ tramite il teorema della probabilità totale:

$$\overline{bel}(\mathbf{x}_t) = p(\mathbf{x}_t \mid \mathbf{z}_{1:t-1}, \mathbf{u}_{1:t}) = \int_{\Omega} p(\mathbf{x}_t, \mathbf{x}_{t-1} \mid \mathbf{z}_{1:t-1}, \mathbf{u}_{1:t})\, \mathbf{x}_{t-1}$$

Applicando la regola del prodotto congiunto:

$$p(\mathbf{x}_t, \mathbf{x}_{t-1} \mid \mathbf{z}_{1:t-1}, \mathbf{u}_{1:t}) = p(\mathbf{x}_t \mid \mathbf{x}_{t-1}, \mathbf{z}_{1:t-1}, \mathbf{u}_{1:t})\, p(\mathbf{x}_{t-1} \mid \mathbf{z}_{1:t-1}, \mathbf{u}_{1:t})$$

Semplifichiamo i due fattori sfruttando la struttura causale dell'HMM:

1. **Completezza dello stato sulla transizione**: conoscendo la posa precedente $\mathbf{x}_{t-1}$ e l'ultimo comando $\mathbf{u}_t$, la storia passata dei sensori $\mathbf{z}_{1:t-1}$ e dei controlli pregressi non aggiunge informazione sulla transizione allo stato $\mathbf{x}_t$:

$$p(\mathbf{x}_t \mid \mathbf{x}_{t-1}, \mathbf{z}_{1:t-1}, \mathbf{u}_{1:t}) = p(\mathbf{x}_t \mid \mathbf{x}_{t-1}, \mathbf{u}_t)$$

2. **Causalità temporale**: il comando di controllo futuro $\mathbf{u}_t$ impartito tra il tempo $t-1$ e il tempo t non può viaggiare a ritroso nel tempo per influenzare lo stato antecedente $\mathbf{x}_{t-1}$:

$$p(\mathbf{x}_{t-1} \mid \mathbf{z}_{1:t-1}, \mathbf{u}_{1:t}) = p(\mathbf{x}_{t-1} \mid \mathbf{z}_{1:t-1}, \mathbf{u}_{1:t-1}) = bel(\mathbf{x}_{t-1})$$

Sostituendo all'interno dell'integrale di marginalizzazione:

$$\overline{bel}(\mathbf{x}_t) = \int_{\Omega} p(\mathbf{x}_t \mid \mathbf{x}_{t-1}, \mathbf{u}_t)\, bel(\mathbf{x}_{t-1})\, \mathbf{x}_{t-1} \quad _$$

Limiti dell'Ipotesi Di Markov E Verso I Filtri Gaussiani

L'eleganza teorica del filtro di Bayes si scontra con la realtà sperimentale, in cui l'**assunzione markoviana è quasi sempre violata**:

1. **Dinamiche Ambientali Non Modellate**:
   - La mappa dell'ambiente viene assunta statica. Ostacoli mobili (persone, porte aperte temporaneamente, altri veicoli) violano l'indipendenza condizionale delle misure inducendo correlazioni temporali spurie;
2. **Inaccuratezza e Correlazione del Rumore Sensoriale**:
   - Se il modello matematico $p(\mathbf{z}_t \mid \mathbf{x}_t)$ assume rumore bianco scorrelato mentre nella realtà gli errori di misura sono cromatici o correlati nel tempo, il filtro tende a diventare **sovra-fiducioso (*overconfident*)**, restringendo artificialmente la covarianza attorno a stime errate;
3. **Costo Computazionale**:
   - La valutazione su griglie discrete sconta la maledizione della dimensionalità (*curse of dimensionality*).

> **La Svolta Gaussiana: Il Filtro di Kalman**
>
> Se le distribuzioni di probabilità possono essere assunte rigorosamente gaussiane:
>
> $$p(\mathbf{x}_t) = \mathcal{N}(\mu_t, \Sigma_t)$$
>
> **non è più necessario valutare la densità di probabilità punto per punto nell'intero spazio!**
> Poiché una gaussiana è descritta in modo esaustivo unicamente dalla sua media μ_t e dalla sua matrice di covarianza Σ_t, il filtro di Bayes si riduce a un insieme chiuso di equazioni algebriche matriciali (le equazioni del **Filtro di Kalman**), aprendo la strada alla stima ottimale real-time per sistemi robotici complessi.

5. Localizzazione Con Histogram Filter

Nel capitolo precedente 4. Localizzazione e Filtri di Bayes abbiamo derivato la formulazione analitica del filtro di Bayes ricorsivo nello spazio continuo delle pose. Sebbene matematicamente ineccepibile, quel modello continuo pone un limite insormontabile per l'ingegnere che deve far girare il software su un calcolatore reale: l'algoritmo richiede di integrare su tutte le possibili pose pregresse e di valutare la funzione di belief su ogni singolo punto del continuo.

In questo capitolo affrontiamo la prima grande soluzione computazionale non-parametrica: l'**Histogram Filter** (noto in letteratura anche come *Markov Localization su griglia discreta*). Trasformiamo lo spazio continuo in una maglia finita di celle adiacenti, convertendo integrali intrattabili in somme discrete e rendendo la localizzazione globale e il tracciamento pienamente eseguibili in tempo reale.

Dal Filtro Continuo Alla Tassellatura Dello Spazio

Rivediamo per un istante lo pseudocodice concettuale del filtro di Bayes continuo:

$$
\begin{aligned}
&\textbf{Algorithm Bayes_Filter}(bel(\mathbf{x}_{t-1}), \mathbf{u}_t, \mathbf{z}_t):\\
&\quad \textbf{for all } \mathbf{x}_t \in \Omega \textbf{ do}\\
&\quad\quad \overline{bel}(\mathbf{x}_t) = \int_{\Omega} p(\mathbf{x}_t \mid \mathbf{x}_{t-1}, \mathbf{u}_t)\, bel(\mathbf{x}_{t-1})\, \mathrm{d}\mathbf{x}_{t-1}\\
&\quad\quad bel(\mathbf{x}_t) = \eta\, p(\mathbf{z}_t \mid \mathbf{x}_t)\, \overline{bel}(\mathbf{x}_t)\\
&\quad \textbf{end}\\
&\quad \textbf{return } bel(\mathbf{x}_t)
\end{aligned}
$$

Limite Computazionale Del Belief Continuo

Lo pseudocodice mostra con evidenza il blocco applicativo:

- L'istruzione `for all x_t` imporrebbe di iterare su un numero infinito non-numerabile di punti nello spazio continuo Ω (ad esempio $\mathbb{R}^2 \times [-\pi, \pi)$ per un robot planare a guida differenziale);
- Il passo di predizione richiede la risoluzione di un integrale su tutto lo spazio continuo Ω, risolvibile in forma chiusa solo se le distribuzioni sono rigorosamente gaussiane lineari (come vedremo con il Filtro di Kalman);
- La funzione di belief $bel(\mathbf{x}_t)$ non è un semplice valore scalare, ma una funzione che deve essere memorizzata e calcolata sull'intero dominio di supporto.

Nessun processore può eseguire un ciclo infinito né allocare una funzione arbitraria a dimensionalità infinita.

Tassellazione Dello Spazio Delle Configurazioni

L'idea fondante dell'Histogram Filter consiste nel **tassellare** (*tessellate*) lo spazio continuo delle configurazioni del robot, partizionandolo in un numero finito K di regioni disgiunte dette **celle** o **bin**:

$$
\Omega = \bigcup_{k=1}^{K} C_k, \quad \text{con } C_i \cap C_j = \emptyset \quad \forall i \neq j
$$

All'interno di ciascuna cella C_k, la densità di probabilità viene assunta costante a tratti (*piecewise constant*):

$$
p(\mathbf{x}) = \frac{p_{k,t}}{|C_k|}, \quad \forall \mathbf{x} \in C_k
$$

dove $|C_k|$ è il volume (o area in 2D, lunghezza in 1D) della cella, e $p_{k,t}$ è la probabilità complessiva che la posa reale del veicolo ricada all'interno della cella k-esima.

Sostituendo il continuo con un insieme discreto $x_1, x_2, \ldots, x_K$ di pose rappresentative (tipicamente i baricentri geometrici delle celle), il problema si riduce da uno spazio continuo infinito a uno spazio di probabilità a dimensione finita K.

Risoluzione Dei Tre Problemi Classici Di Localizzazione

Grazie alla sua natura non-parametrica, l'Histogram Filter non impone alcuna forma preconcetta alla densità di probabilità. Può rappresentare picchi multipli, valli, asimmetrie e distribuzioni totalmente piatte. Questo consente di gestire con disinvoltura tutti e tre i problemi canonici della mobilità autonoma:

- **Position Tracking**: la posa iniziale è nota con precisione limitata; il filtro mantiene un cluster compatto di celle ad alta probabilità e ne segue l'evoluzione nel tempo;
- **Global Localization**: la posa iniziale è totalmente sconosciuta; il filtro viene inizializzato con una distribuzione uniforme su tutte le K celle ($p_{k,0} = 1/K$), e fa convergere la massa di probabilità man mano che arrivano misure significative dall'ambiente;
- **Kidnapped Robot Problem**: il robot viene afferrato e teletrasportato in un punto arbitrario dell'ambiente senza ricevere comandi odometrici; le misure successive non coerenti con la predizione fanno collassare l'ipotesi precedente e generano nuovi picchi in corrispondenza della nuova posizione reale.

Formulazione Discreta Del Filtro Con Mappa

Nel filtro continuo teorico la mappa dell'ambiente è spesso sottintesa nel condizionamento. Con l'Histogram Filter introduciamo esplicitamente la **mappa** M, in quanto le coordinate delle celle e le proprietà geometriche degli ostacoli sono ancorate al reticolo metrologico definito da M.

Variabili Di Stato E Notazione Matriciale Delle Celle

All'istante temporale t, definiamo la variabile casuale discreta di stato X_t, che può assumere uno tra i valori finiti $x_1, x_2, \dots, x_K$.

Definiamo formalmente il belief discreto a posteriori e a priori della cella k-esima:

$$p_{k,t} := bel_t(x_k) := p(X_t = x_k \mid \mathbf{z}_{1:t}, \mathbf{u}_{1:t}, M)$$

$$p_{k,t} := \overline{bel}_t(x_k) := p(X_t = x_k \mid \mathbf{z}_{1:t-1}, \mathbf{u}_{1:t}, M)$$

con il vincolo fondamentale di normalizzazione su tutto l'insieme discreto:

$$\sum_{k=1}^{K} bel_t(x_k) = 1, \quad bel_t(x_k) \geq 0 \quad \forall k$$

> **Significato della Variabile x_k**
>
> Nelle formule discrete, x_k non indica la posizione vera del robot (che rimane ignota), ma una specifica **ipotesi di posa** (il punto rappresentativo della cella k) che il robot sta valutando all'interno del ciclo iterativo di stima.

Modello Di Movimento Discreto

La probabilità di transizione di stato continua $p(\mathbf{x}_t \mid \mathbf{x}_{t-1}, \mathbf{u}_t)$ viene convertita nella matrice di transizione tra coppie di celle:

$$p(X_t = x_k \mid X_{t-1} = x_i, \mathbf{u}_t, M)$$

Questa quantità risponde a una domanda operativa chiarissima: se all'istante precedente il robot si trovava nella cella x_i e applichiamo l'azione di movimento $\mathbf{u}_t$, qual è la probabilità che il veicolo finisca all'interno della cella x_k?

Nella pratica software, questa probabilità viene calcolata interrogando direttamente il nostro modello di movimento stocastico:

$$p(X_t = x_k \mid X_{t-1} = x_i, \mathbf{u}_t, M) = \mathrm{motion_model}(x_k, x_i, \mathbf{u}_t)$$

Modello Di Misura E Verosimiglianza

Il secondo ingrediente fondamentale è la **verosimiglianza della misura** (*measurement likelihood*):

$$p(\mathbf{z}_t \mid X_t = x_k, M) = \mathrm{measurement_model}(\mathbf{z}_t, x_k, M)$$

Essa quantifica la plausibilità di osservare la lettura sensoriale $\mathbf{z}_t$ condizionatamente all'ipotesi che il robot si trovi esattamente nella cella x_k, tenendo conto della disposizione geometrica degli elementi noti della mappa M (pareti, porte, beacon radio).

Algoritmo Dell'Histogram Filter

Possiamo ora scrivere l'algoritmo completo dell'Histogram Filter sia in notazione probabilistica compatta che in forma procedurale di alto livello.

Passo Di Predizione E Convoluzione Discreta

Per calcolare il belief a priori $\overline{bel}_t(x_k)$ per ciascuna cella x_k, applichiamo la versione discreta del teorema della probabilità totale, sostituendo l'integrale continuo con la sommatoria su tutte le K possibili celle di provenienza x_i:

$$\overline{bel}_t(x_k) = \sum_{i=1}^{K} p(X_t = x_k \mid X_{t-1} = x_i, \mathbf{u}_t, M) \cdot bel_{t-1}(x_i)$$

Geometricamente, questa operazione corrisponde a una **convoluzione discreta** tra la distribuzione precedente e il kernel di incertezza introdotto dal comando di attuazione:

- Il baricentro della distribuzione trasla nella direzione imposta dal comando $\mathbf{u}_t$;
- La varianza complessiva aumenta, poiché il rumore degli attuatori disperde la probabilità sulle celle limitrofe, appiattendo progressivamente i picchi.

Passo Di Correzione E Normalizzazione Bayesiana

Non appena il robot acquisisce la misura sensoriale $\mathbf{z}_t$, aggiorniamo puntualmente la probabilità di ciascuna cella moltiplicando la predizione a priori per la verosimiglianza della misura corrente:

$$bel_t(x_k) = \eta\, p(\mathbf{z}_t \mid X_t = x_k, M) \cdot \overline{bel}_t(x_k)$$

Il fattore di normalizzazione η garantisce che la somma delle probabilità sull'intero reticolo rimanga rigorosamente unitaria:

$$\eta = \frac{1}{\sum_{k=1}^{K} p(\mathbf{z}_t \mid X_t = x_k, M) \cdot \overline{bel}_t(x_k)}$$

Struttura Algoritmica E Confronto Formale

Riassumiamo la procedura algoritmica implementata sui sistemi di bordo:

```
# Algoritmo procedurale dell'Histogram Filter
def histogram_filter(bel_prev, u_t, z_t, M):
    bel_bar = {}
    bel = {}

    # 1. Passo di Predizione (Total Probability Theorem)
    for x_k in all_cells:
        bel_bar[x_k] = 0.0
        for x_i in all_cells:
            prob_trans = motion_model(x_k, x_i, u_t, M)
            bel_bar[x_k] += prob_trans * bel_prev[x_i]

    # 2. Passo di Correzione (Bayes Rule)
    eta_inv = 0.0
    for x_k in all_cells:
        likelihood = measurement_model(z_t, x_k, M)
        bel[x_k] = likelihood * bel_bar[x_k]
        eta_inv += bel[x_k]
```

```
    # 3. Normalizzazione
    for x_k in all_cells:
        bel[x_k] /= eta_inv

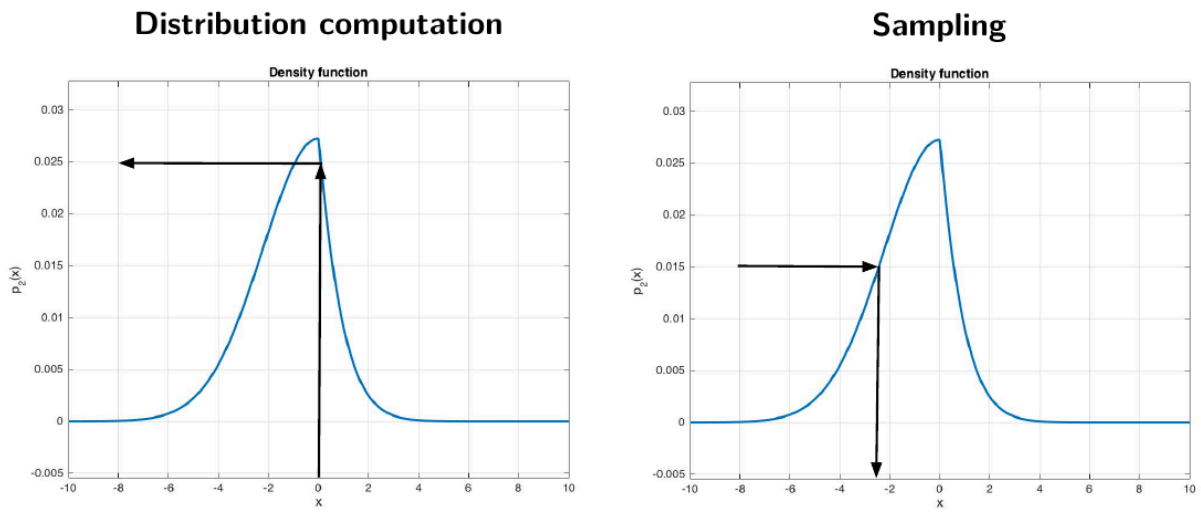
    return bel
```

L'algoritmo opera in tempo polinomiale rispetto al numero di celle K:

- La predizione valuta una doppia iterazione su tutte le coppie (x_k, x_i), comportando una complessità temporale pari a $O(K^2)$;
- La correzione e la normalizzazione scorrono il reticolo una sola volta per cella, con complessità pari a $O(K)$;
- Per approfondire come questa operazione si esprima elegantemente in algebra lineare tramite matrici colonna-stocastiche e vettori di probabilità, si rimanda a zz. Argomenti extra: Formulazione Matriciale dell'Histogram Filter.

Calcolo Della Distribuzione Vs Campionamento Stocastico

Un passaggio didattico cardine chiarito a lezione riguarda la netta distinzione concettuale e algoritmica tra due operazioni probabilistiche che spesso vengono confuse: la **valutazione della densità di distribuzione** (*distribution computation*) e l'**estrazione di campioni** (*sampling*).



Esaminiamo in dettaglio cosa accade nei due grafici:

Distribution Computation Nei Filtri A Griglia

Nel grafico di sinistra (*Distribution computation*), abbiamo una variabile nota (ad esempio una specifica posa ipotetica x_k o una discrepanza geometrica $\coloneqq x_k - x_i - u_x$) e vogliamo determinarne l'esatto valore di probabilità o densità:

- Si entra sull'asse orizzontale delle ascisse con il valore numerico x;
- Si risale verticalmente fino a intersecare la curva analitica $p(x)$;
- Si proietta il punto sull'asse verticale delle ordinate leggendo il valore di densità $p(x)$.

Questo è esattamente il meccanismo impiegato dall'Histogram Filter: i punti x_k sono fissati deterministicamente dai baricentri delle celle, e il filtro valuta la funzione analitica (ad esempio la curva gaussiana) per popolare i pesi di transizione e di verosimiglianza.

Sampling Stocastico Nei Metodi Monte Carlo

Nel grafico di destra (*Sampling*), il flusso operativo è opposto:

- Non conosciamo a priori il valore x, ma disponiamo della curva di densità $p(x)$;
- Generiamo un numero casuale uniforme $u \in [0, 1]$ (sull'asse delle ordinate della funzione cumulativa CDF);
- Tramite inversione della cumulativa o algoritmi di trasformazione (es. Box-Muller, rejection sampling), scendiamo sull'asse delle ascisse ricavando un campione numerico concreto $x \sim p(x)$.

Questo secondo approccio è il cuore pulsante dei **Filtri a Particelle (Particle Filters)** e degli algoritmi di propagazione Monte Carlo visti nei modelli di movimento stocastici.

Progettazione Di Un Histogram Filter 1D

Per comprendere a fondo la dinamica del filtro senza l'appesantimento geometrico dell'orientamento angolare, progettiamo un Histogram Filter per un veicolo mobile che si sposta lungo un corridoio rettilineo monodimensionale.

Modello Cinematico E Rumore Odometrico

Il robot si muove lungo l'asse $x \in [x_{\min}, x_{\max}]$. Il comando di attuazione è una traslazione richiesta $u = u_x$.

Il moto reale è affetto da un disturbo stocastico additivo gaussiano a media nulla:

$$x_t = x_{t-1} + u_x + \varepsilon_x \quad \varepsilon_x \sim \mathcal{N}(0, \sigma_x^2)$$

Date due celle ipotetiche con coordinate x_k (arrivo) e $x_{t-1} = x_i$ (partenza), l'errore cinematico corrispondente a questa transizione vale:

$$\varepsilon_x = x_k - (x_i + u_x)$$

Poiché il rumore segue una normale $\mathcal{N}(0, \sigma_x^2)$, la probabilità di transizione di stato tra le due celle si calcola valutando la densità gaussiana sull'errore ε_x:

$$p(X_t = x_k \mid X_{t-1} = x_i, u_x) = \text{pdf}_{\mathcal{N}}(\varepsilon_x; 0, \sigma_x^2) = \frac{1}{\sqrt{2\pi\sigma_x^2}} \exp\left(-\frac{(x_k - x_i - u_x)^2}{2\sigma_x^2} \right)$$

Modello Di Misura Da Beacon E Distanza Radiale

L'ambiente contiene punti di riferimento fissi a coordinate note detti **beacon** (es. transponder a radiofrequenza o marker ottici, come descritto in 2. Sensori per la Robotica Mobile).

Il robot dispone di un sensore di distanza radiale (*range sensor*) che misura la distanza scalare rispetto a un beacon situato in posizione nota m_1:

$$z_t = \|x_t - m_1\| + \nu_z = |x_t - m_1| + \nu_z \quad \nu_z \sim \mathcal{N}(0, \sigma_z^2)$$

Valutando la misura sull'ipotesi che il robot si trovi nella cella x_k, la distanza teorica attesa vale $|x_k - m_1|$. L'errore residuo sensoriale è:

$$\nu_z = z_t - |x_k - m_1|$$

La funzione di verosimiglianza di misura risulta quindi:

$$p(z_t \mid X_t = x_k, m_1) = \text{pdf}_{\mathcal{N}}(\nu_z; 0, \sigma_z^2) = \frac{1}{\sqrt{2\pi\sigma_z^2}} \exp\left(-\frac{(z_t - |x_k - m_1|)^2}{2\sigma_z^2} \right)$$

Analisi Della Verosimiglianza: Bimodalità Con Singolo Beacon

Prima di avviare qualsiasi simulazione numerica, esaminiamo criticamente la forma analitica della funzione di verosimiglianza $p(z_t \mid x_k, m_1)$.

Poiché la metrica di distanza radiale in 1D contiene il modulo $|x_k - m_1|$, l'errore si annulla ogniqualvolta la distanza ipotetica uguaglia la misura:

$$|x_k - m_1| = z_t \implies x_k - m_1 = z_t \implies x_k = m_1 \pm z_t$$

> **L'Ambiguità Speculare del Singolo Beacon**
>
> Un sensore che misura unicamente il modulo della distanza in 1D non è in grado di distinguere se il robot si trova a sinistra o a destra del beacon.
> Di conseguenza, la funzione di verosimiglianza associata a un singolo beacon è rigorosamente **bimodale** (ha due picchi simmetrici uguali centrati in $m_1 - z_t$ e $m_1 + z_t$).
> Se il robot parte con incertezza globale uniforme, una singola lettura non potrà mai localizzarlo univocamente: produrrà due ipotesi speculari identiche.

Disambiguazione Globale Tramite Beacon Multipli

Per risolvere l'ambiguità geometrica, introduciamo nella mappa un secondo beacon posizionato in $m_2 \neq m_1$.

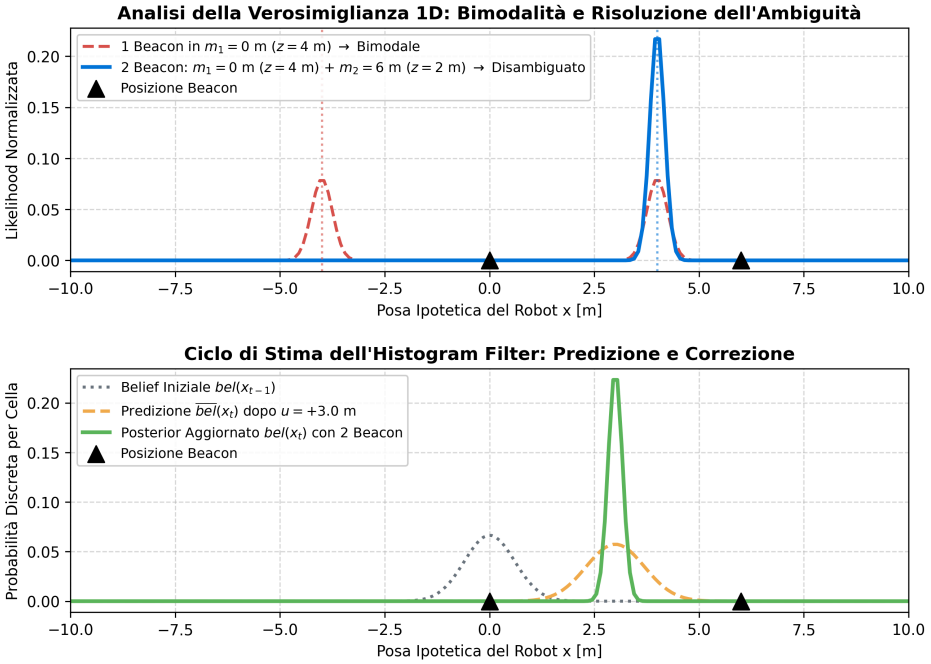
Supponendo che i rumori di misura dei sensori siano reciprocamente scorrelati (indipendenza condizionale delle misure data la posa), la funzione di verosimiglianza congiunta si ottiene moltiplicando le singole verosimiglianze:

$$p(\mathbf{z}_t \mid X_t = x_k, M) = p(z_{t,1} \mid x_k, m_1) \cdot p(z_{t,2} \mid x_k, m_2)$$

Ciascun beacon genera una coppia di picchi simmetrici rispetto alla propria coordinata:

- Il beacon 1 produce picchi a $x \in \{m_1 - z_{t,1},\ m_1 + z_{t,1}\}$;
- Il beacon 2 produce picchi a $x \in \{m_2 - z_{t,2},\ m_2 + z_{t,2}\}$.

Poiché il veicolo fisico si trova in un'unica posizione reale x, esattamente uno dei due picchi del primo beacon coinciderà con uno dei due picchi del secondo beacon (quello in cui $x \approx m_1 \pm z_{t,1} \approx m_2 \pm z_{t,2}$). Negli altri punti distanti, il prodotto tra un valore esponenzialmente piccolo e un picco vicino a zero abbatte la probabilità residua: l'ambiguità collassa e il belief diventa marcatamente **unimodale**.



La figura sopra illustra con limpidezza questo fenomeno fisico:

- Nel pannello superiore, la linea tratteggiata rossa (singolo beacon in $m_1 = 0\ \text{m}$ con misura $z = 4\ \text{m}$) mostra i due picchi simmetrici a $-4\ \text{m}$ e $+4\ \text{m}$;
- Aggiungendo il secondo beacon in $m_2 = 6\ \text{m}$ (linea blu continua), il picco spuro a sinistra scompare completamente e sopravvive solo la reale posizione del veicolo a $+4\ \text{m}$;
- Nel pannello inferiore, seguiamo l'intero ciclo del filtro: il belief a priori (arancione) si allarga durante il movimento a causa del rumore di attuazione, ma l'aggiornamento di misura con i due beacon (verde) comprime l'incertezza restituendo una stima ad altissima accuratezza.

Analisi Di Sensibilità Rispetto Alla Risoluzione Spaziale

La scelta della dimensione della cella Δx è il parametro di compromesso principale dell'Histogram Filter:

- **Risoluzione fine (Δx piccolo)**:
  - L'errore di quantizzazione spaziale decade proporzionalmente a Δx;
  - La rappresentazione delle PDF è estremamente fedele;
  - Il numero di celle $K = (x_{\max} - x_{\min})/\Delta x$ cresce linearmente, e il tempo di calcolo della predizione cresce quadraticamente come $O(K^2) = O(1/\Delta x^2)$.
- **Risoluzione grossolana (Δx grande)**:
  - Esecuzione rapidissima con poche celle;
  - Se Δx $\gg \sigma_x$, la discretizzazione introduce un pesante rumore di quantizzazione deterministico che degrada le prestazioni e maschera l'incertezza del sensore.

> **Regola Empirica per la Scelta di Δx**
>
> Per evitare distorsioni da aliasing numerico nella convoluzione, la larghezza della cella Δx deve essere scelta tale da campionare adeguatamente la deviazione standard del rumore di transizione:
>
> $$\Delta x \leq \frac{\sigma_x}{2}$$
>
> garantendo che il kernel gaussiano occupi almeno 4-5 celle nella sommatoria discreta.

Limiti Computazionali E Transizione Verso Spazi 2D

L'Histogram Filter 1D ha mostrato la semplicità e la robustezza del metodo su griglia. Cosa accade tuttavia quando passiamo al problema reale di localizzazione planare di un veicolo terrestre su un pavimento?

La Maledizione Della Dimensionalità In SE(2)

Nel piano orizzontale, la posa del robot è un vettore tridimensionale $\mathbf{x} = [x, y, \theta]^T \in SE(2)$.

Se discretizziamo ciascuna dimensione spaziale e angolare con un numero moderato di intervalli:

- $N_x = 1000$ celle lungo l'asse x (es. corridoio di $50\ \mathrm{m}$ con risoluzione $5\ \mathrm{cm}$);
- $N_y = 1000$ celle lungo l'asse y (es. corridoio di $50\ \mathrm{m}$ con risoluzione $5\ \mathrm{cm}$);
- $N_\theta = 360$ celle per l'orientamento angolare (risoluzione di 1°);

il numero totale di celle discrete nel reticolo 3D vale:

$$K = N_x \times N_y \times N_\theta = 1000 \times 1000 \times 360 = 3.6 \times 10^{\text{celle}}$$

Memorizzare un belief a 360 milioni di float a 64 bit richiede circa $2.8\ \mathrm{GB}$ di RAM. Peggio ancora, eseguire il passo di predizione tramite convoluzione naïve richiederebbe $K^2 \approx 1.3 \times 10^{17}$ operazioni aritmetiche per ciascun ciclo di controllo, un calcolo del tutto proibitivo per qualunque calcolatore embedded.

Questo collo di bottiglia è noto come **maledizione della dimensionalità** (*curse of dimensionality*).

Prospettive E Confronto Tra Famiglie Di Filtri

Per superare la maledizione della dimensionalità della griglia senza perdere la capacità di gestire sistemi complessi, la robotica moderna ha intrapreso due direzioni complementari:

1. **Filtri Parametrici Gaussiani (Kalman Filter, EKF, UKF)**: rinunciano alla forma arbitraria assumendo densità rigorosamente unimodali; non memorizzano griglie ma solo media $\mu \in \mathbb{R}^3$ e covarianza $\Sigma \in \mathbb{R}^{3\times3}$, abbattendo la complessità a $O(1)$;
2. **Filtri Non-Parametrici Monte Carlo (Particle Filters / MCL)**: invece di tassellare l'intero spazio di stato esaustivamente, campionano solo le regioni in cui la probabilità è significativamente non nulla tramite un insieme di M particelle pesate ($M \approx 10^2 \div 10^4$), combinando la flessibilità dell'istogramma con l'efficienza computazionale.

Nei prossimi capitoli estenderemo il modello al piano 2D e introdurremo la trattazione formale del Filtro di Kalman esteso.

6. Localizzazione Con Particle Filter

Nel capitolo precedente abbiamo affrontato la prima discretizzazione computazionale del filtro di Bayes: l'Histogram Filter. Sebbene la decomposizione dello spazio in una griglia fissa permetta di superare l'intrattenibilità dell'integrale continuo, presenta limiti severi quando passiamo da scenari monodimensionali giocattolo alla reale cinematica nel piano $SE(2) = \mathbb{R}^2 \times S^1$.

In questo capitolo introduciamo il **Particle Filter** (noto in robotica come **Monte Carlo Localization** o **MCL**), che abbandona la tassellazione rigida dello spazio continuo per rappresentare la densità di probabilità tramite un insieme discreto di campioni casuali.

Dal Limite Dell'Histogram Filter Alla Rappresentazione A Particelle

L'approccio su griglia discreta richiede di allocare in memoria e aggiornare a ogni passo temporale tutte le possibili celle dello spazio di stato.

In uno spazio 1D con segmento $L = 20\ \mathrm{m}$ e risoluzione $\Delta x = 0.1\ \mathrm{m}$, dobbiamo gestire $K = 200$ celle. Tuttavia, per un robot mobile con posa $\mathbf{x} = [x, y, \theta]^T$ in un'arena di $20\ \mathrm{m} \times 20\ \mathrm{m}$ discretizzata a $0.05\ \mathrm{m}$ e risoluzione angolare di 1° (360 bin), il numero totale di celle esplode a:

$$K = \left(\frac{20}{0.05}\right) \times \left(\frac{20}{0.05}\right) \times 360 = 400 \times 400 \times 360 \approx 5.76 \times 10^7 \text{ celle}$$

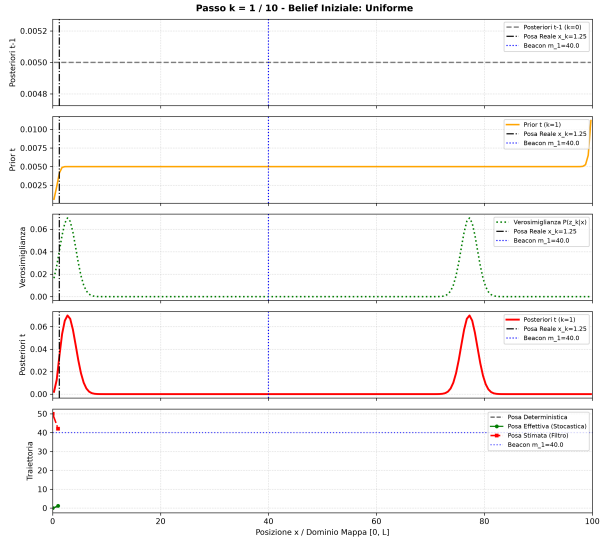
Calcolare la convoluzione cinematica $O(K^2)$ su quasi 60 milioni di stati a frequenze di controllo ordinarie (10--$50\ \mathrm{Hz}$) è impensabile. Inoltre, la stragrande maggioranza di queste celle possiede probabilità identicamente nulla, costringendo la CPU a sprecare potenza di calcolo su porzioni di spazio in cui il robot non potrà mai trovarsi.

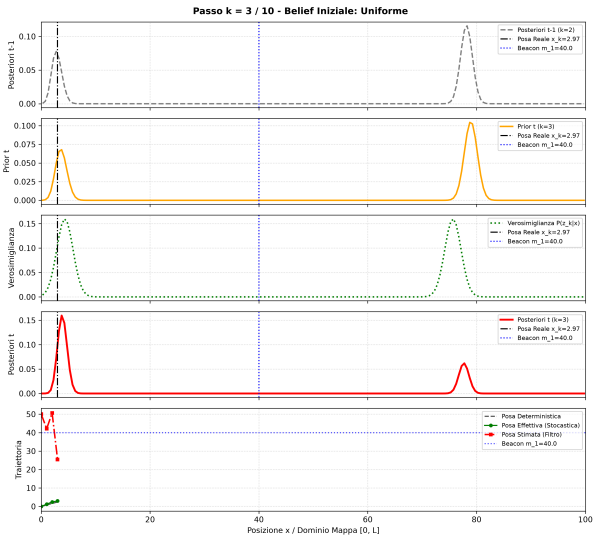
Richiamiamo la struttura a tre blocchi concettuali del filtro di Bayes evidenziata dal docente:

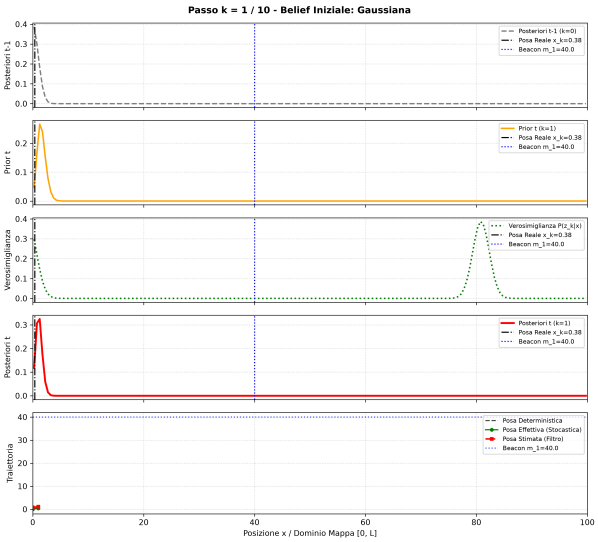
1. Un ciclo computazionale che scorre le possibili pose ipotizzate nello spazio;
2. Il calcolo del belief a priori (fase di predizione basata su sensori **propriocettivi**, come encoder di ruota, odometria e giroscopi/IMU);
3. Il calcolo del belief a posteriori (passo di correzione basato su sensori esterocettivi, come laser scanner, telecamere o beacon).

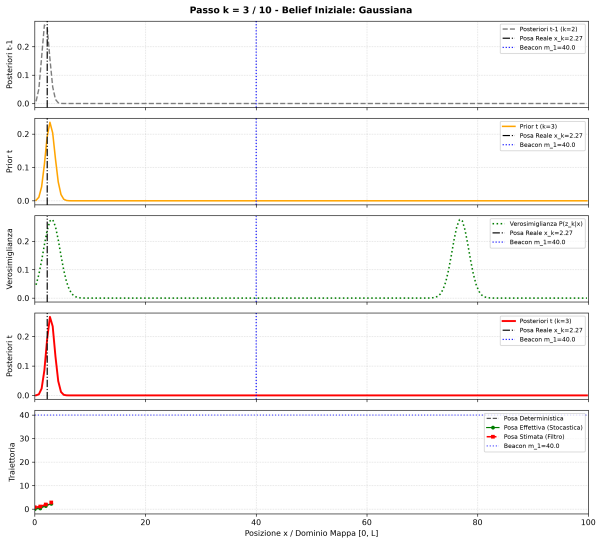
Questa architettura predizione-correzione è lo standard universale della robotica mobile e della sensor fusion per stimare lo stato a partire dalla combinazione di informazioni **propriocettive** (moto interno del veicolo) ed **esterocettive** (misura dell'ambiente esterno).

Rivediamo le simulazioni 1D dell'Histogram Filter: con prior uniforme il filtro impiega svariati passi di misura per collassare, mentre con prior gaussiano l'incertezza iniziale viene rapidamente ridotta ma rimane vincolata alla risoluzione della griglia Δx.









Se invece di integrare analiticamente la griglia campionassimo la curva bidentata di verosimiglianza generata dal beacon (la curva rossa nei grafici di misura), i campioni stocastici formerebbero naturalmente due distinte nuvolette di punti concentrate attorno ai due massimi simmetrici locali.

L'idea alla base del Particle Filter è capovolgere la prospettiva: invece di campionare rigidamente lo spazio geometrico, campioniamo direttamente la distribuzione di probabilità. Il Particle Filter riesce infatti nell'intento fondamentale di generare campioni di una variabile aleatoria pur non conoscendo in forma analitica chiusa la distribuzione a posteriori di arrivo.

Rappresentiamo la funzione di belief continuo $bel(x_t)$ tramite una nube finita di M ipotesi stocastiche, dette **particelle**, raggruppate nell'insieme indicato con $\mathcal{X}_t$ (X calligrafico):

$$\mathcal{X}_t = x_t^{[1]}, x_t^{[2]}, \ldots, x_t^{[M]},$$

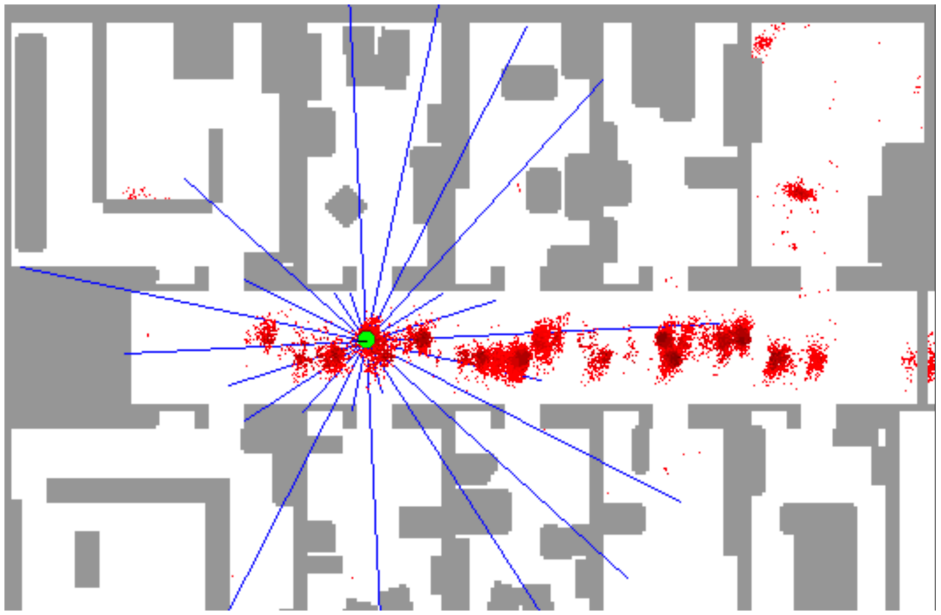
Ciascuna particella $x_t^{[m]} \in \mathcal{X}_t$ rappresenta una posa concreta e puntiforme del robot nello spazio continuo di configurazione:

$$x_t^{[m]} = \begin{bmatrix} x_t^{[m]} \\ y_t^{[m]} \\ \theta_t^{[m]} \end{bmatrix} \in SE(2)$$

La densità spaziale delle particelle approssima la densità di probabilità: nelle regioni dello spazio in cui la probabilità della posa è elevata avremo una concentrazione densa di campioni, mentre nelle regioni a probabilità trascurabile non ci sarà alcuna particella. Il consumo di memoria e il carico di calcolo scalano con la complessità dell'incertezza e con il numero di campioni M, non con il volume dello spazio di stato.

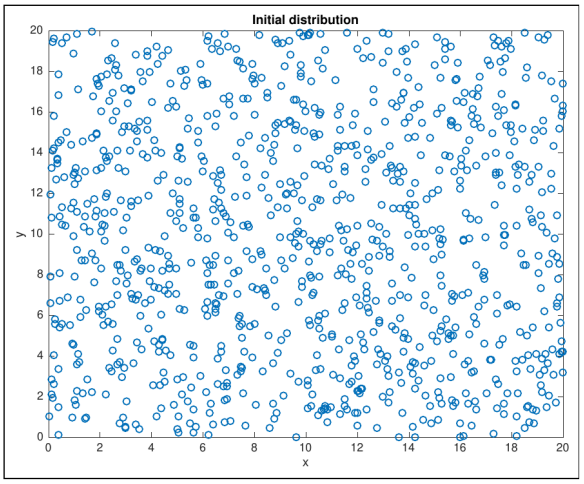
Panoramica Visiva Ed Evoluzione Nel Piano 2D

Il video sperimentale classico di Sebastian Thrun a Stanford mostra un robot mobile dotato di LiDAR che deve localizzarsi all'interno di un vasto edificio universitario a partire da una condizione di totale incertezza iniziale.



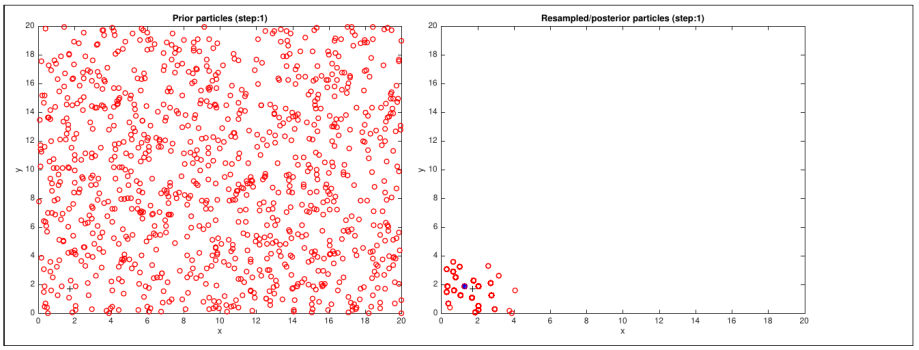
I raggi laser rossi intersecano le pareti dell'edificio, mentre i punti blu rappresentano le particelle distribuite nello spazio. All'inizio della missione il robot affronta il problema della **Global Localization**: non ha alcuna informazione a priori su dove si trovi, per cui le particelle vengono distribuite con densità uniforme all'interno di tutta la planimetria nota.

Consideriamo l'esperimento didattico in un piano delimitato $[0, 20]\ \mathrm{m} \times [0, 20]\ \mathrm{m}$. All'istante iniziale $t = 0$ generiamo M campioni uniformi:

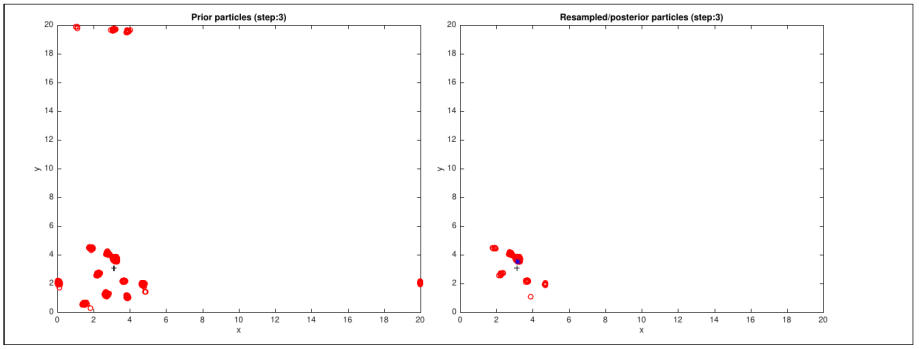


Non appena il robot esegue un comando di movimento e acquisisce le letture dai sensori (ad esempio distanze da beacon noti), il filtro attraversa una rapida sequenza di convergenza.

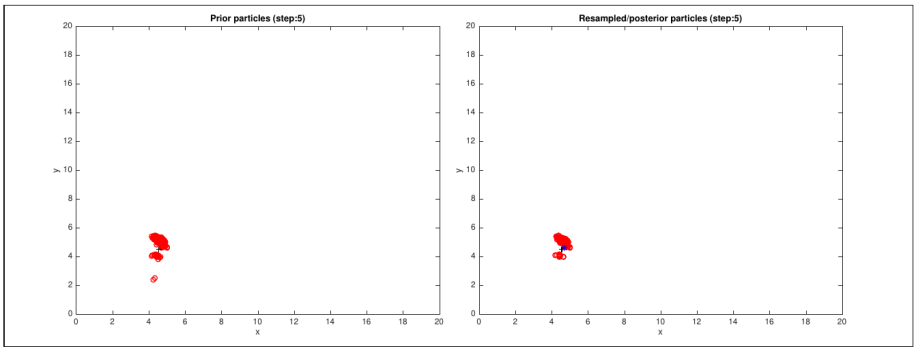
Allo **Step 1**, il robot si muove verso destra e misura la distanza da un beacon. A sinistra osserviamo le particelle a priori propagate dal modello di moto; a destra, il ricampionamento basato sui pesi di misura seleziona solo le particelle coerenti con la lettura sensoriale, formando un anello circolare di verosimiglianza:



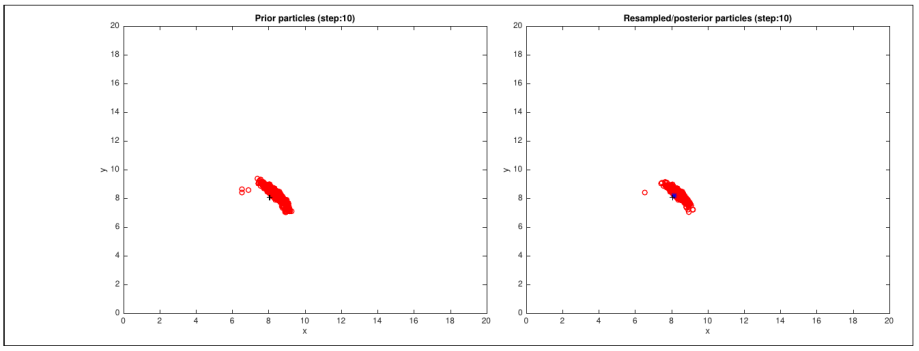
Allo **Step 3**, dopo ulteriori comandi di moto e nuove misure di distanza, le particelle a priori si disperdono lungo la direzione di avanzamento, ma il passo di correzione concentra i campioni unicamente nelle regioni di intersezione tra le varie osservazioni:



Allo **Step 5**, l'ambiguità multimodale è completamente risolta. La nube di particelle collassa in un unico agglomerato compatto che segue fedelmente la vera posizione del veicolo:

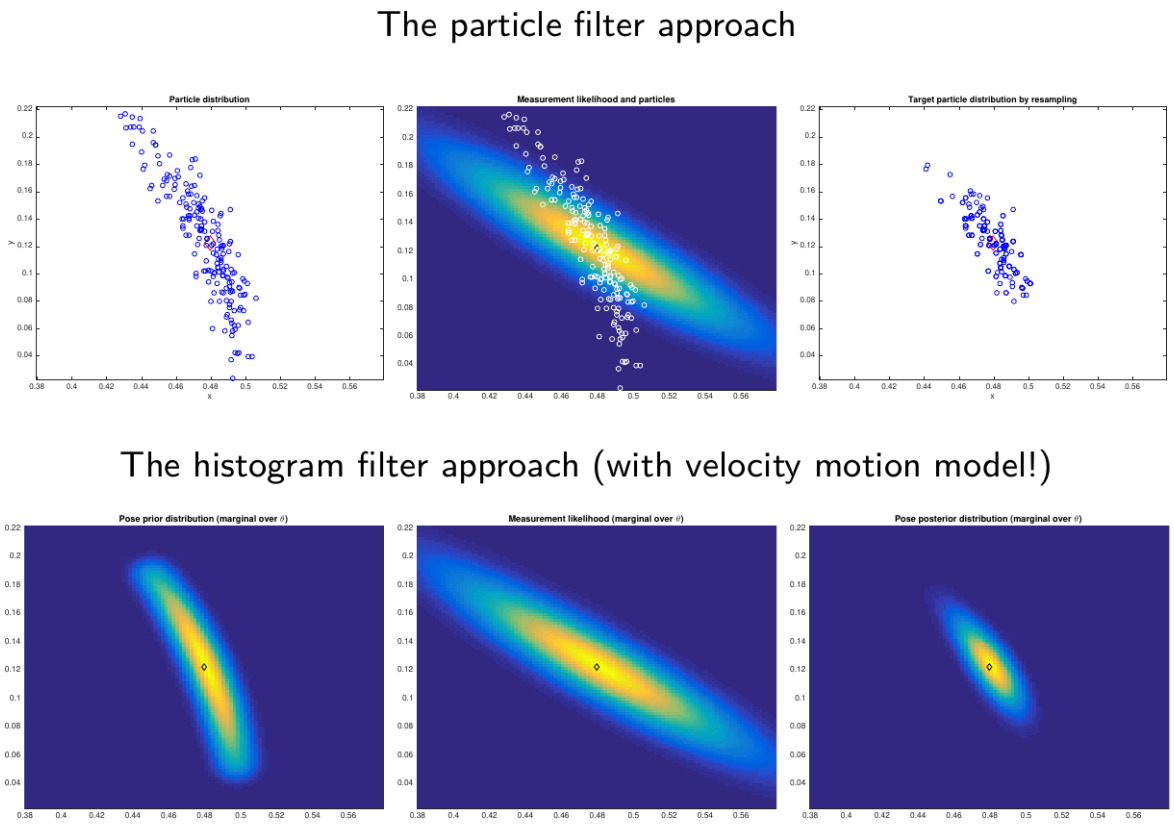


Allo **Step 10**, il filtro è transitato stabilmente dalla fase di localizzazione globale al regime di **Position Tracking**: le particelle restano coese attorno alla posa reale del robot, compensando in tempo reale il drift odometrico grazie alle continue correzioni sensoriali:

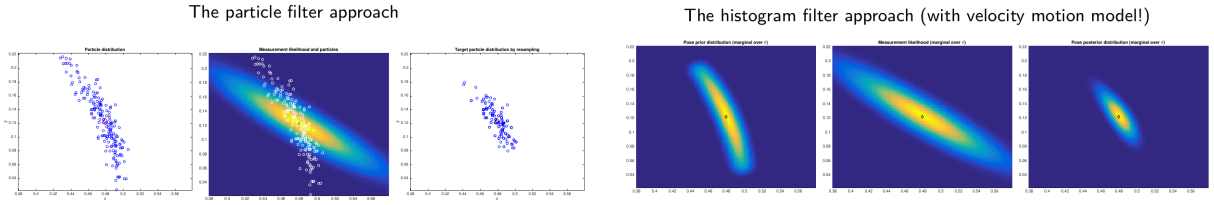


Architettura Del Filtro: Ciclo Di Predizione, Correzione E Ricampionamento

La differenza concettuale tra la Markov Localization su griglia e il Particle Filter è sintetizzata nello schema di confronto presentato dal docente:



Possiamo analizzare in dettaglio le due pipeline operative affiancate:



Nel caso dell'**Histogram Filter** (pannello inferiore):

1. Nel primo grafico la curva rappresenta la densità di probabilità continua calcolata dal *Velocity Motion Model*;
2. Si valuta la likelihood della misura su ciascuna cella della mappa;
3. Si calcola il prodotto puntuale normalizzato per ottenere la posterior su griglia discreta.

Nel caso del **Particle Filter** (pannello superiore):

1. Nel primo grafico i campioni rappresentano tutte le possibili pose candidate ipotizzate dal robot. A parità di comando di controllo u_t, ogni particella si sposta in modo leggermente diverso nello spazio, perché il moto è governato da un modello stocastico che vi aggiunge rumore pseudo-casuale indipendente;
2. Si assegna a ciascuna particella un **fattore di importanza** (peso $w_t^{[m]}$) proporzionale alla likelihood della misura valutata su quella specifica posa;
3. Si esegue il **ricampionamento stocastico con rimpiazzo** (*resampling*), generando la popolazione a posteriori in cui le particelle con peso maggiore vengono duplicate e quelle con peso trascurabile eliminate, preparando la popolazione per il passo temporale successivo.

La Dicotomia Operativa di Ciascuna Particella: Sampling e Distribution Computation

Ogni singola particella vive una duplice natura operativa nel ciclo di stima:

1. **Sampling**: nella fase di predizione la particella si sposta nello spazio continuo campionando una realizzazione stocastica dal modello di moto;
2. **Distribution Computation**: nella fase di correzione la particella riceve il proprio peso calcolando deterministicamente il valore puntuale della funzione di densità di verosimiglianza del sensore (likelihood).

Inoltre, mentre nell'Histogram Filter eravamo costretti a calcolare analiticamente la probabilità di transizione $p(x_t \mid u_t, x_{t-1})$ per ogni coppia di celle, nel Particle Filter non serve alcuna forma chiusa: basta una procedura generativa (`sample_motion_model`) capace di simulare numericamente il passo di moto.

Fondamenti Matematici Dell'Importance Sampling

Il cuore teorico del Particle Filter risiede nel principio dell'**Importance Sampling**.

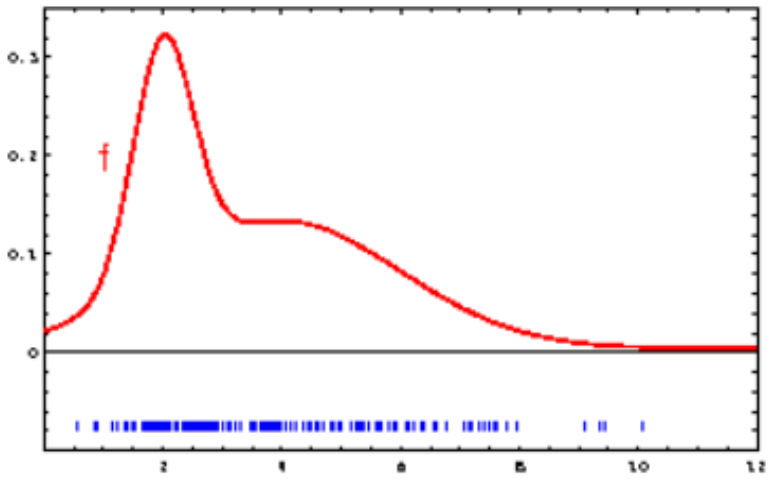
Nel filtro di Bayes ideale vorremmo estrarre campioni direttamente dalla distribuzione a posteriori:

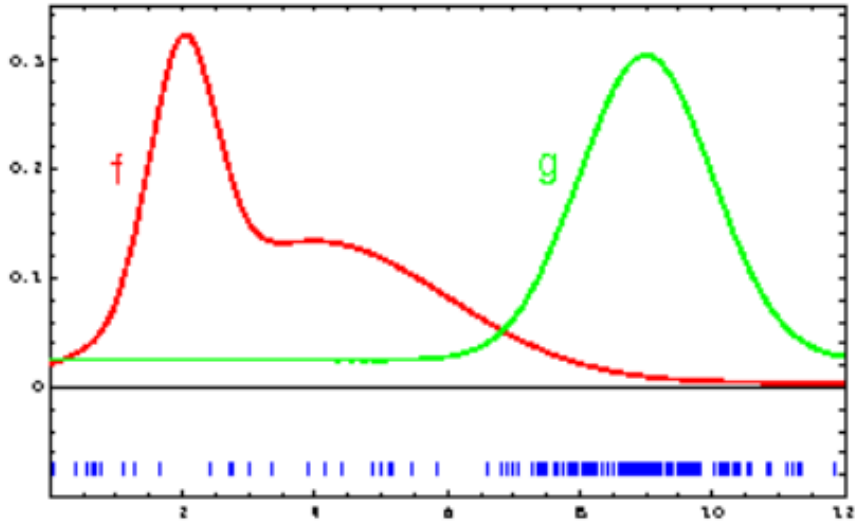
$$f(x_t) = bel(x_t) = p(x_t \mid z_{1:t}, u_{1:t})$$

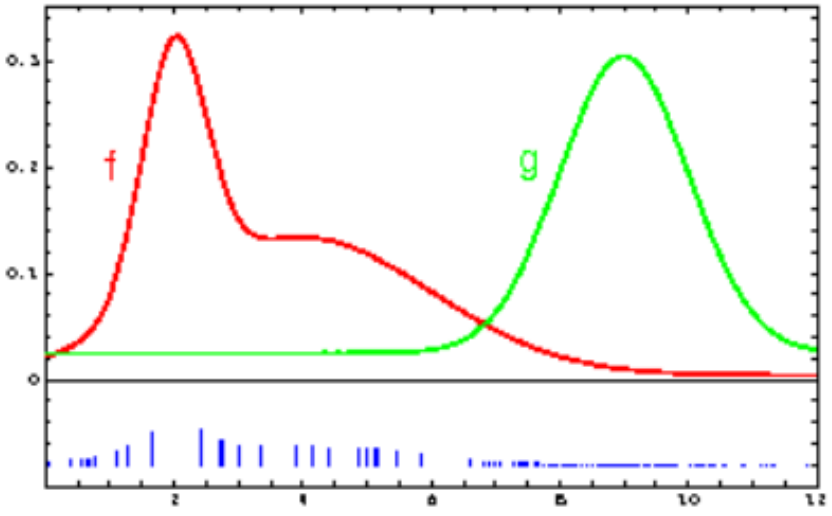
Tuttavia, la distribuzione target $f(x_t)$ non è nota a priori in forma analitica e non possediamo alcun algoritmo in grado di estrarre campioni direttamente da essa.

Possiamo invece generare agevolmente campioni casuali da una distribuzione ausiliaria correlata, detta **proposal distribution** (o densità di proposta) $g(x_t)$, data dalla distribuzione di predizione del moto:

$$g(x_t) = \overline{bel}(x_t) = p(x_t \mid z_{1:t-1}, u_{1:t})$$







Come illustrato nello schema a tre livelli:

1. Abbiamo una funzione obiettivo target $f(x)$ che vorremmo approssimare;
2. Estraiamo M campioni casuali indipendenti dalla densità proposal $g(x)$;
3. Per correggere la discrepanza statistica tra la proposta g e il target f, associamo a ciascun campione un fattore moltiplicativo di peso:

$$w(x) = \frac{f(x)}{g(x)}$$

Consideriamo il calcolo del valore atteso di una qualsiasi funzione test o indicatrice $\mathbb{1}(x \in \cdot)$ rispetto alla distribuzione target f:

$$\mathbb{E}_{f}[\mathbb{1}(x \in \cdot)] = \int f(x)\, \mathbb{1}(x \in \cdot)\, dx$$

Moltiplicando e dividendo per la densità di proposta $g(x)$ (sotto l'ipotesi di supporto comune $f(x) > 0 \implies g(x) > 0$):

$$\mathbb{E}_{f}[\mathbb{1}(x \in \cdot)] = \int \frac{f(x)}{g(x)}\, g(x)\, \mathbb{1}(x \in \cdot)\, dx = \int w(x)\, g(x)\, \mathbb{1}(x \in \cdot)\, dx = \mathbb{E}_{g}[w(x)\, \mathbb{1}(x \in \cdot)]$$

Stimatore Standard Vs Self-Normalized Importance Sampling (SNIS)

Distinguiamo la formulazione statistica ideale dal caso applicativo della stima bayesiana robotica:

1. **Importance Sampling Standard (Densità Note e Normalizzate)**: se la distribuzione target $f(x)$ e la proposta $g(x)$ sono entrambe note in forma chiusa e integrate a 1, il peso analitico esatto $w(x) = f(x)/g(x)$ ha valore atteso unitario sotto la proposta ($\mathbb{E}_{g}[w(x)] = \int f(x)\, dx = 1$). In questo caso lo stimatore Monte Carlo non polarizzato è la semplice media campionaria, priva di costanti moltiplicative esterne:

$$\frac{1}{M} \sum_{m=1}^{M} w\left(x^{[m]}\right) \mathbb{1}\left(x^{[m]} \in \cdot\right) \overset{M \to \infty}{\longrightarrow} \int f(x)\, dx$$

2. **Stima Bayesiana e Costante di Normalizzazione Incognita**: nella localizzazione robotica la distribuzione target a posteriori è nota unicamente a meno del fattore scalare di normalizzazione $\eta = 1/p(z_t)$:

$$f(x_t) = bel(x_t) = \eta\, p(z_t \mid x_t)\, \overline{bel}(x_t)$$

   Non possiamo quindi calcolare il rapporto esatto $w(x_t) = f(x_t)/g(x_t)$. Scegliendo la densità di proposta pari alla predizione odometrica $g(x_t) = \overline{bel}(x_t)$, calcoliamo invece i **pesi grezzi non normalizzati** come la sola verosimiglianza di misura:

$$w_t^{[m]} = p(z_t \mid x_t^{[m]})$$

   Poiché la costante incognita a denominatore è il valore atteso della verosimiglianza $\eta^{-1} = \int p(z_t \mid x_t)\, \overline{bel}(x_t)\, dx_t = \mathbb{E}_{g}[w_t]$, possiamo stimarla empiricamente tramite la media campionaria dei pesi grezzi: $\eta^{-1} \approx \frac{1}{M} \sum_{j=1}^{M} w_t^{[j]}$.
3. **Stimatore Self-Normalized (SNIS) con Pesi Normalizzati**: sostituendo la stima della costante di normalizzazione a denominatore, otteniamo lo stimatore campionario asintoticamente corretto con i **pesi normalizzati** $w_t^{[m]}$:

$$\frac{\frac{1}{M} \sum_{m=1}^{M} w_t^{[m]}\, \mathbb{1}\left(x_t^{[m]} \in \cdot\right)}{\frac{1}{M} \sum_{j=1}^{M} w_t^{[j]}} = \sum_{m=1}^{M} w_t^{[m]}\, \mathbb{1}\left(x_t^{[m]} \in \cdot\right) \overset{M \to \infty}{\longrightarrow} \int bel(x_t)\, dx_t$$

> **Nota delle Lezioni: Notazione Sintetica dei Lucidi**
>
> Nei lucidi del docente (slide 37) la scrittura $\eta \sum_{m=1}^{M} w^{[m]}\, \mathbb{1}(x^{[m]} \in \cdot)$ rappresenta una contrazione notazionale che anticipa il coefficiente di scala prima della normalizzazione dei pesi: nella pratica numerica η viene calcolato proprio come $\left(\sum_{j=1}^{M} w_t^{[j]}\right)^{-1}$, riconducendosi esattamente allo stimatore normalizzato SNIS.
>
> La dimostrazione analitica estesa sull'intera storia temporale $bel(x_{0:t})$ e la successiva marginalizzazione a $bel(x_t)$ sono riportate in zz. Argomenti extra: Derivazione Formale della Traiettoria Completa.

Algoritmi Di Ricampionamento (Resampling)

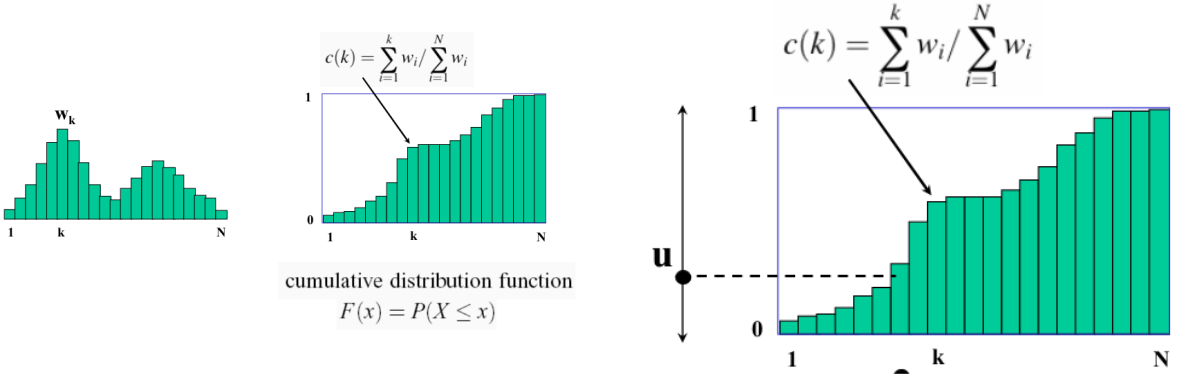
Una volta calcolati i pesi non normalizzati $w_t^{[m]} = p(z_t \mid x_t^{[m]})$ per tutte le M particelle, dobbiamo trasformare l'insieme pesato a priori in un insieme equiprobabile a posteriori.

Il primo passo consiste nella normalizzazione a somma unitaria:

$$w_t^{[m]} = \frac{w_t^{[m]}}{\sum_{j=1}^{M} w_t^{[j]}}$$

A partire dai pesi normalizzati costruiamo la **funzione di ripartizione discreta** (Cumulative Distribution Function - CDF):

$$c_k = \sum_{j=1}^{k} w_t^{[j]}, \quad k = 1, \ldots, M \quad \text{con } c_0 = 0 \text{ e } c_M = 1$$



Roulette Wheel / Inversion Sampling

Il metodo classico di campionamento da CDF consiste nell'estrarre un numero pseudo-casuale uniforme $u \in [0, 1)$ e selezionare la particella corrispondente all'intervallo che contiene u:

$$\text{seleziona particella } i \quad \text{se } c_{i-1} < u \leq c_i$$

Questo meccanismo equivale a una ruota della roulette in cui ogni particella possiede un settore circolare di ampiezza angolare proporzionale a $w_t^{[m]}$. Le particelle con peso elevato hanno una probabilità molto maggiore di essere selezionate e duplicate, mentre le particelle con peso trascurabile vengono rapidamente scartate. Ripetendo l'estrazione M volte in modo indipendente otteniamo la nuova popolazione $\bar{X}_t$.

Se a valle del ricampionamento otteniamo particelle identiche tra loro (ad esempio 100 copie dello stesso campione ad altissima verosimiglianza), esse possiedono la stessa posa e la stessa probabilità; tuttavia, al passo temporale successivo non rimarranno bloccate: il modello di moto stocastico aggiungerà a ciascuna copia un rumore casuale indipendente, disperdendole nuovamente nello spazio ed esplorando in modo differenziato l'intorno dell'ipotesi più accreditata.

- **Complessità computazionale**: per ciascuna delle M estrazioni indipendenti, localizzare l'indice i sulla CDF ordinata tramite ricerca dicotomica richiede $O(\log M)$ confronti, portando il costo complessivo a $O(M \log M)$;
- **Limite statistico**: l'estrazione indipendente introduce varianza stocastica aggiuntiva (varianza di campionamento), rischiando di scartare campioni rilevanti anche se dotati di peso non trascurabile.

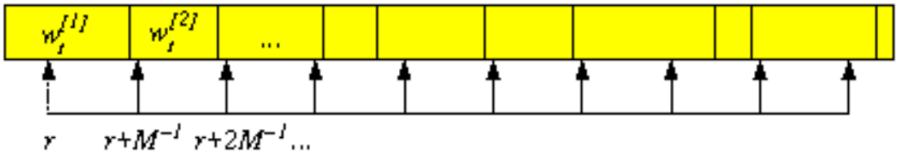
Low-Variance Resampling (Campionamento Sistematico)

Nelle implementazioni industriali e nel framework ROS/ROS2 si adotta quasi universalmente l'algoritmo di **Low-Variance Resampling** (introdotto da Thrun):

1. Si estrae un unico numero casuale iniziale $r \in [0, M^{-1})$;
2. Si costruisce un pettine a M denti equidistanziati di passo fisso M^{-1}:

$$u_m = r + (m-1)M^{-1}, \quad m = 1, \ldots, M$$

3. Si scorrono congiuntamente i puntatori u_m e la CDF c_k con un unico ciclo lineare `while`.



L'idea geometrica è chiara: immaginiamo le M particelle disposte su un nastro in cui ciascuna occupa un intervallo proporzionale al proprio peso normalizzato $w_t^{[m]}$. Il pettine a denti fissi M^{-1} trasla rigidamente di un offset casuale $r \in [0, M^{-1})$ ed estrae tutte le particelle intersecate dai suoi denti:

$$i = \arg\min_j \sum_{m=1}^{j} w_t^{[m]} \geq u_k$$

```python
def low_variance_sampling(Xt_bar, w, M):
    Xt = []
    r = np.random.uniform(0.0, 1.0 / M)
    c = w[0]
    i = 0

    for m in range(M):
        U = r + m * (1.0 / M)
        while U > c:
            i += 1
            c += w[i]
        Xt.append(Xt_bar[i])

    return Xt
```

Questo approccio garantisce complessità lineare deterministica $O(M)$, esplora uniformemente l'intera popolazione azzerando la varianza di estrazione tra i campioni e assicura che qualsiasi particella con peso normalizzato $w_t^{[m]} \geq M^{-1}$ venga selezionata con certezza almeno una volta.

Algoritmo Formale Del Particle Filter

L'algoritmo standard SIR (**Sampling Importance Resampling**), noto anche come Vanilla MCL, unisce i tre blocchi in un ciclo ricorsivo compatto:

```python
def particle_filter(Xt_prev, ut, zt, M):
    Xt_bar = []
    w = []

    # 1. Prediction & Weighting
    for m in range(M):
        # Sample from motion model (Proposal)
        x_m = sample_motion_model(ut, Xt_prev[m])
        # Compute importance weight (Likelihood)
        w_m = measurement_model(zt, x_m)

        Xt_bar.append(x_m)
        w.append(w_m)

    # 2. Resampling step (with replacement)
    Xt = resample(Xt_bar, w, M)

    return Xt
```

In termini procedurali:

1. **Inizializzazione**: se non conosciamo la posa iniziale, estraiamo M particelle con distribuzione uniforme nello spazio libero; se abbiamo una stima a priori attorno a $\mathbf{x}_0$ con matrice di covarianza $_0$, campioniamo da una gaussiana multivariata $(\mathbf{x}_0, _0)$;
2. **Predizione (Motion Model)**: per ogni particella m, propaghiamo lo stato antecedente $x_{t-1}^{[m]}$ applicando il comando u_t corrotto da rumore casuale secondo il modello odometrico o cinematico del veicolo;
3. **Pesatura (Measurement Model)**: valutiamo la verosimiglianza delle misure osservate z_t rispetto alla posa predetta $x_t^{[m]}$;
4. **Ricampionamento**: estraiamo M nuove particelle con probabilità proporzionale ai pesi $w_t^{[m]}$ con rimpiazzo, producendo campioni identicamente distribuiti che rappresentano la posterior $bel(x_t)$.

Progettazione Di Un Particle Filter 1D

Per comprendere la meccanica computazionale analizziamo il modello di laboratorio proposto dal docente per un robot puntiforme vincolato a muoversi su una guida monodimensionale di lunghezza L.

Modello Di Moto 1D

Il comando di traslazione è u_x. Il moto lungo la coordinata x è affetto da rumore gaussiano additivo a media nulla e varianza ^2_x:

$$x_t = x_{t-1} + u_x + , \quad \sim (0, ^2_x)$$

Per ciascuna particella m, la funzione di campionamento estrae un numero da una distribuzione normale:

$$x_t^{[m]} = x_{t-1}^{[m]} + u_x + \text{randn}() \cdot _x$$

Modello Di Misura Con Beacon 1D

Il sensore calcola la distanza dal beacon di riferimento collocato nella coordinata nota m_1, corrotto da rumore gaussiano a deviazione standard $_z$:

$$z_{1,t} = |x_t - m_1| + , \quad \sim (0, ^2_z)$$

Data una particella con posa $x_t^{[m]}$, la distanza attesa dal beacon è $\hat{z}^{[m]} = |x_t^{[m]} - m_1|$. Il residuo di misura è:

$$e^{[m]} = z_{1,t} - |x_t^{[m]} - m_1|$$

Il fattore di importanza (peso) associato alla particella è il valore della densità di probabilità gaussiana del residuo:

$$w_t^{[m]} = p(z_t \mid x_t^{[m]}) = \frac{1}{2 ^2_z} \exp\left(- \frac{\left(z_{1,t} - |x_t^{[m]} - m_1|\right)^2}{2 ^2_z} \right)$$

Nel capitolo precedente abbiamo dimostrato che un singolo beacon genera una likelihood bimodale simmetrica centrata nei due punti $x = m_1 \ z_{1,t}$.

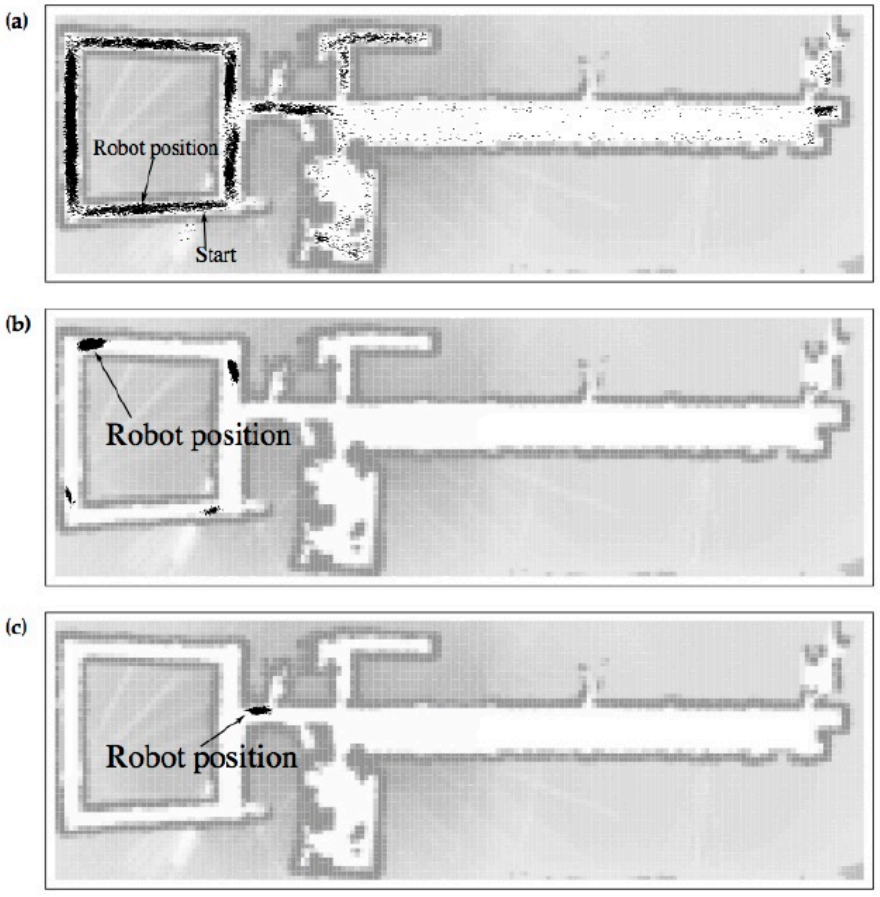
Nel Particle Filter accade esattamente lo stesso fenomeno geometrico: le particelle ricampionate si divideranno inizialmente in due raggruppamenti speculari attorno ai due picchi. Non appena il robot esegue un comando di traslazione $u_x \neq 0$ e acquisisce una seconda misura, le particelle appartenenti al picco errato non troveranno riscontro nelle nuove letture ricevendo pesi vicini allo zero, lasciando sopravvivere unicamente la nube coincidente con la posa reale.

Criticità Ingegneristiche E Compromessi Real-World

Nonostante la flessibilità nel trattare distribuzioni arbitrarie e non gaussiane, l'impiego del Particle Filter in sistemi reali richiede di gestire tre problematiche fondamentali:

Degenerazione Delle Particelle (Particle Depletion)

Se il robot rimane fermo sul posto ($u_t = 0$) ed esegue cicli continui di misura e ricampionamento, la varianza statistica della popolazione tende a collassare: le particelle con peso leggermente superiore generano copie multiple, eliminando tutte le altre. Dopo poche iterazioni, tutte le M particelle collassano su un unico identico valore numerico, perdendo la capacità di rappresentare l'incertezza dello stato.



Nel pannello (a) osserviamo la popolazione iniziale distribuita nell'atrio; nel pannello (b) il robot esegue tracking; nel pannello (c) si verifica il fenomeno di **particle deprivation**: il robot si trova nel corridoio, ma nessuna particella ha campionato quella specifica area. Poiché il ricampionamento standard può selezionare solo particelle preesistenti, il filtro non potrà mai generare campioni nella reale posa del veicolo, portando alla perdita irreversibile della traccia.

Per mitigare la degenerazione ordinaria si ricampiona solo quando il numero effettivo di particelle N_{ef}scende sotto una soglia critica:

$$N_{\text{ef}} = \frac{1}{\sum_{m=1}^{M} \left(w_t^{[m]} \right)^2}$$

Se tutti i pesi sono uniformi ($w = 1/M$), allora $N_{\text{ef}} = M$; se un singolo campione domina l'intera popolazione, $N_{\text{ef}} \to 1$.

Il Problema Del Kidnapped Robot Ed Augmented MCL

Se un operatore sposta fisicamente il robot in un'altra stanza senza che i suoi sensori propriocettivi registrino alcun movimento (problema del robot rapito), le particelle continuano a gravitare nella stanza precedente. Poiché nessuna particella si trova in prossimità della nuova posa reale, tutte le particelle riceveranno pesi di verosimiglianza quasi nulli ($w_t^{[m]} \approx 0$).

Il ricampionamento ordinario estrarrà comunque copie delle particelle preesistenti (essendo i pesi riscalati a somma unitaria), rendendo il filtro matematicamente incapace di fuggire dal minimo locale e recuperare la localizzazione corretta.

Per risolvere questa criticità, Sebastian Thrun ha introdotto l'algoritmo **Augmented MCL**: il filtro monitora l'andamento della verosimiglianza media delle osservazioni:

$$w_{\text{ag}} = \frac{1}{M}\sum_{m=1}^{M} w_t^{[m]} \approx p(z_t \mid z_{1:t-1}, u_{1:t}, m)$$

Poiché w_{ag} oscilla per effetto del rumore sensoriale, ne calcoliamo due stime a media mobile esponenziale con costanti di tempo differenti:

- Un filtro lento a lungo termine (w_{slow}con parametro $_{\text{slow}}$):

$$w_{\text{slow}}w_{\text{slow}} + _{\text{slow}}(w_{\text{ag}} - w_{\text{slow}}) = (1 - _{\text{slow}})w_{\text{slow}} + _{\text{slow}}w_{\text{ag}}$$

- Un filtro veloce a breve termine (w_{fast} con parametro $_{\text{fast}}$):

$$w_{\text{fast}}\ \ w_{\text{fast}} + _{\text{fast}}(w_{\text{ag}} - w_{\text{fast}}) = (1 - _{\text{fast}})w_{\text{fast}} + _{\text{fast}}w_{\text{ag}}$$

con vincolo di scala $0 \leq _{\text{slow}}{}_{\text{fast}}\ \ 1$.

In condizioni nominali di tracking la verosimiglianza è stazionaria e $w_{\text{fast}} \approx w_{\text{slow}}$Se il robot viene rapito, la verosimiglianza crolla istantaneamente: w_{fast} decade rapidamente verso lo zero, mentre w_{slow}conserva memoria del passato recente. Il rapporto $w_{\text{fast}}/w_{\text{slow}}$scende bruscamente al di sotto di 1, segnalando una grave incongruenza di stima.

A ogni passo di ricampionamento, iniettiamo una posa casuale uniforme nello spazio libero con probabilità:

$$p_{\text{random}} = \max\left(0,\ 1 - \frac{w_{\text{fast}}}{w_{\text{slow}}}\right)$$

mentre con probabilità complementare $1 - p_{\text{random}}$ procediamo con il ricampionamento ordinario guidato da $w_t^{[m]}$.

```
def augmented_mcl(Xt_prev, zt, ut, m, M, w_slow, w_fast, alpha_slow, alpha_fast):
    Xt_bar = []
    w = []
    w_avg = 0.0

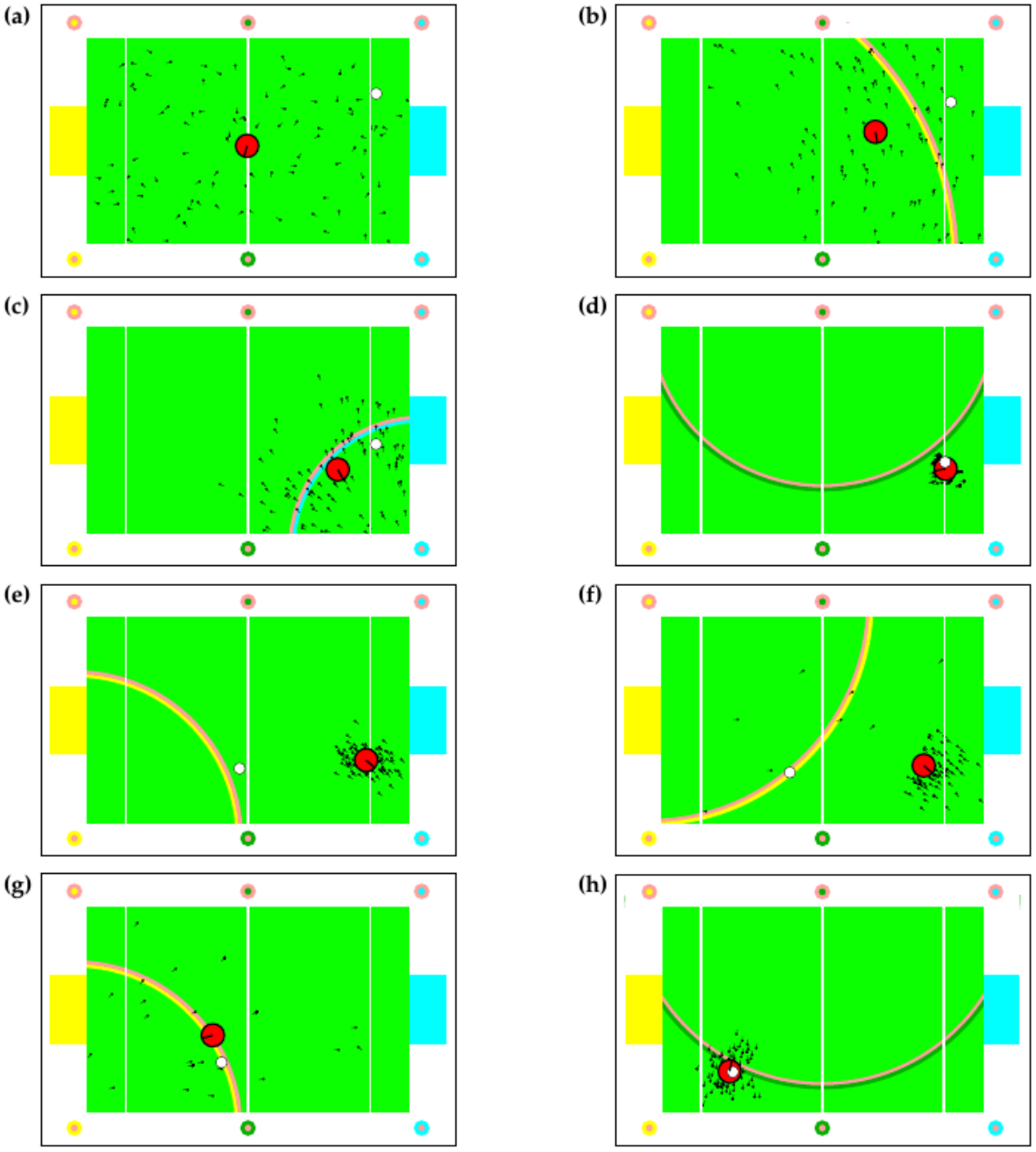
    # 1. Prediction and Weighting
    for m_idx in range(M):
        x_m = sample_motion_model(ut, Xt_prev[m_idx])
        w_m = measurement_model(zt, x_m, m)
        Xt_bar.append(x_m)
        w.append(w_m)
        w_avg += (1.0 / M) * w_m

    # 2. Update running averages
    w_slow += alpha_slow * (w_avg - w_slow)
    w_fast += alpha_fast * (w_avg - w_fast)

    # 3. Resampling with random pose injection
    Xt = []
    prob_rand = max(0.0, 1.0 - w_fast / w_slow)

    for m_idx in range(M):
        if np.random.uniform(0.0, 1.0) < prob_rand:
            Xt.append(sample_random_pose(m))
        else:
            Xt.append(draw_particle_proportional_to_weight(Xt_bar, w))
```

```
    return Xt, w_slow, w_fast
```



La sequenza di simulazione RoboCup illustra perfettamente il comportamento di Augmented MCL:

- Nei pannelli (a)-(d), il robot esegue localizzazione globale: la popolazione uniforme iniziale collassa rapidamente nell'intersezione delle curve di misura dei beacon noti, convergendo verso la vera posa;
- Nei pannelli (e)-(h), il robot viene rapito: le misure sensoriali non concordano con la stima corrente (e), determinando il crollo di w_{fast}. Il filtro inietta campioni casuali (f) che intersecano i landmark corretti (g), fino a ricatturare stabilmente la nuova posa del robot (h).

Disambiguazione dell'Acronimo AMCL: Augmented vs Adaptive

È importante chiarire la duplice accezione dell'acronimo AMCL tra teoria e middleware robotici:

1. **Augmented MCL**: è la denominazione introdotta da Thrun per l'algoritmo di recupero dal rapimento basato su iniezione probabilistica guidata dai filtri w_{slow} e w_{fast};

2. **Adaptive Monte Carlo Localization (AMCL)**: è la denominazione ufficiale del package di localizzazione 2D nel framework **ROS / ROS2 (Nav2)**. Il package prende originariamente il nome di *Adaptive* perché implementa la tecnica del **KLD-Sampling** (Fox, 2001) per variare dinamicamente la dimensione della popolazione M;
3. **L'implementazione industriale reale**: il nodo `amcl` di ROS/ROS2 fonde entrambi i contributi: adatta la cardinalità M tramite KLD-Sampling e gestisce il rapimento tramite i parametri `recovery_alpha_slow` e `recovery_alpha_fast` di Augmented MCL.

L'analisi del codice C sorgente di ROS (`pf_update_sensor` , `pf_update_resample` , `pf_resample_limit`) è consultabile in zz. Argomenti extra: Implementazione Industriale di AMCL in ROS.

Adaptive MCL Tramite KLD-Sampling

Il dimensionamento statico del numero di campioni M impone un rigido compromesso ingegneristico:

- Durante la **Global Localization**, l'incertezza è estesa a tutta la mappa e richiede decine di migliaia di campioni (10^4--10^5) per garantire una copertura sufficiente dello spazio;
- Durante il **Position Tracking**, la densità di probabilità collassa in un lobo unimodale concentrato e poche centinaia di particelle (10^2--10^3) sono più che sufficienti.

Mantenere M costantemente elevato saturerebbe la CPU del robot, mentre ridurlo staticamente provocherebbe divergenza e fallimento nella localizzazione globale.

La soluzione ottima è il **KLD-Sampling** (Kullback-Leibler Divergence Sampling), introdotto da Dieter Fox (2001/2003): la dimensione della popolazione M viene calcolata dinamicamente a ogni ciclo di stima in modo da garantire che, con probabilità $1-\delta$, la distanza statistica di Kullback-Leibler tra la vera posterior continua e l'approssimazione campionaria non superi una soglia di errore ε:

$$KL(p \| \hat{p}) \leq \varepsilon \quad \text{con probabilità } 1-\delta$$

Approssimando lo spazio continuo tramite un istogramma a celle discrete, Fox dimostra che la soglia minima di campioni necessari M_X dipende unicamente dal numero k di bin non vuoti (ossia bin contenenti almeno una particella generata):

$$M_X = \frac{k-1}{2\varepsilon}\left(1 - \frac{2}{9(k-1)} + \sqrt{\frac{2}{9(k-1)}}\, z_{1-\delta}\right)^3$$

dove:

- k: numero di celle dell'istogramma occupate da almeno una particella estratta ($k > 1$);
- ε: errore massimo tollerato sulla divergenza KL (parametro `kld_err` in ROS, tipicamente 0.05);
- $z_{1-\delta}$: quantile superiore della distribuzione normale standard per il livello di confidenza desiderato (parametro `kld_z` in ROS; per un'accuratezza al 99% con $\delta = 0.01$, si ha $z_{0.99} \approx 2.326$).

Algoritmo KLD-Sampling MCL

L'algoritmo integra la predizione, la pesatura e l'adattamento dinamico di M all'interno di un unico ciclo incrementale:

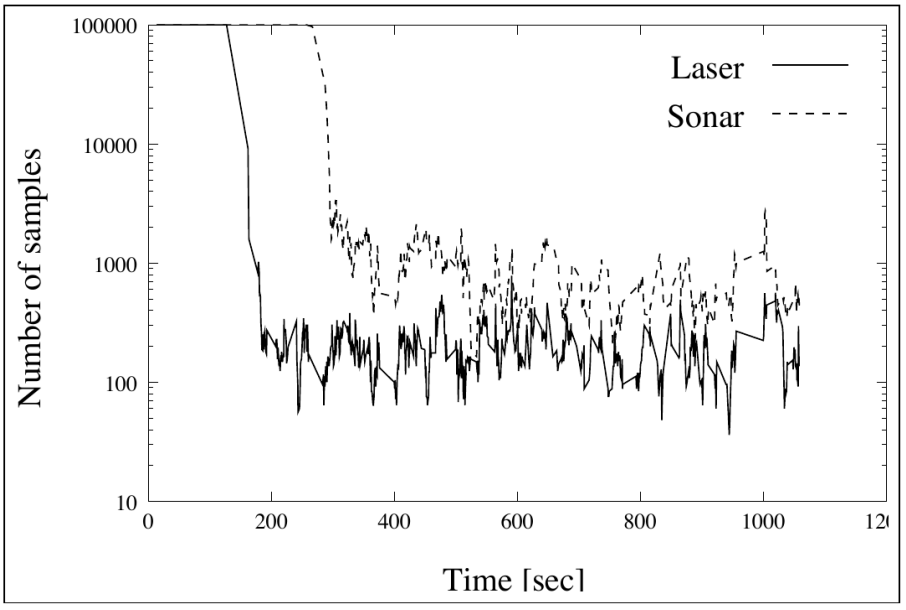
```
def kld_sampling_mcl(Xt_prev, zt, ut, m, epsilon, delta, M_min, M_max):
    Xt = []
    M = 0
    MX = M_max
    k = 0
    # Tabella hash o KD-Tree per tracciare i bin occupati
    bins_occupied = set()
```

```
    while (M < MX or M < M_min) and M < M_max:
        # Estrai indice proporzionalmente al peso antecedente
        i = draw_particle_with_probability(Xt_prev)
        # Propaga con modello di moto
        x_m = sample_motion_model(ut, Xt_prev[i].pose)
        # Calcola peso di verosimiglianza
        w_m = measurement_model(zt, x_m, m)
        Xt.append(Particle(x_m, w_m))

        # Identifica il bin geometrico della posa x_m
        b = get_bin_index(x_m)
        if b not in bins_occupied:
            bins_occupied.add(b)
            k += 1
            if k > 1:
                term = 2.0 / (9.0 * (k - 1))
                z = scipy.stats.norm.ppf(1.0 - delta)
                val = 1.0 - term + np.sqrt(term) * z
                MX = ((k - 1) / (2.0 * epsilon)) * (val ** 3)

        M += 1

    return Xt
```



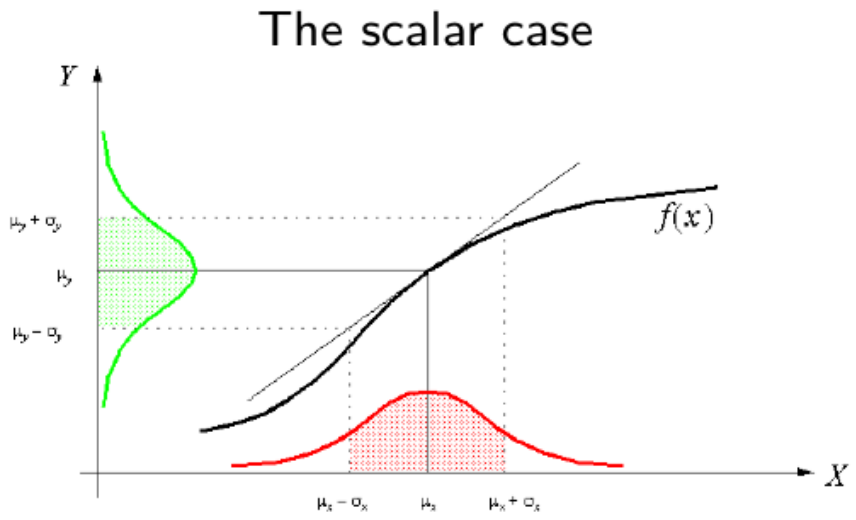
Il grafico sperimentale evidenzia il risparmio computazionale del KLD-Sampling:

- All'inizio della missione ($t \ 200\ \mathrm{s}$), il robot esplora l'ambiente in regime di localizzazione globale: le particelle occupano un gran numero di bin spaziali (k elevato) e il filtro alloca il tetto massimo di 100.000 campioni;
- Non appena le letture sensoriali (Laser o Sonar) delimitano la posa, le particelle collassano in poche celle contigue: k crolla e il numero di campioni richiesti precipita di **2-3 ordini di grandezza**, stabilizzandosi tra 100 e 1000 campioni per il laser e tra 500 e 2000 per il sonar.

Il filtro modula autonomamente il proprio fabbisogno computazionale: spende risorse solo quando serve (incertezza multimodale o rapimento) e le riduce al minimo indispensabile durante la navigazione nominale.

Propagazione Non-Lineare Dell'Incertezza E Linearizzazione

Il vantaggio intrinseco dei metodi a particelle rispetto ai filtri analitici gaussiani (come il filtro di Kalman esteso / EKF) emerge con chiarezza analizzando la propagazione dell'incertezza attraverso trasformazioni non-lineari.



Nel filtro EKF una variabile gaussiana x viene trasformata mediante una funzione non-lineare (x) linearizzata al primo ordine tramite sviluppo in serie di Taylor nell'intorno della media ;.

$$(x) \approx ()+ ()(x -)$$

L'approssimazione forza la densità trasformata a rimanere una gaussiana unimodale, commettendo gravi errori di distorsione se la non-linearità è accentuata o se la distribuzione reale si divide in lobi multipli. Il Particle Filter, campionando direttamente la funzione non-lineare senza alcuna linearizzazione analitica, riproduce con esattezza qualsiasi forma asimmetrica o multimodale.

7. Modelli Di Misura per la Localizzazione

Nei capitoli precedenti abbiamo visto come il filtro di Bayes (Filtri di Bayes) e le sue realizzazioni computazionali (Histogram Filter e Particle Filter) integrino ricorsivamente due sorgenti di informazione complementari: la predizione basata sul moto e la correzione guidata dalle osservazioni sensoriali esterocettive.

In questo capitolo affrontiamo il **Measurement Model** (o modello di osservazione): la formulazione analitica che descrive la verosimiglianza statistica di ottenere una lettura sensoriale data una posa ipotizzata del veicolo e una mappa nota dell'ambiente.

Il Ruolo Del Modello Di Misura Nel Ciclo Di Stima Bayesiano

Nel passo di correzione bayesiana calcoliamo la distribuzione a posteriori:

$$bel(x_t) = \eta\, p(z_t \mid x_t, m)\, \overline{bel}(x_t)$$

Il termine cruciale è la funzione di verosimiglianza $p(z_t \mid x_t, m)$. Questa probabilità condizionata risponde a una domanda specifica: *"Se il robot si trovasse esattamente nella posa ipotizzata x_t all'interno della mappa m, qual è la densità di probabilità di misurare il vettore di letture z_t?"*.

Nel Particle Filter ogni particella riceve il proprio peso di importanza $w_t^{[m]}$ valutando puntualmente questa funzione.

Distinguiamo due grandi categorie di modelli di misura per robotica mobile:

1. **Modelli Basati su Feature (Feature-Based)**: la scansione sensoriale grezza viene filtrata per estrarre landmark geometrici compatti (linee, spigoli, beacon);

2. **Modelli Raw-Data (Scan-Based)**: l'intero pacchetto di letture del telemetro laser o LiDAR (centinaia di raggi angolari) viene processato direttamente senza estrazione preventiva di primitive. Tra questi analizzeremo il **Likelihood Field Model** (lo standard industriale per efficienza) e il **Beam Model** (fondato sulla simulazione ottica del raggio).

Cinematica Dei Sistemi Di Riferimento Del Sensore Laser 2D

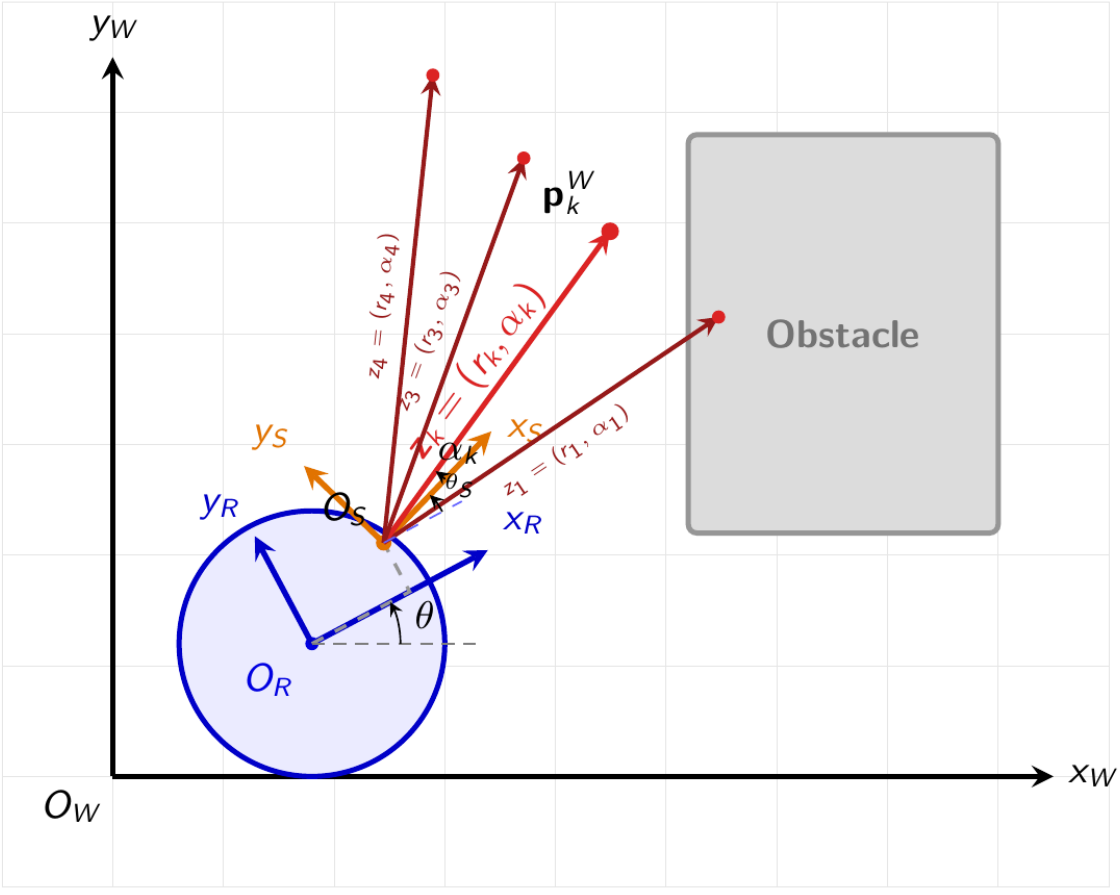
Per calcolare la verosimiglianza di una scansione laser dobbiamo prima ricondurre ogni singolo punto misurato in coordinate polari locali al sistema di coordinate cartesiane globali della mappa.

Assumiamo che il robot operi su una superficie orizzontale piana, con configurazione nello spazio speciale euclideo $SE(2)$:

$$\mathbf{x} = \begin{bmatrix} x \\ y \\ \theta \end{bmatrix} \in \mathbb{R}^2 \times S^1$$

L'architettura geometrica del robot mobile equipaggiato con un LiDAR 2D o telemetro laser (Laser Range Finder - LRF) coinvolge quattro distinti sistemi di riferimento correlati tra loro:

1. **Sistema Mondo W (World / Map Frame)**: terna fissa globale $O_W - (x_W, y_W)$ in cui è definita la mappa dell'ambiente circostante;
2. **Sistema Robot R (Robot Body Frame)**: terna mobile $O_R - (x_R, y_R)$ solidale al telaio del robot, con baricentro in (x, y) e orientamento θ rispetto all'asse x_W;
3. **Sistema Sensore S (Sensor Frame)**: terna mobile $O_S - (x_S, y_S)$ solidale al dispositivo laser, fissato sul telaio con un offset geometrico $(x_{\text{sens}}, y_{\text{sens}})$ e disallineamento angolare θ_{sens} rispetto alla terna del robot;
4. **Fascio Laser k-esimo**: direzione fisica di propagazione del singolo impulso luminoso, individuato in coordinate polari locali dal raggio di scansione r_k (distanza misurata dal Time-of-Flight) e dall'angolo di scansione α_k rispetto all'asse x_S.



Catena Cinematica Di Rototraslazione Nel Piano

Vogliamo calcolare le coordinate globali del punto di impatto dell'ostacolo $\mathbf{p}_k^W = [x_k^W, y_k^W]^T$.

Richiamiamo la matrice di rotazione planare $R(\psi) \in SO(2)$:

$$R(\psi) = \begin{bmatrix} \cos\psi & -\sin\psi \\ \sin\psi & \cos\psi \end{bmatrix}$$

Procediamo componendo le trasformazioni rigide passo dopo passo:

1. **Dal Fascio al Sensore**: il raggio k-esimo rileva un punto alle coordinate polari (ρ_k, φ_k) nel piano del sensore:

$$p_k^S = \begin{bmatrix} x_k^S \\ y_k^S \end{bmatrix} = \begin{bmatrix} \rho_k \cos\varphi_k \\ \rho_k \sin\varphi_k \end{bmatrix}$$

2. **Dal Sensore al Robot**: ruotiamo di θ_{sens} e trasliamo dell'offset di montaggio $t_{\text{sens}} = [x_{\text{sens}}, y_{\text{sens}}]^T$:

$$p_k^R = \begin{bmatrix} x_{\text{sens}} \\ y_{\text{sens}} \end{bmatrix} + R(\theta_{\text{sens}}) \begin{bmatrix} \rho_k \cos\varphi_k \\ \rho_k \sin\varphi_k \end{bmatrix}$$

3. **Dal Robot al Mondo**: ruotiamo dell'orientamento del robot θ e sommiamo la posizione globale $[x, y]^T$:

$$p_k^W = \begin{bmatrix} x \\ y \end{bmatrix} + R(\theta) p_k^R$$

Sostituendo l'espressione di p_k^R sfruttando la proprietà di composizione delle rotazioni $R(\theta)R(\theta_{\text{sens}}) = R(\theta + \theta_{\text{sens}})$ otteniamo la formula rototraslatoria completa:

$$\begin{bmatrix} x_k^W \\ y_k^W \end{bmatrix} = \begin{bmatrix} x \\ y \end{bmatrix} + R(\theta) \begin{bmatrix} x_{\text{sens}} \\ y_{\text{sens}} \end{bmatrix} + R(\theta + \theta_{\text{sens}}) \begin{bmatrix} \rho_k \cos\varphi_k \\ \rho_k \sin\varphi_k \end{bmatrix}$$

Esplicitando in forma scalare riga per riga:

$$\begin{aligned} x_k^W &= x + x_{\text{sens}} \cos\theta - y_{\text{sens}} \sin\theta + \rho_k \cos(\theta + \theta_{\text{sens}} + \varphi_k) \\ y_k^W &= y + x_{\text{sens}} \sin\theta + y_{\text{sens}} \cos\theta + \rho_k \sin(\theta + \theta_{\text{sens}} + \varphi_k) \end{aligned}$$

Questa trasformazione proietta istantaneamente ogni raggio laser nel sistema di riferimento globale della mappa conoscendo l'ipotesi di posa (x, y, θ) del veicolo.

Modello Basato Su Feature (Feature-Based Measurement Model)

L'approccio basato su feature (FB-MM) riduce la complessa nuvola di punti raw a un insieme compatto di landmark geometrici salienti estratti tramite algoritmi di segmentazione (come visto in Estrazione di Feature):

$$z_{f,i} = \begin{bmatrix} r_i \\ \varphi_i \\ s_i \end{bmatrix}$$

dove r_i è la distanza stimata dal landmark, φ_iè l'angolo di vista rispetto alla prora del robot (bearing) e s_i è una segnatura descrittiva (colore, etichetta identificativa o riflettanza del beacon).

Data la posa ipotizzata x_t e la posizione nota del landmark sulla mappa $m_j = [m_{j,x}, m_{j,y}]^T$, la misura ideale attesa priva di rumore è:

$$\hat{z}_i = \begin{bmatrix} \sqrt{(m_{j,x} - x)^2 + (m_{j,y} - y)^2} \\ \operatorname{atan2}(m_{j,y} - y,\, m_{j,x} - x) - \theta \end{bmatrix}$$

Generalizzazione Probabilistica: Dalla Gaussiana a Funzioni Arbitrarie

Nel modello scalare ideale si assume un rumore additivo gaussiano a media nulla con varianza σ^2:

$$z_f = z_f^* + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \sigma^2)$$

Tuttavia, come evidenziato dal docente a lezione (aggiornamento di `013_1` rispetto alla versione preliminare `012_2`), imporre rigidamente una campana gaussiana è spesso inadeguato nei contesti operativi reali per via di riflessioni anomale, code pesanti e saturazioni.

Si adotta quindi una formulazione probabilistica più generale:

$$p(z_f \mid x_t, m) = \text{prob}(z_f - z_f^*, \sigma)$$

dove $\text{prob}(\Delta, \sigma)$ rappresenta una funzione di densità arbitraria (gaussiana, triangolare, o miscela con code uniformi per modellare falsi allarmi).

Per un insieme di feature indipendenti condizionatamente alla posa del robot, la verosimiglianza complessiva è il prodotto delle singole componenti:

$$p(z_t \mid x_t, m) = \prod_{i=1}^{N_t} p(z_{f,i} \mid x_t, m)$$

> **La Criticità della Corrispondenza Dati (Data Association)**
>
> Il limite fondamentale dei modelli basati su feature risiede nel problema della corrispondenza (*data association*): per calcolare la verosimiglianza dobbiamo sapere a quale landmark della mappa m_j corrisponde la lettura $z_{f,i}$. Se i landmark non possiedono una segnatura univoca, un'associazione errata produce gravissimi picchi spuri di verosimiglianza che possono far divergere la stima.

Modello a Campo Di Verosimiglianza (Likelihood Field Model)

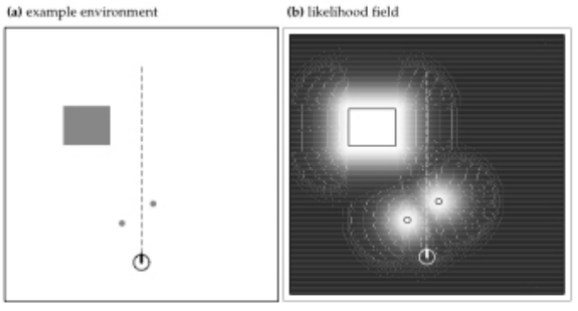
Per superare la fragilità della data association e l'onere computazionale dell'estrazione di primitive, nella moderna navigazione robotica (incluso il package `amcl` di ROS/ROS2) si adotta prevalentemente il **Likelihood Field Model**.

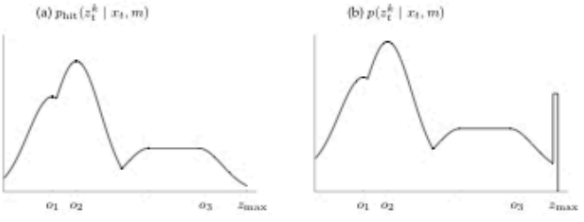
L'idea cardine è semplice ed elegante: invece di cercare corrispondenze discrete o simulare la traiettoria ottica del raggio nello spazio, ogni singolo punto misurato dal laser viene proiettato direttamente nella mappa globale tramite la trasformazione cinematica $T_{t,k}$. La sua verosimiglianza viene quindi calcolata in funzione della **distanza minima dall'ostacolo più vicino**.

Formulazione Geometrica E Campo Di Distanza

Sia $p_k = [x_k, y_k]^T$ il punto globale calcolato per il raggio k-esimo. Definiamo la distanza euclidea minima tra p_k e qualsiasi cella occupata nella mappa m:

$$dist = \min_{x',y'} \sqrt{(x_k - x')^2 + (y_k - y')^2} \quad (x', y') \text{ occupato in } m$$





Nel pannello di sinistra osserviamo l'ambiente reale con ostacoli e il corrispondente **Likelihood Field** generato attorno alle pareti e agli oggetti: l'intensità luminosa rappresenta la densità di verosimiglianza, massima in corrispondenza del bordo dell'ostacolo ($ist = 0$) e via via decrescente man mano che ci si allontana.

Nel pannello di destra è tracciata la curva di risposta del sensore al variare della distanza misurata rispetto agli ostacoli $1, 2, 3$ e al fondoscala $z_{\max}$

Modello Di Rumore a Miscela a Tre Componenti

Il modello reale combina tre distinte sorgenti di incertezza:

1. **Rumore di Misura Gaussiano (p_{ht})**: modella l'incertezza intrinseca del telemetro laser tramite una gaussiana a media nulla calcolata sulla distanza minima ist:

$$p_{\text{ht}}(z_k \mid x_t, m) = \frac{1}{2\pi_{\text{ht}}^2} \exp\left(-\frac{ist^2}{2_{\text{ht}}^2}\right)$$

2. **Falsi Allarmi e Rumore Casuale (p_{rand})**: modella riflessioni diffuse anomale e interferenze elettroniche tramite una distribuzione uniforme sull'intero campo di misura del sensore $[0, z_{\max}]$:

$$p_{\text{rand}}(z_k \mid x_t, m) = \frac{1}{z_{\max}}$$

3. **Fondo Scala e Scarto Letture Spurie ($z_{\max}$)**: se il sensore restituisce la lettura massima $_k = z_{\max}$ significa che il raggio non ha intersecato alcun ostacolo entro la portata massima (ad esempio fasci sparati lungo un lungo corridoio aperto o finestre trasparenti). Come rimarcato dal docente a lezione (slide 14 di `013_1`), la lettura a fondoscala **è spuria e priva di informazione geometrica localizzante** e va scartata dal prodotto di calcolo.

La densità di verosimiglianza per il singolo raggio valido ($_k \neq z_{\max}$) è data dalla combinazione convessa pesata:

$$p(z_k \mid x_t, m) = _{\text{ht}}\, p_{\text{ht}}(z_k \mid x_t, m) + _{\text{rand}}\, \frac{1}{z_{\max}}$$

con $_{\text{ht}} + _{\text{rand}} = 1$.

Assumendo che i singoli raggi laser siano statisticamente indipendenti condizionatamente alla posa x_t e alla mappa m, la verosimiglianza complessiva dell'intera scansione $z_t = _1, \ldots, z_{N_z}$ è il prodotto cumulativo:

$$= \prod_{k=1,\ _k \neq z_{\max}}^{N_} p(z_k \mid x_t, m)$$

Algoritmo Del Likelihood Field

L'implementazione algoritmica standard unisce la rototraslazione cinematica, la ricerca della distanza minima e la combinazione a miscela:

```
def likelihood_field_range_finder_model(zt, xt, m, params):
    # xt = (x, y, theta) posa del robot
    # zt = array di raggi [(r_1, alpha_1), ..., (r_K, alpha_K)]
    q = 1.0

    for r_k, alpha_k in zt:
        # 1. Scarta le letture a fondo scala (spurie)
        if r_k >= params.z_max:
            continue

        # 2. Rototraslazione del punto di impatto nel frame Mondo
        x_w = (xt.x + params.x_sens * np.cos(xt.theta)
               - params.y_sens * np.sin(xt.theta)
               + r_k * np.cos(xt.theta + params.theta_sens + alpha_k))
```

```
        y_w = (xt.y + params.x_sens * np.sin(xt.theta)
               + params.y_sens * np.cos(xt.theta)
               + r_k * np.sin(xt.theta + params.theta_sens + alpha_k))

        # 3. Lookup O(1) della distanza minima sulla griglia precalcolata
        dist = get_distance_to_nearest_obstacle(m, x_w, y_w)

        # 4. Modello a miscela (Gaussiana + Uniforme)
        p_hit = (1.0 / np.sqrt(2.0 * np.pi * params.sigma_hit**2)) * np.exp(-0.5 * (dist**2) /
(params.sigma_hit**2))
        p_rand = 1.0 / params.z_max

        p = params.alpha_hit * p_hit + params.alpha_rand * p_rand
        q *= p

    return q
```

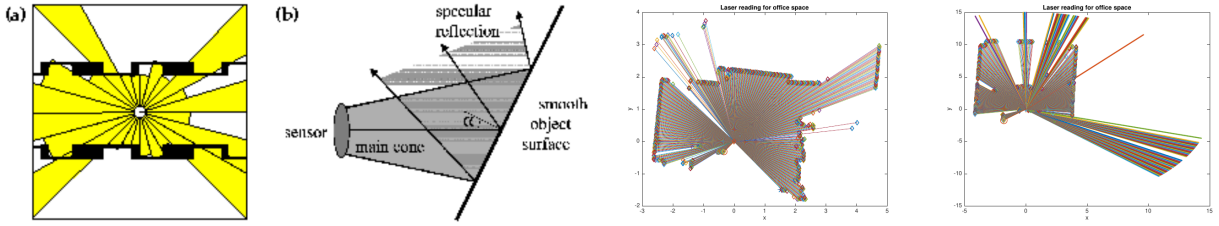
Compromessi Ingegneristici e Limiti del Likelihood Field

Il Likelihood Field domina le applicazioni industriali per una serie di motivi operativi, ma presenta compromessi teorici precisi:

1. **Debole Fondamento Fisico (*No Physical Ground*)**: il modello valuta solo gli endpoint finali del raggio e ignora completamente la traiettoria del fascio. Se un raggio interseca una parete e l'endpoint cade su una stanza successiva a distanza compatibile, riceverà comunque un peso elevato; il modello "vede attraverso i muri", ignorando le occlusioni fisiche;
2. **Impossibilità di Modellare Ostacoli Dinamici**: persone che camminano o arredi spostati non compaiono nella mappa statica precalcolata; la lettura riceverà semplicemente la probabilità residua $_{\text{rand}}/z_{\max}$;
3. **Vantaggio Computazionale Decisivo ($O(1)$ a Runtime)**: il calcolo di ist non richiede ray-tracing costosi: la mappa dell'ambiente viene elaborata **una sola volta offline** calcolando la *Distance Transform* su griglia 2D. A runtime la valutazione di ogni raggio si riduce a un banale accesso in memoria (look-up $O(1)$ in una tabella matriciale).

Modello a Raggio Fisico (Beam Model)

A differenza del Likelihood Field, il **Beam Model** (noto anche come *Beam Range Finder Model*) ricostruisce fedelmente la fisica di propagazione del singolo fascio laser nell'ambiente.

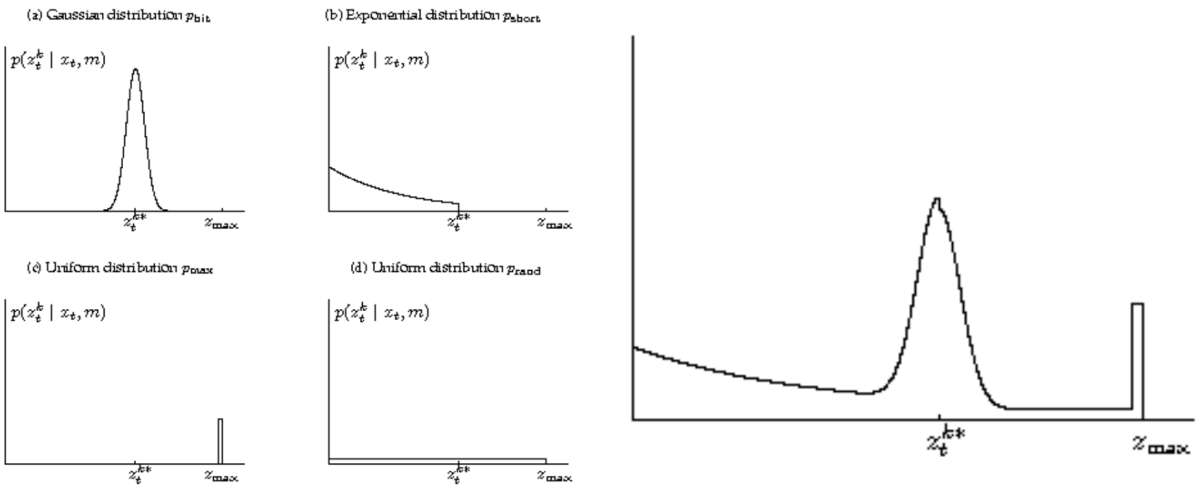


Nel pannello (a) di sinistra osserviamo il processo di **Ray Casting**: per ogni raggio emesso dal laser a un dato angolo, il filtro proietta una linea virtuale nello spazio fino a intersecare il primo ostacolo opaco presente nella mappa, ricavando la distanza teorica ideale attesa z_t^{k*}.

Nel pannello (b) sono illustrati i fenomeni fisici non ideali che degradano la misura: riflessioni speculari su superfici inclinate lisce in cui il fascio devia senza tornare al ricevitore, aperture angolari finite del fascio cono (*beam cone*) e assorbimenti. A destra vediamo una reale scansione laser 2D in un ambiente d'ufficio con corridoi e uffici.

I Quattro Modelli Di Errore Fisico

Il Beam Model decompone l'errore del sensore in quattro sorgenti statistiche fisicamente motivate:



1. **Errore Locale di Misura (p_{hit})**: rumore gaussiano centrato sulla vera distanza geometrica z_t^{k*} calcolata per ray-casting:

$$p_{hit}(z_k \mid x_t, m) = \begin{cases} \eta_{hit} \frac{1}{\sqrt{2\pi\sigma_{hit}^2}} \exp\left(-\frac{(z_k - z_t^{k*})^2}{2\sigma_{hit}^2}\right) & \text{se } 0 \le z_k \le z_{max} \\ 0 & \text{altrimenti} \end{cases}$$

2. **Ostacoli Inattesi (p_{short})**: persone che camminano, porte temporaneamente chiuse o arredi mobili non presenti sulla mappa provocano una lettura *più corta* del raggio ideale. Questo fenomeno è modellato da una densità esponenziale decrescente troncata a z_t^{k*}:

$$p_{short}(z_k \mid x_t, m) = \begin{cases} \eta_{short} \lambda_{short} e^{-\lambda_{short} z_k} & \text{se } 0 \le z_k \le z_t^{k*} \\ 0 & \text{altrimenti} \end{cases}$$

3. **Mancata Lettura / Errore di Fondo Scala (p_{max})**: superfici opache nere che assorbono la luce, riflessioni speculari estreme o bersagli oltre il range utile provocano il timeout del ricevitore ToF, restituendo il valore di saturazione z_{max}. È modellato da un impulso a gradino nell'intorno ristretto di z_{max}:

$$p_{max}(z_k \mid x_t, m) = \mathbb{1}(z_k = z_{max})$$

4. **Rumore Casuale Inspiegabile (p_{rand})**: fenomeni non prevedibili o multipath generano un rumore uniforme piatto su tutto l'intervallo $[0, z_{max}]$:

$$p_{rand}(z_k \mid x_t, m) = \begin{cases} \frac{1}{z_{max}} & \text{se } 0 \le z_k \le z_{max} \\ 0 & \text{altrimenti} \end{cases}$$

La distribuzione complessiva è la combinazione convessa delle quattro componenti (pannello di destra):

$$p(z_k \mid x_t, m) = \alpha_{hit} p_{hit} + \alpha_{short} p_{short} + \alpha_{max} p_{max} + \alpha_{rand} p_{rand}$$

con vincolo di normalizzazione $\alpha_{hit} + \alpha_{short} + \alpha_{max} + \alpha_{rand} = 1$.

I pesi e i parametri $\sigma_{hit}, \lambda_{short}$ vengono tarati a partire da dataset reali di calibrazione tramite stimatori di massima verosimiglianza (zz. Argomenti extra: Stima MLE dei Parametri del Beam Model).

Algoritmo Del Beam Model E Complessità Del Ray Casting

```
def beam_range_finder_model(zt, xt, m, params):
    q = 1.0

    for k, (r_k, alpha_k) in enumerate(zt):
```

```
            # 1. Calcolo del raggio virtuale atteso tramite ray-casting
            z_star = ray_casting(xt, alpha_k, m)

            # 2. Valutazione delle quattro componenti
            p_hit = compute_p_hit(r_k, z_star, params.sigma_hit, params.z_max)
            p_short = compute_p_short(r_k, z_star, params.lambda_short)
            p_max = compute_p_max(r_k, params.z_max)
            p_rand = compute_p_rand(r_k, params.z_max)

            # 3. Combinazione convessa
            p = (params.alpha_hit * p_hit +
                 params.alpha_short * p_short +
                 params.alpha_max * p_max +
                 params.alpha_rand * p_rand)

            q *= p

    return q
```

Sebbene il Beam Model sia teoricamente più rigoroso del Likelihood Field, in contesti real-time presenta una grave controindicazione: per ciascun raggio laser di ciascuna delle M particelle occorre calcolare un'operazione di ray-casting nella mappa. Con $M = 1000$ particelle e $N_z = 360$ raggi, servirebbero 360.000 ray-cast per ogni singolo frame del sensore (decine di volte al secondo), un carico proibitivo su hardware embedded.

Per questa ragione, nei sistemi industriali e in ROS il Beam Model viene confinato a casi specifici o rimpiazzato dal Likelihood Field con precalcolo della griglia di distanza.

8. Filtro Di Kalman Lineare

Nei capitoli precedenti abbiamo esplorato le approssimazioni computazionali non-parametriche del filtro di Bayes: l'Histogram Filter su griglia discreta e il Particle Filter stocastico. Entrambi i metodi eccellono nel rappresentare distribuzioni arbitrarie e multimodali (come le ambiguità speculari generate da beacon e pareti), ma richiedono un pesante carico computazionale legato alla granularità delle celle o al numero di campioni M.

In questo capitolo introduciamo il **Filtro di Kalman (Kalman Filter - KF)**, che affronta la stima bayesiana da una prospettiva analitica esatta: assumendo dinamiche di moto e di misura lineari corrotte da rumori gaussiani, il filtro di Bayes ammette una soluzione analitica chiusa ed estremamente efficiente in cui l'intero belief è parametrizzato unicamente dal vettore di media e dalla matrice di covarianza.

Trattiamo qui l'architettura teorica fondamentale per sistemi lineari discreti variabili nel tempo, che costituisce il presupposto metodologico irrinunciabile per la successiva estensione non-lineare (Extended Kalman Filter - EKF) applicata alla localizzazione robotica.

Dal Filtro Di Bayes Ai Filtri Gaussiani Analitici

Richiamiamo la forma continua ricorsiva del filtro di Bayes:

$$\overline{bel}(x_t) = \int p(x_t \mid x_{t-1}, u_t)\, bel(x_{t-1})\, \mathrm{d}x_{t-1}$$

$$bel(x_t) = \eta\, p(y_t \mid x_t)\, \overline{bel}(x_t)$$

Nel caso generale il calcolo analitico dell'integrale di predizione (integrale di Chapman-Kolmogorov) è intrattabile. Tuttavia, se imponiamo due vincoli strutturali:

1. **Linearità delle trasformazioni**: le equazioni che governano l'evoluzione dello stato e la generazione delle uscite sono combinazioni lineari delle variabili;

2. **Gaussianità dei segnali aleatori**: lo stato iniziale e tutte le perturbazioni esterne sono variabili aleatorie gaussiane a media nulla.

accade una proprietà matematica formidabile: la famiglia delle densità normali è una *famiglia coniugata* rispetto a trasformazioni lineari. La convoluzione di due gaussiane è ancora una gaussiana, e il prodotto normalizzato di due gaussiane è ancora rigorosamente una gaussiana.

Non occorre quindi approssimare numericamente lo spazio continuo: basta calcolare analiticamente le equazioni ricorsive di aggiornamento dei soli parametri della distribuzione (valore atteso $\hat{x}$ e matrice di covarianza P).

Modellazione Dei Sistemi Dinamici Lineari Stocastici

Consideriamo un processo dinamico a tempo discreto $t \in \mathbb{Z}$ descritto in uno schema di Hidden Markov Model (HMM) continuo:

- Vettore di stato nascosto da stimare: $x_t \in \mathbb{R}^n$;
- Vettore degli ingressi di controllo noti: $u_t \in \mathbb{R}^m$;
- Vettore delle uscite / osservazioni misurate dai sensori: $y_t \in \mathbb{R}^p$.

Il sistema è governato dalle equazioni lineari stocastiche:

$$x_t = _x_{t-1} + _u_t + w_t$$

$$y_t = C_t x_t + v_t$$

dove:

- $_ \in \mathbb{R}^{n \times n}$ è la matrice dinamica di transizione di stato;
- $_ \in \mathbb{R}^{n \times m}$ è la matrice degli ingressi di controllo;
- $C_t \in \mathbb{R}^{p \times n}$ è la matrice di osservazione o di uscita sensoriale.

> **Il Ruolo del Modello Dinamico nella Stima**
>
> Come evidenziato dal docente a lezione, modellare la dinamica dello stato non significa necessariamente possedere le equazioni fisiche esatte delle forze o della meccanica del veicolo. Se dobbiamo stimare la traiettoria di un punto nello spazio, possiamo semplicemente ipotizzare un modello cinematico a velocità costante ($\dot{x} = v$) o ad accelerazione costante ($\ddot{x} = a$): *"quel punto si muove a velocità costante, perché ci risulta così fisicamente non ci interessa"*. La discrepanza tra il modello semplificato e la complessa realtà fisica viene assorbita statisticamente dal rumore di processo w_t.

Ipotesi Statistiche Sui Rumori

Le sequenze stocastiche che corrompono il sistema soddisfano le seguenti proprietà fondamentali:

1. **Rumore di Processo / Stato (w_t)**: sequenza bianca gaussiana a media nulla con matrice di covarianza simmetrica semidefinita (o definita) positiva $_ \geq 0$:

$$w_t \sim \mathcal{N}(0, _), \quad \mathbb{E}[w_t] = 0, \quad \mathbb{E}[w_t w_t^T] = __t$$

2. **Rumore di Misura (v_t)**: sequenza bianca gaussiana a media nulla con matrice di covarianza definita positiva $_ > 0$:

$$v_t \sim \mathcal{N}(0, _), \quad \mathbb{E}[v_t] = 0, \quad \mathbb{E}[v_t v_t^T] = __t$$

3. **Scorrelazione Mutua**: i rumori di processo e di misura sono reciprocamente indipendenti a qualsiasi istante:

$$\mathbb{E}[w_t v_t^T] = 0 \quad \forall t$$

4. **Condizione Iniziale Incerta**: lo stato iniziale x_0 è una variabile gaussiana con media x_0 e covarianza P_0, indipendente dai rumori per ogni $t \geq 0$:

$$x_0 \sim \mathcal{N}(x_0, P_0), \quad \mathbb{E}[x_0 w_t^T] = 0, \quad \mathbb{E}[x_0 v_t^T] = 0$$

> ⚠ **Convenzione Fondamentale del Corso: Notazione delle Covarianze Q ed R**
>
> Prestare la massima attenzione alla notazione utilizzata dal docente a lezione:
>
> - Q: matrice di covarianza del rumore di stato/processo w_t;
> - R: matrice di covarianza del rumore di misura v_t.
>
> Questa notazione è lo standard universale della **Teoria dei Controlli e del Filtraggio Ottimo**. Si segnala che alcuni testi di robotica (incluso talvolta il testo di Thrun *Probabilistic Robotics*) e librerie software invertono i simboli utilizzando R per il processo e Q per le misure. Nel corso e in questi appunti si adotta rigorosamente la convenzione dei controlli (Q per il processo, R per le misure).

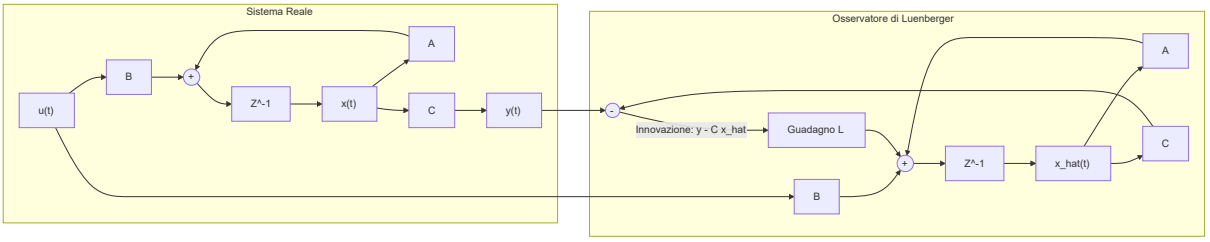
Il Ponte Con I Controlli Automatici: L'Osservatore Di Luenberger

Per comprendere l'architettura interna del filtro di Kalman consideriamo anzitutto il caso deterministico privo di rumore ($w_t = 0, v_t = 0$):

$$x_{t+1} = Ax_t + Bu_t, \quad y_t = Cx_t$$

Poiché lo stato interno x_t non è accessibile per la misura diretta (ad esempio la velocità reale del robot o la corrente di armatura), costruiamo un sistema dinamico ausiliario in grado di ricostruirlo asintoticamente conoscendo solo gli ingressi applicati u_t e le uscite misurate y_t. Questo sistema è l'**Osservatore di Stato di Luenberger**:

$$\hat{x}_{t+1} = A\hat{x}_t + Bu_t + L\left(y_t - C\hat{x}_t\right)$$



L'osservatore si articola in due componenti concettuali:

1. **Replica del Modello ($A\hat{x}_t + Bu_t$)**: simula in catena aperta l'evoluzione nominale del sistema sfruttando la conoscenza della dinamica e degli ingressi;
2. **Termine di Correzione in Feedback ($L(y_t - C\hat{x}_t)$)**: calcola l'**innovazione** (o residuo) $e_t = y_t - C\hat{x}_t$, ossia lo scostamento tra la misura reale y_t e la misura attesa calcolata sulla stima $C\hat{x}_t$, pesandola attraverso la matrice di guadagno L.

Dinamica dell'Errore Di Stima

Definiamo l'errore di stima $x_t = x_t - \hat{x}_t$. Sottraendo l'equazione dell'osservatore da quella del sistema deterministico:

$$x_{t+1} = x_{t+1} - \hat{x}_{t+1} = (Ax_t + Bu_t) - \left(A\hat{x}_t + Bu_t + LC(x_t - \hat{x}_t)\right)$$

$$x_{t+1} = (A - LC)x_t$$

L'evoluzione dell'errore è un sistema lineare autonomo governato dalla matrice dinamica $(A - LC)$. Se la coppia (A, C) è **osservabile**, possiamo scegliere ad arbitrio gli autovalori di $(A - LC)$ posizionandoli all'interno del disco unitario del piano complesso ($|\lambda_i| < 1$), garantendo che l'errore converga esponenzialmente a zero indipendentemente dall'errore iniziale:

$$\lim_{t \to \infty} x_t = 0 \implies \lim_{t \to \infty} \hat{x}_t = x_t$$

Nel contesto stocastico, tuttavia, la presenza simultanea del rumore di processo w_t e del rumore di misura v_t impedisce una convergenza deterministica a zero. Il **Filtro di Kalman** risolve esattamente questo trade-off calcolando il guadagno L_t ottimale che bilancia istante per istante la fiducia nella dinamica interna rispetto alla fiducia nei sensori.

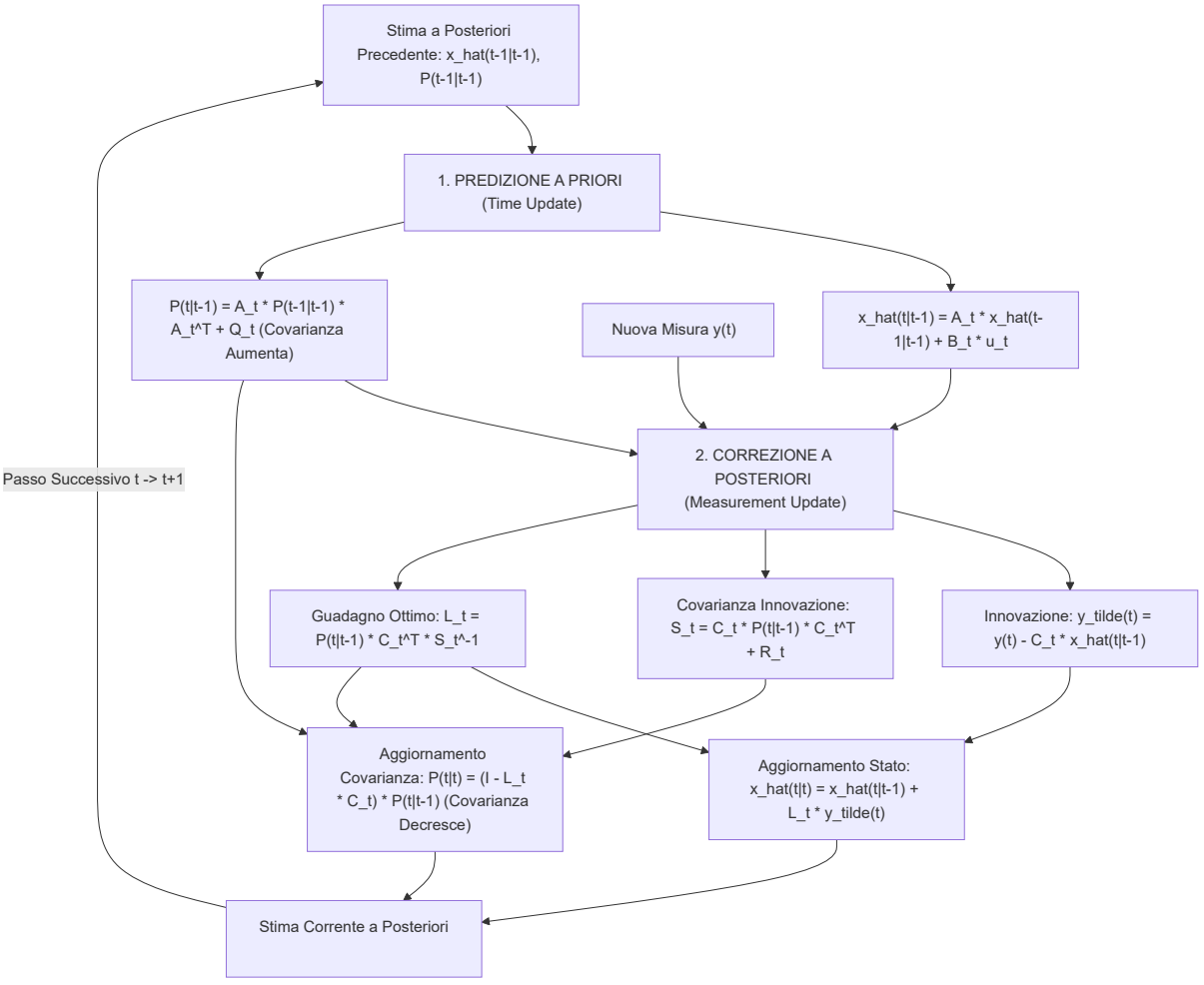
Il Filtro Di Kalman Lineare Time-Varying

Nel caso generale non stazionario le matrici del modello A_t, C_t, Q_t, R_t possono variare nel tempo.

Adottiamo la notazione standard di filtraggio ottimo a tempo discreto:

- $\hat{x}_{t|t-1}$: stima dello stato all'istante t basata sulle misure disponibili fino all'istante antecedente $t-1$ (**stima a priori** o predizione);
- $P_{t|t-1}$: matrice di covarianza dell'errore a priori: $\mathbb{E}[(x_t - \hat{x}_{t|t-1})(x_t - \hat{x}_{t|t-1})^T]$;
- $\hat{x}_{t|t}$: stima dello stato all'istante t aggiornata con l'osservazione corrente y_t (**stima a posteriori**);
- $P_{t|t}$: matrice di covarianza dell'errore a posteriori: $\mathbb{E}[(x_t - \hat{x}_{t|t})(x_t - \hat{x}_{t|t})^T]$.

L'algoritmo opera ciclicamente in due macro-fasi:



1. Fase Di Predizione a Priori (Time Update)

Conoscendo la stima passata $\hat{x}_{t-1|t-1}$ e il comando di controllo u_t, propaghiamo lo stato nel tempo applicando il modello nominale:

$$\hat{x}_{t|t-1} = A_t \hat{x}_{t-1|t-1} + B_t u_t$$

La covarianza dell'errore associata alla predizione è data da:

$$P_{t|t-1} = A_t P_{t-1|t-1} A_t^T + R_t$$

> **La Covarianza Aumenta nella Predizione**
>
> La trasformazione cinematica disperde l'incertezza ($A_t P_{t-1|t-1} A_t^T$) e vi somma la covarianza del rumore di processo $R_t \geq 0$. In assenza di misure sensoriali esterne, l'incertezza cresce a ogni passo temporale: il robot perde progressivamente accuratezza per via del drift odometrico.

2. Fase Di Correzione a Posteriori (Measurement Update)

All'istante t acquisiamo la lettura sensoriale y_t. Valutiamo lo scostamento rispetto alla misura attesa:

$$y_t = y_t - C_t \hat{x}_{t|t-1}$$

Il termine y_t è il processo di **innovazione**: rappresenta la frazione di informazione contenuta nella misura sensoriale che non era già prevedibile a partire dalla storia passata.

La matrice di covarianza dell'innovazione vale:

$$S_t = C_t P_{t|t-1} C_t^T + Q_t$$

Il **Guadagno Ottimo di Kalman** (o *Kalman Gain*) $L_t \in \mathbb{R}^{n \times p}$ è calcolato come:

$$L_t = P_{t|t-1} C_t^T S_t^{-1} = P_{t|t-1} C_t^T \left(C_t P_{t|t-1} C_t^T + Q_t \right)^{-1}$$

Aggiorniamo la stima dello stato correggendo la predizione con l'innovazione pesata da L_t:

$$\hat{x}_{t|t} = \hat{x}_{t|t-1} + L_t y_t$$

Infine, aggiorniamo la matrice di covarianza dell'errore a posteriori:

$$P_{t|t} = (I - L_t C_t)\, P_{t|t-1}$$

oppure nella forma esplicita simmetrica Joseph:

$$P_{t|t} = P_{t|t-1} - L_t S_t L_t^T$$

> **La Covarianza Decresce nella Correzione**
>
> Poiché $L_t S_t L_t^T$ è una matrice semidefinita positiva, la covarianza a posteriori soddisfa la disuguaglianza matriciale $P_{t|t} \leq P_{t|t-1}$. L'incorporazione della misura esterocettiva riduce rigorosamente l'incertezza della stima.

Corrispondenza Delle Notazioni

Grandezza Fisica / Concettuale	Notazione Controlli Ottimi (Prof. Valigi)	Notazione Robotica Probabilistica (Thrun)
Stato stimato a priori (predizione)	$\hat{x}_{t\|t-1}$	$\overline{bel}(x_t)$
Covarianza a priori	$P_{t\|t-1}$	$\bar{\Sigma}_t$
Stato stimato a posteriori	$\hat{x}_{t\|t}$	$bel(x_t)$
Covarianza a posteriori	$P_{t\|t}$	Σ_t
Matrice dinamica di transizione	A_t	A_t
Matrice degli ingressi di controllo	B_t	B_t
Matrice di osservazione / misura	C_t	C_t
Covarianza rumore di stato / processo	Q_t	R_t *(simbolo invertito!)*
Covarianza rumore di misura	R_t	Q_t *(simbolo invertito!)*
Guadagno di Kalman	L_t	K_t
Innovazione / Residuo	y_t	$z_t - C_t\bar{\mu}_t$
Covarianza dell'innovazione	S_t	$C_t\bar{\Sigma}_tC_t^T + Q_t$

Interpretazione Bayesiana Del Filtro Di Kalman

Come evidenziato dal docente (slide 21-22 di `013_2`), le equazioni del filtro di Kalman discendono in modo naturale dall'applicazione del teorema di Bayes a variabili aleatorie gaussiane.

La Predizione come Convoluzione

Nel filtro di Bayes la predizione è data dall'integrale:

$$\overline{bel}(x_t) = \int_{\mathbb{R}^n} p(x_t \mid x_{t-1}, u_t)\, bel(x_{t-1})\, \mathrm{d}x_{t-1}$$

Dato il modello dinamico lineare $x_t = A_t x_{t-1} + B_t u_t + w_t$, la probabilità di transizione condizionata è una gaussiana traslata:

$$p(x_t \mid x_{t-1}, u_t) = p_{w_t}(x_t - A_t x_{t-1} - B_t u_t) \sim \mathcal{N}(A_t x_{t-1} + B_t u_t,\ Q_t)$$

Poiché sia $bel(x_{t-1}) \sim \mathcal{N}(\hat{x}_{t-1|t-1}, P_{t-1|t-1})$ sia p_{w_t} sono gaussiane, l'integrale è una **convoluzione di funzioni di densità gaussiane**.

La convoluzione di gaussiane genera una densità identicamente gaussiana. Ne calcoliamo valore atteso e varianza applicando l'operatore speranza matematica:

$$\mathbb{E}[x_t] = \mathbb{E}[A_t x_{t-1} + B_t u_t + w_t] = A_t \hat{x}_{t-1|t-1} + B_t u_t = \hat{x}_{t|t-1}$$

$$\mathrm{Cov}(x_t) = \mathbb{E}\left[(A_t x_{t-1|t-1} + w_t)(A_t x_{t-1|t-1} + w_t)^T\right] = A_t P_{t-1|t-1} A_t^T + Q_t = P_{t|t-1}$$

Ritroviamo esattamente le equazioni del passo di Time Update.

La Correzione come Prodotto Bayesiano

Il passo di correzione bayesiana combina la prior e la verosimiglianza:

$$bel(x_t) = \eta\, p(y_t \mid x_t)\, \overline{bel}(x_t) = \eta\, p_{v_t}(y_t - C_t x_t)\, \overline{bel}(x_t)$$

Sia la prior $\overline{bel}(x_t) \sim \mathcal{N}(\hat{x}_{t|t-1}, P_{t|t-1})$ sia la verosimiglianza sensoriale $p(y_t \mid x_t) \sim \mathcal{N}(C_t x_t, R_t)$ sono gaussiane.

Il loro prodotto produce all'esponente la somma di due forme quadratiche nelle variabili di stato:

$$bel(x_t) \propto \exp\left(-\frac{1}{2}\left|(x_t - \hat{x}_{t|t-1})^T P_{t|t-1}^{-1}(x_t - \hat{x}_{t|t-1}) + (y_t - C_t x_t)^T \cdot^{-1}(y_t - C_t x_t)\right|\right)$$

Espandendo i termini e completando il quadrato matriciale (zz. Argomenti extra: Derivazione del Prodotto di Gaussiane), la distribuzione a posteriori $bel(x_t)$ si ricompone come una singola gaussiana:

$$bel(x_t) \sim \mathcal{N}(\hat{x}_{t|t}, P_{t|t})$$

con $\hat{x}_{t|t} = \hat{x}_{t|t-1} + L_t(y_t - C_t\hat{x}_{t|t-1})$ e $P_{t|t} = (\cdot - L_t C_t)P_{t|t-1}$.

L'equivalenza tra il Filtro di Bayes ideale e il Filtro di Kalman è perfetta e priva di approssimazioni.

Proprietà Di Ottimalità E Considerazioni Sulla Stabilità

Il filtro di Kalman gode di formidabili proprietà teoriche che ne spiegano il successo universale nell'ingegneria aerospaziale, robotica e industriale:

1. **BLUE (Best Linear Unbiased Estimator)**:
   - Se i rumori sono rigorosamente gaussiani, il filtro di Kalman è lo **stimatore ottimo assoluto**: minimizza l'errore quadratico medio per qualsiasi combinazione lineare dello stato, massimizza la verosimiglianza e coincide con la stima di massimo a posteriori (MAP);
   - Se i rumori non sono gaussiani ma hanno media nulla e covarianze note, il filtro di Kalman non è più l'ottimo assoluto tra tutti gli stimatori possibili, ma rimane dimostrabilmente il **miglior stimatore lineare** a minima varianza.
2. **Stabilità nei Sistemi Variabili nel Tempo (Time-Varying)**:
   - Come sottolineato dal docente a lezione (slide 20 di `013_2`), nel caso a guadagno variabile L_t non possiamo dare per scontata a priori la stabilità esponenziale incondizionata dell'errore di stima: se la matrice $(\cdot - L_t C_t)$ perde temporaneamente proprietà di contrazione, la covarianza d'errore può crescere;
   - In robotica mobile, quando il robot attraversa aree povere di feature geometriche (ad esempio un lungo corridoio uniforme privo di ostacoli longitudinali), la matrice C_t non garantisce la completa osservabilità di tutte le componenti di stato; tuttavia, il filtro rimane stabile dal punto di vista operativo e ripristina la convergenza non appena il veicolo interseca nuovi riferimenti sensoriali.
3. **Estensione Stazionaria (Steady-State Kalman Filter)**:
   - Se le matrici del modello sono invarianti nel tempo ($A_t = A$, $B_t = B$, $C_t = C$, $R_t = R$, $Q_t = Q$), sotto le condizioni di **rilevabilità** della coppia (A, C) e di **stabilizzabilità** della coppia $(A, Q^{1/2})$, la matrice di covarianza $P_{t|t-1}$ converge asintoticamente alla matrice costante P_∞ soluzione unica semidefinita positiva dell'**Equazione Algebrica Discreta di Riccati (DARE)**:

     $$P_\infty = A P_\infty A^T + Q - A P_\infty C^T \left(C P_\infty C^T + R\right)^{-1} C P_\infty A^T$$

   - Di conseguenza, il guadagno di Kalman converge a una matrice statica $L_\infty = P_\infty C^T (C P_\infty C^T + R)^{-1}$, trasformando il filtro in un osservatore asintotico a guadagno costante estremamente leggero da eseguire.

I passaggi algebrici completi della DARE e le dimostrazioni formali della convergenza sono riportati in zz. Argomenti extra: Filtro di Kalman Stazionario.

Zz. Argomenti Extra

Questo documento raccoglie dimostrazioni, approfondimenti matematici e argomenti collaterali utili alla comprensione completa degli argomenti del corso di *Autonomous Robotics*, ma non strettamente richiesti come parte teorica principale d'esame.

Metodi Di Integrazione ODE (Eulero, RK2, RK4)

Tutti i modelli cinematici continui dei robot mobili sono descritti da sistemi di equazioni differenziali ordinarie (ODE) del primo ordine:

$$\dot{\mathbf{x}} = f(\mathbf{x}(t), \mathbf{u}(t), t), \quad \mathbf{x}(t_0) = \mathbf{x}_0$$

Per simulare il comportamento del robot o calcolarne la posa su un controllore digitale dobbiamo integrare numericamente queste equazioni su una griglia temporale discreta con passo T costante ($t = kT$).

Metodo Di Eulero Esplicito (Ordine 1)

Il metodo più semplice consiste nell'approssimare la derivata temporale $\dot{\mathbf{x}}$ con il rapporto incrementale in avanti del primo ordine:

$$\dot{\mathbf{x}} \approx \frac{\mathbf{x}((k+1)T) - \mathbf{x}(kT)}{T}$$

Uguagliando al campo vettoriale calcolato all'inizio dell'intervallo:

$$\frac{\mathbf{x}((k+1)T) - \mathbf{x}(kT)}{T} \approx f(\mathbf{x}(kT), \mathbf{u}(kT), kT)$$

Otteniamo la formula ricorsiva di aggiornamento:

$$\mathbf{x}(k+1) = \mathbf{x}(k) + \Delta\mathbf{x}_E(k)$$

Dove l'incremento di Eulero è:

$$\Delta\mathbf{x}_E(k) = T\, f(\mathbf{x}(kT), \mathbf{u}(kT), kT)$$

- **Vantaggio**: richiede una sola valutazione della funzione f per passo (computazionalmente immediato).
- **Limite**: ha un errore locale di troncamento $O(T^2)$ e un errore globale $O(T)$. Se il passo T non è piccolissimo, la traiettoria accumula rapidamente errori visibili.

Metodo Di Runge-Kutta Al Secondo Ordine (RK2 / Midpoint)

Per aumentare l'accuratezza senza calcolare derivate analitiche superiori, i metodi Runge-Kutta valutano la funzione f in punti intermedi del passo temporale.

Il metodo RK2 (noto anche come metodo del punto medio) calcola due stadi:

1. Una prima stima di prova della pendenza all'inizio del passo:

$$k_1 = T\, f(\mathbf{x}(kT), \mathbf{u}(kT), kT)$$

2. Una seconda valutazione della pendenza nel punto medio dell'intervallo ($t + T/2$), usando lo stato stimato con metà incremento k_1:

$$k_2 = T\, f\left(\mathbf{x}(kT) + \frac{1}{2}k_1,\ \mathbf{u}\left(kT + \frac{T}{2}\right),\ kT + \frac{T}{2}\right)$$

La posa finale al passo successivo è:

$$\mathbf{x}(k+1) = \mathbf{x}(k) + k_2$$

Nel caso del robot a guida differenziale stazionario con ingressi tenuti costanti nel passo (*sample and hold*, $\mathbf{u}(t) = \mathbf{u}(k)$):

$$\mathbf{x}(k+1) = \mathbf{x}(k) + T\, f\left(\mathbf{x}(k) + \frac{1}{2}k_1,\ \mathbf{u}(k)\right)$$

Questo porta direttamente alle equazioni viste nel capitolo 0. Modelli Cinematici e di Movimento, dove la direzione viene ruotata di metà dell'angolo incrementale ($\theta_t + \frac{\omega_t T}{2}$).

- Ha un errore locale $O(T^3)$ e un errore globale $O(T^2)$.

Metodo Di Runge-Kutta Al Quarto Ordine (RK4)

Il metodo Runge-Kutta classico di quarto ordine rappresenta il punto di riferimento nell'integrazione numerica scientifica (è la base usata anche da risolutori come `scipy.integrate.odeint` o `ode45`).

Esegue quattro valutazioni intermedie del campo vettoriale:

$$
\begin{aligned}
k_1 &= T\, f(\mathbf{x}(kT), \mathbf{u}(kT), kT) \\
k_2 &= T\, f\left(\mathbf{x}(kT) + \tfrac{1}{2}k_1,\ \mathbf{u}\left(kT + \tfrac{T}{2}\right),\ kT + \tfrac{T}{2}\right) \\
k_3 &= T\, f\left(\mathbf{x}(kT) + \tfrac{1}{2}k_2,\ \mathbf{u}\left(kT + \tfrac{T}{2}\right),\ kT + \tfrac{T}{2}\right) \\
k_4 &= T\, f\left(\mathbf{x}(kT) + k_3,\ \mathbf{u}(kT + T),\ kT + T\right)
\end{aligned}
$$

Lo stato al passo successivo si ottiene combinando le quattro pendenze con una media pesata (regola di Simpson):

$$\mathbf{x}(k+1) = \mathbf{x}(k) + \frac{1}{6}k_1 + \frac{1}{3}k_2 + \frac{1}{3}k_3 + \frac{1}{6}k_4$$

- **Vantaggi**: garantisce un'accuratezza elevatissima con errore globale $O(T^4)$.
- **Costo**: richiede 4 valutazioni di f per ciascun passo di campionamento.

Tabella Comparativa Dei Metodi Di Integrazione

La formula generale di aggiornamento a un passo è:

$$\mathbf{x}(k+1) = \mathbf{x}(k) + \Delta f$$

Metodo	Incremento Δf	Valutazioni di f	Ordine Globale	Proprietà Geometrica
Eulero	$\Delta f_E = k_1$	1	$O(T)$	Avanzamento lungo la retta tangente iniziale
Runge-Kutta 2 (Midpoint)	$\Delta f_{K2} = k_2$	2	$O(T^2)$	Valutazione della pendenza nel punto medio temporale
Runge-Kutta 4	$\Delta f_{K4} = \frac{1}{6}k_1 + \frac{1}{3}k_2 + \frac{1}{3}k_3 + \frac{1}{6}k_4$	4	$O(T^4)$	Media pesata con integrazione di Simpson a 4 stadi

Derivazione Analitica Del Drift Quadratico Nelle IMU

Nelle Unità di Misura Inerziali (trattate in 2. Sensori per la Robotica Mobile), la stima della posizione mediante navigazione inerziale pura (*dead reckoning*) richiede la doppia integrazione delle accelerazioni lineari depurate dalla gravità:

$$(t) = (0) + (0)t + \int_0^t \left(\int_0^{'} \mathbf{a}_{\text{moto}}(s)\, s \right)$$

L'accelerazione di moto $\mathbf{a}^b_{\text{moto}}$ nel frame corpo b, viene ricavata sottraendo la gravità $\mathbf{g}^w = [0, 0, -]^T$:

$$\mathbf{a}^b_{\text{moto}} = \mathbf{a}^b_{\text{misurata}} - {}^b_w R(t)\, \mathbf{g}^w$$

Propagazione Dell'Errore Angolare Del Giroscopio

Supponiamo che i giroscopi siano affetti da un bias angolare costante residuo $\boldsymbol{\omega}$. L'orientamento stimato ${}^b_w \hat{R}(t)$ differisce dalla rotazione reale ${}^b_w R(t)$ per un errore angolare che cresce linearmente nel tempo:

$$\theta(t) \approx \boldsymbol{\omega} \cdot t$$

Nella compensazione della gravità, un errore angolare θ attorno a un asse orizzontale proietta una componente spuria del vettore di gravità $g \approx 9.8\ \mathrm{m/s^2}$ sugli assi orizzontali del robot:

$$\Delta_x(t) \approx g \sin(\theta(t)) \approx g\, \boldsymbol{\omega} \cdot t$$

Integrando doppiamente questa accelerazione spuria per ottenere la posizione orizzontale:

$$\Delta p_x(t) = \int_0^t \int_0^{t'} (g\, \boldsymbol{\omega} \cdot s)\, ds\, dt' = \frac{1}{6} g\, \boldsymbol{\omega} \cdot t^3$$

L'errore angolare nel giroscopio provoca quindi una deriva di posizione che cresce addirittura come $O(t^3)$ a causa dell'accoppiamento con la gravità.

Bias Costante Dell'Accelerometro E Deriva Quadratica $O(t^2)$

Se consideriamo invece un bias residuo costante proprio dell'accelerometro Δ_{bias} (anche assumendo l'orientamento perfettamente noto):

$$\Delta p(t) = \int_0^t \int_0^{t'} \Delta_{\text{bias}}\, ds\, dt' = \frac{1}{2} \Delta_{\text{bias}}\, t^2$$

L'errore di posizione cresce in modo rigorosamente **quadratico** $O(t^2)$.

Esempio Numerico Quantitativo

Consideriamo un accelerometro MEMS di fascia commerciale con un bias residuo di appena:

$$\Delta_{\text{bias}} = 0.01\ \mathrm{m/s^2} \quad (\approx 1\ \mathrm{mg})$$

Dopo un intervallo di navigazione libera di $t = 60\ \text{secondi}$ (un minuto), l'errore di posizionamento accumulato vale:

$$\Delta p(60) = \frac{1}{2}(0.01)(60)^2 = \frac{1}{2}(0.01)(3600) = 18 \text{metri}$$

Dopo 5 minuti ($300\ \mathrm{s}$), l'errore raggiunge:

$$\Delta p(300) = \frac{1}{2}(0.01)(90000) = 450\ \text{metri}$$

Questo risultato dimostra analiticamente l'impossibilità fisica di utilizzare una IMU come sensore di posizionamento stand-alone senza frequenti correzioni esterne assolute (GPS, LiDAR o telecamere).

Giroscopi Ottici FOG Ed Effetto Sagnac

Nelle applicazioni aerospaziali e nei veicoli autonomi ad alta affidabilità si impiegano i giroscopi a fibra ottica (**FOG**, *Fiber Optic Gyro*, come il sensore KVH 1750 citato nel capitolo principale).

Il FOG sfrutta l'**Effetto Sagnac**: due fasci di luce laser a stato solido vengono iniettati in versi opposti (orario e antiorario) all'interno di una bobina di fibra ottica avvolta per centinaia di metri o chilometri di lunghezza.

Se la bobina ruota con velocità angolare Ω attorno al proprio asse normale, il fascio concorde con la rotazione deve percorrere un tragitto ottico leggermente più lungo rispetto al fascio discorde. La differenza di cammino ottico ΔL genera uno sfasamento ottico di interferenza Δ_{ϕ}:

$$\Delta_{\phi} = \frac{4\pi N A}{\lambda_0 c} \Omega$$

dove:

- A è l'area racchiusa da ciascuna spira della bobina.
- N è il numero totale di spire di fibra ottica.
- λ_0 è la lunghezza d'onda del laser nel vuoto.
- c è la velocità della luce.

Rilevando interferometricamente lo sfasamento Δ_{ϕ}, il sensore misura la velocità angolare istantanea Ω con un bias di deriva estremamente contenuto (frazioni di decimi di grado all'ora, $0.01^\circ/h$), consentendo l'integrazione di rotta per durate temporali notevolmente superiori rispetto ai MEMS.

Geometria Della Visione Stereoscopica

Nella visione binoculare (citata in 2. Sensori per la Robotica Mobile), due telecamere identiche sono montate su una struttura rigida con assi ottici paralleli e distanziate orizzontalmente di una distanza nota b, detta **baseline**.

I piani immagine delle due telecamere sono complanari (condizione di telecamere rettificate). Un punto dello spazio tridimensionale $P = [X, Y, Z]^T$ (espresso nella terna della camera sinistra di riferimento) viene proiettato sui due sensori nei punti immagine:

- Camera sinistra: $p_L = (u_L, v_L)$
- Camera destra: $p_R = (u_R, v_R)$

Dalle equazioni del modello prospettico ideale a focale f:

$$u_L = f\frac{X}{Z}, \quad u_R = f\frac{X - b}{Z}, \quad v_L = v_R = f\frac{Y}{Z}$$

Calcolo Della Disparità E Della Profondità

La differenza tra le ascisse orizzontali sui due piani immagine definisce la **disparità** d:

$$d = u_L - u_R = f\frac{X}{Z} - f\frac{X - b}{Z} = \frac{f \cdot b}{Z}$$

Invertendo l'equazione rispetto a Z otteniamo la formula fondamentale della triangolazione stereoscopica:

$$Z = \frac{f \cdot b}{d}$$

Conoscendo la profondità Z, le coordinate 3D (X, Y) del punto nello spazio si ricostruiscono univocamente:

$$X = \frac{u_L \cdot Z}{f}, \quad Y = \frac{v_L \cdot Z}{f}$$

Analisi Dell'Errore Di Profondità

Differenziando la profondità z rispetto alla disparità d:

$$\frac{\partial z}{\partial d} = \frac{f \cdot b}{d^2} = \frac{z^2}{f \cdot b}$$

L'incertezza sulla profondità stimata Δz indotta da un errore di quantizzazione della disparità sui pixel Δd vale:

$$\Delta z = \frac{z^2}{f \cdot b} \Delta d$$

Questo evidenzia due limiti fisici strutturali della stereo camera:

1. **Incertezza quadratica con la distanza**: l'errore di profondità cresce con il quadrato della distanza z^2.
2. **Ruolo della baseline** b: aumentando la distanza tra le telecamere b si riduce l'errore a lungo raggio, ma aumenta il volume minimo cieco vicino al robot (*blind zone*) e la difficoltà di trovare corrispondenze di feature visive (corrispondenza stereo).

Modellistica Dello Pseudorange Nei Sistemi GNSS / GPS

Nei sistemi satellitari GNSS (presentati in 2. Sensori per la Robotica Mobile), ciascun satellite i-esimo trasmette in continuo un messaggio di navigazione con il proprio vettore di posizione tridimensionale orbitale $\mathbf{x}_i = [X_i, Y_i, Z_i]^T$ e il timestamp di trasmissione $t_{e,i}$ regolato da orologi atomici al cesio o rubidio.

Il ricevitore a terra registra il tempo di arrivo del segnale t_r tramite il proprio oscillatore locale (al quarzo). Poiché l'orologio locale del ricevitore è economico e presenta un disallineamento temporale (bias d'orologio) t_{rx} rispetto al tempo universale GPS, la misura grezza calcolata moltiplicando per la velocità della luce c non è la distanza geometrica vera, ma viene definita **pseudorange** ρ_i:

$$\rho_i = c \cdot (t_r - t_{e,i})$$

Equazione Di Misura A Quattro Incognite

Esplicitando la distanza euclidea tra satellite e ricevitore posto in $\mathbf{x} = [x, y, z]^T$:

$$\rho_i = \sqrt{(X_i - x)^2 + (Y_i - y)^2 + (Z_i - z)^2} + c \cdot t_{\text{rx}} + \varepsilon_i$$

dove ε_i raggruppa i disturbi ambientali di propagazione:

- Ritardo ionosferico e troposferico.
- Errori di effemeridi residue del satellite.
- Rumore termico di ricezione e riflessioni multipath.

Poiché le incognite dello stato da stimare sono **quattro**:

$$\mathbf{s} = [x,\ y,\ z,\ c \cdot t_{\text{rx}}]^T$$

è geometricamente indispensabile ricevere contemporaneamente il segnale da un numero di satelliti:

$$n_{\text{sat}} \geq 4$$

Risolvendo il sistema non lineare con algoritmi di minimizzazione ai minimi quadrati iterati (Gauss-Newton o Filtro di Kalman esteso), il ricevitore stima contemporaneamente la posizione tridimensionale metrica (x, y, z) e sincronizza perfettamente il proprio orologio locale con il tempo atomico GPS.

Principio Operativo Dell'RTK (Real Time Kinematic)

Nelle applicazioni di robotica agricola o veicolare (come la piattaforma Agrobot di ISARLab), la precisione metrica standard di $3 - 5\ \mathrm{m}$ è insufficiente per condurre un veicolo tra i filari. Si impiega pertanto la tecnologia **RTK (Real Time Kinematic)**.

Il sistema RTK prevede:

1. Una **Stazione Base fissa** a terra, installata su un punto geodetico di coordinate millimetriche note.
2. Il **Rover mobile** montato a bordo del robot.

Anziché limitarsi a decodificare i bit del codice pseudo-casuale (PRN) del segnale, i ricevitori RTK tracciano la **fase dell'onda portante radio** a frequenza $L_1 = 1575.42\ \mathrm{MHz}$, la cui lunghezza d'onda nello spazio vale appena:

$$\lambda = \frac{c}{f} = \frac{3 \times 10^{8}\ \mathrm{m/s}}{1.57542 \times 10^{9}\ \mathrm{Hz}} \approx 19.03\ \mathrm{cm}$$

Misurando lo sfasamento della frazione di ciclo dell'onda con risoluzione inferiore all'1%, e risolvendo in tempo reale il problema dell'**ambiguità intera di fase** (il numero intero incognito N di cicli d'onda completi interposti tra satellite e antenna mediante tecniche a doppie differenze tra Base e Rover), il sistema cancella quasi integralmente gli errori atmosferici e di satellite comuni.

La stima della posizione relativa del rover rispetto alla base fissa raggiunge un'accuratezza cinematica istantanea nell'ordine di $1\ \mathrm{centimetro}$.

Parametri Metrologici Avanzati E Fisica Dei Sensori

Questa sezione approfondisce le definizioni metrologiche e i modelli fisici accennati nell'Appendice della Lezione 5.

Quantificazione Del Dynamic Range In Decibel

Il range dinamico descrive la capacità del sensore di misurare contemporaneamente segnali di debole intensità e segnali di fondo scala senza saturazione:

- **Per grandezze energetiche o di potenza** (ad esempio l'intensità luminosa catturata dai fotodiodi del LiDAR o la potenza dei segnali RF):

$$\mathrm{DR}_{\mathrm{dB}} = 10 \log_{10}\left(\frac{P_{\max}}{P_{\min}}\right) \quad [\mathrm{dB}]$$

- **Per grandezze di ampiezza, tensione o campo** (tensioni dei piezometri, segnali elettrici dei ponti di Wheatstone):

$$\mathrm{DR}_{\mathrm{dB}} = 20 \log_{10}\left(\frac{x_{\max}}{x_{\min}}\right) \quad [\mathrm{dB}]$$

Un sensore ToF con un range dinamico di $80\ \mathrm{dB}$ in tensione è in grado di discriminare segnali il cui rapporto di ampiezza è di $10^{80/20} = 10^{4} = 10.000 : 1$.

Risoluzione Di Quantizzazione Nei Sensori Digitali (ADC)

La maggior parte dei trasduttori fisici genera una tensione o carica analogica che viene convertita in un dato digitale tramite un convertitore Analogico-Digitale (ADC) a b bit. Se l'intervallo di fondo scala dell'ADC è $[0, FS]$, la tensione è suddivisa in 2^b intervalli discreti.

La risoluzione teorica minima di quantizzazione Δ vale:

$$\Delta = \frac{FS}{2^b}$$

L'operazione di quantizzazione introduce un rumore uniformemente distribuito nell'intervallo $[-\frac{\Delta}{2}, +\frac{\Delta}{2}]$, con una varianza teorica intrinseca di rumore di quantizzazione:

$$\sigma_q^2 = \frac{(\Delta)^2}{12}$$

Distorsioni Hard-Iron E Soft-Iron Nelle Bussole Magnetiche

I magnetometri a stato solido rilevano il vettore del campo magnetico terrestre $\mathbf{B}_{\text{earth}}$. In un ambiente privo di metalli e correnti parassite, ruotando il sensore sul piano orizzontale di 360° le letture (x, y) descrivono una circonferenza perfetta centrata nell'origine:

$$x^2 + y^2 = \|\mathbf{B}_{\text{earth}}\|^2$$

Sul robot reale intervengono due effetti parassiti:

1. **Distorsione Hard-Iron**: generata da corpi ferromagnetici permanentemente magnetizzati montati a bordo del robot (altoparlanti, viti ferrose, telaio in acciaio, magneti permanenti dei motori DC). Produce un campo magnetico costante $\mathbf{b}_{\text{hard}}$ solidale alla terna del robot che trasla il centro della circonferenza:

$$\mathbf{B}_{\text{misurato}} = \mathbf{B}_{\text{reale}} + \mathbf{b}_{\text{hard}}$$

2. **Distorsione Soft-Iron**: generata da materiali ad alta permeabilità magnetica passivi (ferro dolce, schermature metalliche, piste di rame con correnti variabili) che deflettono e comprimono le linee di flusso del campo magnetico esterno a seconda dell'orientamento. È descritta da una matrice di deformazione 3×3 simmetrica M_{soft}, che deforma la circonferenza in un'ellisse inclinata.

Il modello completo di misura della bussola è:

$$\mathbf{B}_{\text{misurato}} = M_{\text{soft}}\, \mathbf{B}_{\text{earth}} + \mathbf{b}_{\text{hard}}$$

Procedura Di Calibrazione Ad Ellisse

Per calibrare il magnetometro:

1. Si fa compiere al robot una rotazione lenta di 360° raccogliendo un insieme di punti sul piano.
2. Si esegue un fitting di regressione ai minimi quadrati per individuare i parametri dell'ellisse (centro $\mathbf{b}_{\text{hard}}$, assi principali e inclinazione della matrice M_{soft}).
3. Si applica la trasformazione inversa:

$$\mathbf{B}_{\text{calibrato}} = M_{\text{soft}}^{-1}(\mathbf{B}_{\text{misurato}} - \mathbf{b}_{\text{hard}})$$

   riportando i dati su un cerchio unitario centrato nell'origine da cui ricavare l'angolo di rotta assoluto privo di offset:

$$\theta = \operatorname{atan2}(y_{\text{calibrato}}, x_{\text{calibrato}})$$

Meccanica Dei Micro-Giroscopi MEMS E Forza Di Coriolis

I giroscopi miniaturizzati realizzati con tecnologia microelettro-meccanica (MEMS) non possono includere rotori rotanti ad alta velocità a causa dell'usura per attrito e dei limiti di fabbricazione microscopica. Sfruttano invece **strutture oscillanti risonanti** (diapason, pettini elettrostatici capacitivi o campane semisferiche al silicio).

Una massa sismica microscopica m viene mantenuta in oscillazione sinusoidale continua a frequenza di risonanza ω, lungo un asse primario di attuazione x con velocità istantanea:

$$(t) = [v_x(t), 0, 0]^T$$

Quando il telaio del robot ruota attorno all'asse perpendicolare z con velocità angolare $= [0, 0, \Omega_z]^T$, nel sistema di riferimento non inerziale della massa si sviluppa la **forza di Coriolis** apparente:

$$\mathbf{F}_c = -2m(\times)$$

Calcolando il prodotto vettoriale:

$$\times = \begin{matrix} \mathbf{i} & \mathbf{j} & \mathbf{k} \\ 0 & 0 & \Omega_z \\ v_x & 0 & 0 \end{matrix} = [0, \Omega_z v_x, 0]^T$$

La forza di Coriolis agisce interamente lungo l'asse ortogonale y:

Questa forza ciclica induce un'oscillazione secondaria lungo l'asse y. La deflessione ortogonale viene misurata con elevata sensibilità rilevando la variazione differenziale di capacità ΔC tra elettrodi microscopici a pettine, fornendo un segnale di tensione proporzionale alla velocità angolare esterna Ω_z.

Derivazione Analitica Del Line Fitting (OLS E WLS) in Coordinate Polari

Nel contesto del rilevamento di ostacoli piani da scansioni LiDAR, la rappresentazione esplicita cartesiana $y = mx + $ fallisce quando la retta è parallela all'asse y (m). Si adotta la **forma normale di Hesse**:

$$x \cos + y \sin = 0$$

dove ≥ 0 è la distanza minima ortogonale dall'origine alla retta e $\in [-\pi, \pi]$ è l'angolo del vettore normale orientato.

Formulazione Del Problema Di Ottimizzazione OLS

Dato un insieme di N punti misurati nel piano $(x_i, y_i)_{i=1}^N$, la distanza ortogonale con segno (residuo geometrico) tra il generico punto P_i e la retta candidata è:

$$= x_i \cos + y_i \sin$$

L'obiettivo dei Minimi Quadrati Ordinari (OLS, *Total Least Squares*) è minimizzare la somma dei quadrati delle distanze perpendicolari:

$$S(,) = \sum_{i=1}^{N} {}^2 = \sum_{i=1}^{N} (x_i \cos + y_i \sin)^2$$

1. Ottimizzazione Rispetto Al Parametro Distanza

Calcoliamo la derivata parziale prima di S rispetto a e poniamola uguale a zero:

$$\frac{S}{} = -2 \sum_{i=1}^{N} (x_i \cos + y_i \sin) = 0$$

Dividendo per $-2N$:

$$\frac{1}{N} \sum_{i=1}^{N} x_i \cos + \frac{1}{N} \sum_{i=1}^{N} y_i \sin = 0$$

Definendo il baricentro (media campionaria) dei punti misurati come:

$$x = \frac{1}{N}\sum_{i=1}^{N} x_i, \quad y = \frac{1}{N}\sum_{i=1}^{N} y_i$$

Otteniamo la relazione fondamentale in forma chiusa per :

$$= x\cos + y\sin$$

Interpretazione geometrica: la retta ottima stimata passa inderogabilmente per il baricentro (x, y) dei punti misurati.

---

2. Ottimizzazione Rispetto all'Angolo Della Normale

Sostituendo l'espressione di all'interno del residuo :

$$= (x_i - x)\cos + (y_i - y)\sin$$

Definiamo per brevità le coordinate scarto dal baricentro:

$$x_i = x_i - x, \quad y_i = y_i - y$$

La funzione di costo S dipende ora unicamente da :

$$S() = \sum_{i=1}^{N} (x_i\cos + y_i\sin)^2 = \sum_{i=1}^{N} \left(x_i^2\cos^2 + y_i^2\sin^2 + 2x_iy_i\cos\sin\right)$$

Sfruttando le note identità trigonometriche di duplicazione degli angoli:

- $\cos^2 = \frac{1+\cos(2)}{2}$
- $\sin^2 = \frac{1-\cos(2)}{2}$
- $2\cos\sin = \sin(2)$

Raggruppiamo i termini:

$$S() = \frac{1}{2}\sum_{i=1}^{N}(x_i^2 + y_i^2) + \frac{1}{2}\cos(2)\sum_{i=1}^{N}(x_i^2 - y_i^2) + \sin(2)\sum_{i=1}^{N}(x_iy_i)$$

Deriviamo $S()$ rispetto ad e imponiamo la condizione di stazionarietà $\frac{S}{} = 0$:

$$\frac{S}{} = -\sin(2)\sum_{i=1}^{N}(x_i^2 - y_i^2) + 2\cos(2)\sum_{i=1}^{N}(x_iy_i) = 0$$

Separando le variabili trigonometriche:

$$\sin(2)\sum_{i=1}^{N}(x_i^2 - y_i^2) = 2\cos(2)\sum_{i=1}^{N}(x_iy_i)$$

Dividendo per $\cos(2)$:

$$\tan(2) = \frac{2\sum_{i=1}^{N} x_iy_i}{\sum_{i=1}^{N}(x_i^2 - y_i^2)} = \frac{-2\sum_{i=1}^{N}(x_i - x)(y_i - y)}{\sum_{i=1}^{N}(y_i - y)^2 - \sum_{i=1}^{N}(x_i - x)^2}$$

L'angolo della normale viene calcolato senza ambiguità di quadrante tramite la funzione $\mathrm{atan2}$:

$$= \frac{1}{2}\mathrm{atan2}\left(-2\sum_{i=1}^{N} x_i y_i,\ \sum_{i=1}^{N}(y_i^2 - x_i^2)\right)$$

Se il valore calcolato di $= x\cos + y\sin$ risulta negativo, si inverte la direzione della normale ($+\pi$, $-$) per preservare la convenzione standard ≥ 0.

Estensione Ai Minimi Quadrati Pesati (WLS)

Quando i singoli punti P_i presentano varianze eterogenee ^{2} (ad esempio a causa della dispersione radiale LiDAR che cresce con la distanza misurata), si introduce il peso stocastico:

$$w_i = \frac{1}{^2}$$

La funzione di costo WLS vale:

$$S_w(,) = \sum_{i=1}^{N} w_i(x_i\cos + y_i\sin -)^2$$

Il baricentro pesato si ottiene come:

$$x_w = \frac{w_i x_i}{w_i}, \quad y_w = \frac{w_i y_i}{w_i} \implies = x_w\cos + y_w\sin$$

E l'orientamento della normale diventa:

$$\tan(2) = \frac{-2\sum_{i=1}^{N} w_i(x_i - x_w)(y_i - y_w)}{\sum_{i=1}^{N} w_i(y_i - y_w)^2 - \sum_{i=1}^{N} w_i(x_i - x_w)^2}$$

Propagazione Delle Incertezze E Matrice Jacobiana $_{}$

I punti di misura laser non sono grandezze deterministiche, ma variabili aleatorie raggruppate nel vettore di misura $\in \mathbb{R}^{2N}$:

$$= \begin{bmatrix} & \theta_1 & & \theta_2 & \dots & & \theta_N & \end{bmatrix}^T$$

con matrice di covarianza congiunta a blocchi diagonali $C_P \in \mathbb{R}^{2N\times 2N}$. Di conseguenza, anche i parametri stimati della retta $= [,]^T \in \mathbb{R}^2$ sono variabili aleatorie con matrice di covarianza $C_ = C_ \in \mathbb{R}^{2\times 2}$:

$$C_ = \begin{vmatrix} ^2 & \\ & ^2 \end{vmatrix}$$

Applicando la legge di propagazione lineare delle covarianze mediante il Jacobiano della trasformazione di stima:

$$C_ = _ C_P \,^T_$$

dove $_ \in \mathbb{R}^{2\times 2N}$ è la matrice Jacobiana delle funzioni di stima () e () rispetto al vettore delle misure .

$$J_{h(x)} = \begin{bmatrix} \frac{\partial h_1}{\partial x_1} & \frac{\partial h_1}{\partial \theta_1} & \cdots & \frac{\partial h_1}{\partial x_N} & \frac{\partial h_1}{\partial \theta_N} \\ \frac{\partial h_2}{\partial x_1} & \frac{\partial h_2}{\partial \theta_1} & \cdots & \frac{\partial h_2}{\partial x_N} & \frac{\partial h_2}{\partial \theta_N} \end{bmatrix}$$

La conoscenza esplicita di C_t è indispensabile nei filtri di localizzazione e SLAM (es. EKF-SLAM basato su linee): consente al robot di calcolare l'ellisse di confidenza e la distanza di Mahalanobis tra le rette osservate all'istante t e le pareti già censite nella mappa globale.

Formulazione Matriciale Dell'Histogram Filter Come Catena Di Markov

Nel capitolo 5. Localizzazione con Histogram Filter abbiamo espresso l'aggiornamento dell'Histogram Filter tramite cicli iterativi `for` sulle singole celle x_k e x_i. Quando lo spazio delle pose viene discretizzato in un numero finito K di celle, il filtro di Bayes discreto può essere formulato con notevole eleganza e potenza computazionale attraverso l'algebra lineare delle **Catene di Markov a tempo discreto (DTMC)**.

Vettore Di Stato E Matrice Di Transizione Stocastica

Rappresentiamo la distribuzione di probabilità dell'intero spazio all'istante t come un vettore colonna di probabilità $\mathbf{p}_t \in \mathbb{R}^K$:

$$\mathbf{p}_t = \begin{bmatrix} p_{1,t} \\ p_{2,t} \\ \cdot \\ p_{K,t} \end{bmatrix} = \begin{bmatrix} bel_t(x_1) \\ bel_t(x_2) \\ \cdot \\ bel_t(x_K) \end{bmatrix}$$

con i vincoli di non negatività $p_{k,t} \geq 0$ e conservazione della massa probabilistica $\sum_{k=1}^{K} p_{k,t} = \mathbf{1}^T \mathbf{p}_t = 1$, dove $\mathbf{1} = [1, 1, \ldots, 1]^T$.

Definiamo la **matrice di transizione di stato** $T(u_t) \in \mathbb{R}^{K \times K}$ associata al comando di controllo u_t:

$$T_{k,i}(u_t) := p(X_t = x_k \mid X_{t-1} = x_i, u_t, M)$$

L'elemento $T_{k,i}$ rappresenta la probabilità di transizione dalla cella di partenza i (colonna) alla cella di destinazione k (riga). La matrice T gode della proprietà fondamentale di essere **colonna-stocastica** (ogni colonna somma a 1):

$$\sum_{k=1}^{K} T_{k,i}(u_t) = 1, \quad \forall i \in 1, \ldots, K, \; \mathbf{1}^T T(u_t) = \mathbf{1}^T$$

Passo Di Predizione Come Prodotto Matrice-Vettore

L'equazione di predizione del filtro discreto (sommatoria su tutti gli stati antecedenti x_i) si riscrive come un unico prodotto matrice-vettore:

$$p_{k,t} = \sum_{i=1}^{K} T_{k,i}(u_t)\, p_{i,t-1} \implies \bar{\mathbf{p}}_t = T(u_t)\, \mathbf{p}_{t-1}$$

Poiché $T(u_t)$ è colonna-stocastica, la conservazione della probabilità a priori è garantita automaticamente dall'algebra lineare:

$$\mathbf{1}^T \bar{\mathbf{p}}_t = \mathbf{1}^T (T(u_t)\, \mathbf{p}_{t-1}) = (\mathbf{1}^T T(u_t))\, \mathbf{p}_{t-1} = \mathbf{1}^T \mathbf{p}_{t-1} = 1$$

Struttura A Banda E Ottimizzazione Sparsa

Nel caso di moto con rumore gaussiano stazionario, la probabilità $T_{k,i}$ decade esponenzialmente all'aumentare della distanza tra le celle $|x_k - (x_i + u_x)|$. Di conseguenza:

- In un ambiente 1D con moto uniforme, $T(u_t)$ assume la struttura di una **matrice di Toeplitz a banda**;

- Ponendo a zero le transizioni oltre la soglia 3_{x} (dove la densità gaussiana è trascurabile), la matrice diventa sparsa con larghezza di banda $.K$;
- La complessità computazionale del passo di predizione crolla da $O(K^2)$ a $O(K \cdot)$, consentendo implementazioni ad alte prestazioni tramite librerie di algebra lineare sparsa (es. `scipy.sparse`).

Passo Di Correzione E Prodotto Di Hadamard

Definiamo il vettore di verosimiglianza delle misure sensoriali $\mathbf{l}_t \in \mathbb{R}^K$, il cui k-esimo elemento è la densità valutata sull'ipotesi x_k:

$$\mathbf{l}_t = \begin{bmatrix} p(z_t \mid X_t = x_1, M) \\ p(z_t \mid X_t = x_2, M) \\ . \\ p(z_t \mid X_t = x_K, M) \end{bmatrix}$$

Il passo di aggiornamento di misura corrisponde al **prodotto elemento per elemento di Hadamard** (indicato con) tra il vettore di likelihood e il vettore di predizione, riscalato dal fattore di normalizzazione η:

$$= \eta \, (\mathbf{l}_t \; t)$$

dove il coefficiente scalare di normalizzazione η è il reciproco del prodotto interno:

$$\eta = \frac{1}{^T \, (\mathbf{l}_t \; t)} = \frac{1}{\mathbf{l}_t^T \; t}$$

Dinamica Asintotica E Teorema Di Perron-Frobenius

Se il robot si muove nell'ambiente senza effettuare alcuna misura correttiva (z_t assente per N passi consecutivi), il belief evolve come:

$$= T^N \; _0$$

Se la matrice di transizione è irriducibile e aperiodica (condizione tipica garantita dal rumore gaussiano che connette debolmente ogni stato a tutti gli altri), il **teorema di Perron-Frobenius** assicura che T ammette un unico autovalore dominante $_1 = 1$, e che per N il vettore converge all'autovettore stazionario $$

$$\lim_{N \to \infty} = \quad \text{con} \quad T =$$

Nel caso di rumore di traslazione simmetrico con pareti riflettenti o periodiche, tale stato asintotico corrisponde alla **distribuzione uniforme** $= [1/K, \ldots, 1/K]^T$: in assenza di retroazione sensoriale, l'accumulo ininterrotto dell'incertezza odometrica cancella ogni informazione pregressa conducendo alla totale perdita di localizzazione.

Derivazione Formale Della Traiettoria Completa Nel Particle Filter

Nella trattazione standard della localizzazione robotica (Monte Carlo Localization), il filtro a particelle calcola ricorsivamente la distribuzione di probabilità a posteriori della sola posa istantanea:

$$bel(x_t) = p(x_t \mid z_{1:t}, u_{1:t})$$

Tuttavia, dal punto di vista dell'analisi teorica dei processi stocastici (come illustrato nelle lezioni avanzate del Prof. Valigi, slide 38–39), il Particle Filter trova la sua giustificazione analitica più rigorosa estendendo lo stato all'intera sequenza storica delle pose attraversate dal robot:

$$x_{0:t} = x_0, x_1, \ldots, x_t$$

Il belief della traiettoria completa è definito come la densità congiunta condizionata a tutte le osservazioni e azioni passate:

$$bel(x_{0:t}) = p(x_{0:t} \mid z_{1:t}, u_{1:t})$$

Espansione Ricorsiva Tramite Regola Di Bayes E Proprietà Di Markov

Isoliamo l'osservazione più recente z_t dal set di tutte le misure sensoriali $z_{1:t} = z_t, z_{1:t-1}$:

$$bel(x_{0:t}) = p(x_{0:t} \mid z_t, z_{1:t-1}, u_{1:t})$$

Applicando la regola di Bayes considerando z_t come l'evidenza osservata condizionata alla traiettoria $x_{0:t}$, alla storia delle misure passate $z_{1:t-1}$ e ai comandi $u_{1:t}$:

$$bel(x_{0:t}) = \frac{p(z_t \mid x_{0:t}, z_{1:t-1}, u_{1:t})\, p(x_{0:t} \mid z_{1:t-1}, u_{1:t})}{p(z_t \mid z_{1:t-1}, u_{1:t})} = \eta\, p(z_t \mid x_{0:t}, z_{1:t-1}, u_{1:t})\, p(x_{0:t} \mid z_{1:t-1}, u_{1:t})$$

Sfruttiamo l'**assunzione markoviana del modello di misura**: la lettura del sensore z_t all'istante t è condizionatamente indipendente da tutte le pose passate $x_{0:t-1}$, dalle misure pregresse $z_{1:t-1}$ e dai comandi di controllo $u_{1:t}$, una volta nota la posa istantanea attuale x_t:

$$p(z_t \mid x_{0:t}, z_{1:t-1}, u_{1:t}) = p(z_t \mid x_t)$$

Sostituendo nella formulazione del belief:

$$bel(x_{0:t}) = \eta\, p(z_t \mid x_t)\, p(x_{0:t} \mid z_{1:t-1}, u_{1:t})$$

Scomponiamo ora il termine congiunto $p(x_{0:t} \mid z_{1:t-1}, u_{1:t})$ applicando la definizione di probabilità condizionata per isolare l'ultimo stato x_t rispetto alla storia $x_{0:t-1}$:

$$p(x_{0:t} \mid z_{1:t-1}, u_{1:t}) = p(x_t \mid x_{0:t-1}, z_{1:t-1}, u_{1:t})\, p(x_{0:t-1} \mid z_{1:t-1}, u_{1:t})$$

Applichiamo nuovamente l'**assunzione markoviana del modello di moto**:

1. Lo stato attuale x_t dipende unicamente dalla posa immediatamente precedente x_{t-1} e dal comando di attuazione corrente u_t:
$$p(x_t \mid x_{0:t-1}, z_{1:t-1}, u_{1:t}) = p(x_t \mid x_{t-1}, u_t)$$
2. La storia passata delle pose $x_{0:t-1}$ è indipendente dall'azione futura u_t:
$$p(x_{0:t-1} \mid z_{1:t-1}, u_{1:t}) = p(x_{0:t-1} \mid z_{1:t-1}, u_{1:t-1}) = bel(x_{0:t-1})$$

Ricongiungendo tutti i termini, otteniamo l'equazione ricorsiva fondamentale per il belief della traiettoria:

$$bel(x_{0:t}) = \eta\, p(z_t \mid x_t)\, p(x_t \mid x_{t-1}, u_t)\, bel(x_{0:t-1})$$

Definendo il belief a priori della traiettoria (dopo il comando di moto ma prima della misura) come:

$$\overline{bel}(x_{0:t}) = p(x_t \mid x_{t-1}, u_t)\, bel(x_{0:t-1})$$

L'espressione si compatta nella forma canonica:

$$bel(x_{0:t}) = \eta\, p(z_t \mid x_t)\, \overline{bel}(x_{0:t})$$

Corrispondenza Tra Target, Proposal E Pesi Di Importanza

Questa derivazione analitica mostra esattamente la convergenza matematica con la teoria dell'**Importance Sampling**:

- La **distribuzione target** (quella reale da cui vorremmo campionare) è la posterior congiunta $f(x_{0:t}) = bel(x_{0:t})$;
- La **distribuzione proposal** (quella operativa da cui siamo capaci di campionare ricorsivamente per propagazione stocastica) è la prior congiunta $\overline{bel}(x_{0:t})$;
- Il **fattore di importanza analitico** è il rapporto puntuale tra le due distribuzioni:

$$w(x_{0:t}) = \frac{f(x_{0:t})}{\overline{bel}(x_{0:t})} = \frac{bel(x_{0:t})}{\overline{bel}(x_{0:t})} = \frac{\eta\, p(z_t \mid x_t)\, \overline{bel}(x_{0:t})}{\overline{bel}(x_{0:t})} = \eta\, p(z_t \mid x_t)$$

Poiché nelle applicazioni pratiche il fattore di normalizzazione η viene calcolato a valle tramite somma empirica sui pesi estratti:

$$\eta = \Big(\sum_{m=1}^{M} w_t^{[m]}\Big)^{-1}$$

Il peso non normalizzato assegnato a ciascuna particella m-esima al passo t coincide esattamente con la verosimiglianza di misura valutata sulla posa corrente:

$$w_t^{[m]} = p(z_t \mid x_t^{[m]})$$

Marginalizzazione E Riduzione Al Filtro Sulla Posa Istantanea

Nelle applicazioni di localizzazione robotica ordinaria, memorizzare l'intera traiettoria $x_{0:t}$ per ciascuna delle M particelle comporterebbe un consumo lineare di memoria $O(M \cdot t)$ che divergerebbe nel tempo.

Poiché il robot necessita esclusivamente di conoscere dove si trova all'istante corrente t per pianificare il controllo, la distribuzione di interesse $bel(x_t)$ si ottiene **marginalizzando** il belief congiunto rispetto a tutte le pose passate $x_{0:t-1}$:

$$bel(x_t) = \int \dots \int bel(x_{0:t})\, \mathrm{d}x_0 \dots \mathrm{d}x_{t-1}$$

Nell'approssimazione a particelle, questa marginalizzazione corrisponde a un'operazione computazionale a costo zero: è sufficiente **dimenticare la storia passata** delle particelle e memorizzare unicamente il vettore dello stato corrente $x_t^{[m]}$ per $m = 1, \dots, M$. Ogni particella al passo t è un campione estratto dalla densità marginale corretta $bel(x_t)$.

Implementazione Industriale Di AMCL in ROS: Analisi Del Codice C Sorgente

Nel framework robotico **ROS / ROS2 (Nav2)**, il package di localizzazione 2D per robot mobili su mappe a griglia è `amcl` (all'interno del repository `ros-planning/navigation`).

L'implementazione in linguaggio C originaria fonde congiuntamente l'adattamento della cardinalità M tramite **KLD-Sampling** e il recupero dal rapimento tramite **Augmented MCL**. Analizziamo le quattro funzioni e routine cardine del sorgente di `amcl`.

1. Calcolo Del Limite Dinamico KLD: `pf_resample_limit`

La funzione `pf_resample_limit(pf_t *pf, int k)` calcola il numero di particelle necessario n dato il numero k di bin non vuoti nell'istogramma KD-Tree:

```
// Compute the required number of samples, given that there are k
// bins with samples in them. This is taken directly from Fox et al.
int pf_resample_limit(pf_t *pf, int k)
{
    double a, b, c, x;
```

```
    int n;

    if (k <= 1)
        return pf->max_samples;

    a = 1.0;
    b = 2.0 / (9.0 * ((double) k - 1.0));
    c = sqrt(2.0 / (9.0 * ((double) k - 1.0))) * pf->pop_z;
    x = a - b + c;

    n = (int) ceil((k - 1.0) / (2.0 * pf->pop_err) * x * x * x);

    if (n < pf->min_samples)
        return pf->min_samples;
    if (n > pf->max_samples)
        return pf->max_samples;

    return n;
}
```

- `pf->pop_err`: corrisponde al parametro ϵ (`kld_err`, default 0.05);
- `pf->pop_z`: quantile normale $z_{1-\delta}$ (`kld_z`, default $0.9 \Longrightarrow 2.326$);
- Il risultato viene saturato tra `min_samples` (default 500) e `max_samples` (default 5000).

2. Aggiornamento Pesi E Filtri Esponenziali: `pf_update_sensor`

All'arrivo di una nuova scansione laser, la funzione valuta il modello di verosimiglianza per ciascun campione e aggiorna le medie mobili esponenziali a lungo e breve termine (w_{slow} e w_{fast}):

```
void pf_update_sensor(pf_t *pf, pf_sensor_model_fn_t sensor_fn, void *sensor_data)
{
    int i;
    pf_sample_set_t *set;
    pf_sample_t *sample;
    double total = 0.0, w_avg = 0.0;

    set = pf->sets + pf->current_set;

    // Calcolo del peso e accumulo della somma
    for (i = 0; i < set->sample_count; i++)
    {
        sample = set->samples + i;
        sample->weight = (*sensor_fn)(sensor_data, sample->pose);
        total += sample->weight;
    }

    // Normalizzazione dei pesi e calcolo della verosimiglianza media
    for (i = 0; i < set->sample_count; i++)
    {
        sample = set->samples + i;
        w_avg += sample->weight;
        sample->weight /= total;
    }
    w_avg /= set->sample_count;

    // Aggiornamento filtri Augmented MCL (Probabilistic Robotics, p. 258)
    if (pf->w_slow == 0.0)
        pf->w_slow = w_avg;
    else
        pf->w_slow += pf->alpha_slow * (w_avg - pf->w_slow);

    if (pf->w_fast == 0.0)
        pf->w_fast = w_avg;
```

```
    else
        pf->w_fast += pf->alpha_fast * (w_avg - pf->w_fast);
}
```

3. Ricampionamento Con Iniezione Casuale: `pf_update_resample`

La funzione genera la nuova popolazione alternando il campionamento proporzionale alla CDF all'iniezione di pose casuali:

```
void pf_update_resample(pf_t *pf)
{
    pf_sample_set_t *set_a, *set_b;
    pf_sample_t *sample_a, *sample_b;
    double *c, r, w_diff;
    int i;

    set_a = pf->sets + pf->current_set;
    set_b = pf->sets + (pf->current_set + 1) % 2;

    // Costruzione della tabella di probabilità cumulativa (CDF)
    c = (double *) malloc(sizeof(double) * (set_a->sample_count + 1));
    c[0] = 0.0;
    for (i = 0; i < set_a->sample_count; i++)
        c[i + 1] = c[i] + set_a->samples[i].weight;

    // Calcolo della discrepanza tra filtro veloce e lento
    w_diff = 1.0 - pf->w_fast / pf->w_slow;
    if (w_diff < 0.0)
        w_diff = 0.0;

    set_b->sample_count = 0;

    while (set_b->sample_count < pf->max_samples)
    {
        sample_b = set_b->samples + set_b->sample_count++;

        // Iniezione probabilistica di pose casuali nello spazio libero
        if (drand48() < w_diff)
        {
            sample_b->pose = (pf->random_pose_fn)(pf->random_pose_data);
        }
        else
        {
            // Campionamento su CDF
            r = drand48();
            for (i = 0; i < set_a->sample_count; i++)
            {
                if ((c[i] <= r) && (r < c[i + 1]))
                    break;
            }
            sample_a = set_a->samples + i;
            sample_b->pose = sample_a->pose;
        }

        sample_b->weight = 1.0;
        // Inserimento nel KD-Tree per determinare k e il limite KLD
        pf_kdtree_insert(set_b->kdtree, sample_b->pose, sample_b->weight);

        // Verifica del limite di campioni dinamico KLD
        if (set_b->sample_count > pf_resample_limit(pf, set_b->kdtree->leaf_count))
            break;
    }

    // Reset delle medie per evitare derive divergenti
    if (w_diff > 0.0)
```

```
        pf->w_slow = pf->w_fast = 0.0;

    free(c);
}
```

4. Configurazione Dei Parametri in ROS Launch File

Nel file di configurazione (`amcl.launch` o `nav2_params.yaml`) i parametri chiave governano direttamente queste routine:

```
<launch>
  <node pkg="amcl" type="amcl" name="amcl" output="screen">
    <!-- Popolazione di particelle adattiva KLD -->
    <param name="min_particles" value="500"/>
    <param name="max_particles" value="5000"/>
    <param name="kld_err" value="0.05"/>
    <param name="kld_z" value="0.99"/>

    <!-- Modello odometrico di moto -->
    <param name="odom_model_type" value="diff"/>
    <param name="odom_alpha1" value="0.2"/>
    <param name="odom_alpha2" value="0.2"/>
    <param name="odom_alpha3" value="0.8"/>
    <param name="odom_alpha4" value="0.2"/>

    <!-- Modello sensoriale (Likelihood Field preferito) -->
    <param name="laser_model_type" value="likelihood_field"/>
    <param name="laser_z_hit" value="0.95"/>
    <param name="laser_z_rand" value="0.05"/>
    <param name="laser_sigma_hit" value="0.2"/>
    <param name="laser_likelihood_max_dist" value="2.0"/>

    <!-- Recupero dal rapimento (Augmented MCL) -->
    <param name="recovery_alpha_slow" value="0.001"/>
    <param name="recovery_alpha_fast" value="0.1"/>
  </node>
</launch>
```

Dettaglio Analitico Dei Modelli Di Misura Beam E Stima MLE Dei Parametri

Nel Capitolo 7 abbiamo introdotto il modello a fascio composto da quattro distribuzioni. Esplicitiamo qui le formule di normalizzazione analitiche e l'algoritmo di stima dei parametri.

Normalizzazione Delle Componenti Del Beam Model

Ciascuna densità di probabilità deve integrare a 1 sull'intervallo di misura consentito dal sensore $[0, z_{\max}]$:

1. **Componente Gaussiana Locale (p_{hit})**:

$$\int_0^{z_{\max}} p_{\text{hit}}(z_k \mid x_t, m)\, \mathrm{d}z_k = 1 \implies \eta_{\text{hit}} = \left(\int_0^{z_{\max}} \frac{1}{2\pi\sigma_{\text{hit}}^2} \exp\left(-\frac{(z_k - z_t^{k*})^2}{2\sigma_{\text{hit}}^2} \right) \mathrm{d}z_k \right)^{-1}$$

   In termini della funzione errore gaussiana $\operatorname{erf}(\cdot)$:

$$\eta_{\text{hit}} = \left[\frac{1}{2} \left(\operatorname{erf}\left(\frac{z_{\max} - z_t^{k*}}{\sqrt{2}\sigma_{\text{hit}}} \right) - \operatorname{erf}\left(\frac{-z_t^{k*}}{\sqrt{2}\sigma_{\text{hit}}} \right) \right) \right]^{-1}$$

2. **Componente Esponenziale per Ostacoli Imprevisti (p_{short})**:

$$\int_0^{z_t^{k^*}} \eta_{\text{short}} \lambda_{\text{short}} e^{-\lambda_{\text{short}} z_k} \, dz_k = 1 \implies \eta_{\text{short}} = \frac{1}{1 - e^{-\lambda_{\text{short}} z_t^{k^*}}}$$

Stima Dei Parametri Tramite Algoritmo Expectation-Maximization (EM)

Sia dato un dataset di calibrazione formato da M misurazioni note in cui il robot si trova in pose note: $\mathcal{Z} = \{z_j\}_{j=1}^{M}$ con valori ideali attesi $\mathcal{Z}^* = \{z_j^*\}_{j=1}^{M}$. Vogliamo stimare i pesi $\boldsymbol{\theta} = (\theta_{\text{hit}}, \theta_{\text{short}}, \theta_{\max}, \theta_{\text{rand}})^T$ e i parametri $\sigma_{\text{hit}}, \lambda_{\text{short}}$.

Poiché per ciascuna lettura z_j non conosciamo a quale dei quattro fenomeni fisici sia imputabile, introduciamo quattro variabili indicatrici latenti binarie $\mathbf{e}_j = [e_{j,\text{hit}}, e_{j,\text{short}}, e_{j,\max}, e_{j,\text{rand}}]^T \in \{0,1\}^4$ tali che $\sum_i e_{j,i} = 1$.

L'algoritmo EM itera due passi fino a convergenza:

1. **Expectation Step (E-step)**: calcolo delle responsabilità a posteriori per ogni misura j:

$$\gamma_{j,i} = \mathbb{E}[e_{j,i}] = \frac{\theta_i \, p_i(z_j \mid z_j^*)}{\sum_{k \in \{\text{hit},\text{short},\max,\text{rand}\}} \theta_k \, p_k(z_j \mid z_j^*)}$$

2. **Maximization Step (M-step)**: aggiornamento in forma chiusa dei parametri:
   - Aggiornamento dei pesi di miscela:

$$\theta_i^{\text{new}} = \frac{1}{M} \sum_{j=1}^{M} \gamma_{j,i}$$

   - Aggiornamento della deviazione standard del rumore di misura:

$$\sigma_{\text{hit}}^{\text{new}} = \sqrt{\frac{\sum_{j=1}^{M} \gamma_{j,\text{hit}} (z_j - z_j^*)^2}{\sum_{j=1}^{M} \gamma_{j,\text{hit}}}}$$

   - Aggiornamento del parametro di decadimento esponenziale:

$$\lambda_{\text{short}}^{\text{new}} = \frac{\sum_{j=1}^{M} \gamma_{j,\text{short}}}{\sum_{j=1}^{M} \gamma_{j,\text{short}} \, z_j}$$

Questo ciclo converge a un massimo locale della log-verosimiglianza del dataset di calibrazione, fornendo una taratura statisticamente fondata per il modello sensoriale del veicolo.

Derivazione Formale Del Filtro Di Kalman Lineare Dalla Teoria Bayesiana

Nel Capitolo 8 abbiamo enunciato le formule di aggiornamento del Filtro di Kalman. Riportiamo qui la derivazione analitica rigorosa basata sul completamento del quadrato matriciale e sulla minimizzazione dell'errore quadratico medio.

Passo Di Correzione: Completamento Del Quadrato Matriciale

Sia dato il belief a priori:

$$\overline{bel}(x_t) = \frac{1}{(2\pi)^{n/2} |P_{t|t-1}|^{1/2}} \exp\left(-\frac{1}{2} (x_t - \hat{x}_{t|t-1})^T P_{t|t-1}^{-1} (x_t - \hat{x}_{t|t-1}) \right)$$

e la verosimiglianza di misura lineare $y_t = C_t x_t + v_t$:

$$p(y_t \mid x_t) = \frac{1}{(2\pi)^{p/2} |R_t|^{1/2}} \exp\left(-\frac{1}{2} (y_t - C_t x_t)^T R_t^{-1} (y_t - C_t x_t) \right)$$

La posterior bayesiana non normalizzata ha all'esponente la forma quadratica:

$$(x_t) = (x_t - \hat{x}_{t|t-1})^T P_{t|t-1}^{-1}(x_t - \hat{x}_{t|t-1}) + (y_t - C_t x_t)^T {}_{t}^{-1}(y_t - C_t x_t)$$

Espandendo i prodotti matriciali:

$$(x_t) = x_t^T P_{t|t-1}^{-1} x_t - 2x_t^T P_{t|t-1}^{-1}\hat{x}_{t|t-1} + \hat{x}_{t|t-1}^T P_{t|t-1}^{-1}\hat{x}_{t|t-1} + x_t^T C_t^{T}{}_{t}^{-1} C_t x_t - 2x_t^T C_t^{T}{}_{t}^{-1} y_t + y_t^T {}_{t}^{-1} y_t$$

Raggruppiamo i termini quadratici e lineari in x_t:

$$(x_t) = x_t^T \underbrace{\left(P_{t|t-1}^{-1} + C_t^{T}{}_{t}^{-1} C_t\right)}_{P_{t|t}^{-1}} x_t - 2x_t^T \underbrace{\left(P_{t|t-1}^{-1}\hat{x}_{t|t-1} + C_t^{T}{}_{t}^{-1} y_t\right)}_{v} + \text{costante}$$

Dall'algebra delle gaussiane multivariate, se l'esponente ha forma $x_t^T P_{t|t}^{-1} x_t - 2x_t^T v$, la matrice di covarianza a posteriori vale:

$$P_{t|t} = \left(P_{t|t-1}^{-1} + C_t^{T}{}_{t}^{-1} C_t\right)^{-1}$$

e la media a posteriori è:

$$\hat{x}_{t|t} = P_{t|t}\, v = P_{t|t}\left(P_{t|t-1}^{-1}\hat{x}_{t|t-1} + C_t^{T}{}_{t}^{-1} y_t\right)$$

Equivalenza Con Il Guadagno Di Kalman Tramite Il Lemma Di Inversione Di Woodbury

Applicando l'identità matriciale di Woodbury all'inversa della covarianza:

$$(A + BCD)^{-1} = A^{-1} - A^{-1}B(C^{-1} + DA^{-1}B)^{-1}DA^{-1}$$

Ponendo $A = P_{t|t-1}^{-1}$, $B = C_t^T$, $C = {}_{t}^{-1}$, $D = C_t$:

$$P_{t|t} = P_{t|t-1} - P_{t|t-1}C_t^T\left(+ C_t P_{t|t-1} C_t^T\right)^{-1} C_t P_{t|t-1}$$

Definendo la covarianza dell'innovazione $S_t = C_t P_{t|t-1} C_t^T + $ e il guadagno ottimo $L_t = P_{t|t-1} C_t^T S_t^{-1}$:

$$P_{t|t} = P_{t|t-1} - L_t C_t P_{t|t-1} = (- L_t C_t) P_{t|t-1}$$

Sostituendo nell'espressione della media a posteriori $\hat{x}_{t|t}$:

$$\hat{x}_{t|t} = \hat{x}_{t|t-1} + L_t\left(y_t - C_t\hat{x}_{t|t-1}\right)$$

Questo completa la dimostrazione analitica dell'equivalenza esatta tra il teorema di Bayes e le equazioni del filtro di Kalman.

Minimizzazione Della Varianza dell'Errore E Derivazione Di L_t

Consideriamo uno stimatore lineare generico con guadagno L_t arbitrario:

$$\hat{x}_{t|t} = \hat{x}_{t|t-1} + L_t(y_t - C_t\hat{x}_{t|t-1})$$

L'errore a posteriori $x_{t|t} = x_t - \hat{x}_{t|t}$ evolve come:

$$x_{t|t} = (- L_t C_t) x_{t|t-1} - L_t v_t$$

Calcolando la covarianza d'errore (forma di Joseph, valida per qualsiasi L_t):

$$P_{t|t} = (I - L_t C_t) P_{t|t-1} (I - L_t C_t)^T + L_t R L_t^T$$

Vogliamo trovare la matrice L_t che minimizza l'errore quadratico medio $\mathbb{E}[\|x_{t|t}\|^2] = \mathrm{tr}(P_{t|t})$.

Calcoliamo la derivata matriciale rispetto a L_t sfruttando le identità $\frac{\partial}{\partial X}\mathrm{tr}(XX^T) = 2X$ e $\frac{\partial}{\partial X}\mathrm{tr}(X) = I$:

$$\frac{\partial \mathrm{tr}(P_{t|t})}{\partial L_t} = -2P_{t|t-1}C_t^T + 2L_t\left(C_t P_{t|t-1} C_t^T + R\right)$$

Ponendo il gradiente uguale a zero:

$$-2P_{t|t-1}C_t^T + 2L_t\left(C_t P_{t|t-1} C_t^T + R\right) = 0$$

$$L_t\left(C_t P_{t|t-1} C_t^T + R\right) = P_{t|t-1} C_t^T$$

Poiché $R > 0$ e $P_{t|t-1} \geq 0$, la matrice $S_t = C_t P_{t|t-1} C_t^T + R$ è simmetrica strettamente definita positiva e dunque invertibile:

$$L_t = P_{t|t-1} C_t^T \left(C_t P_{t|t-1} C_t^T + R\right)^{-1}$$

La matrice L_t del filtro di Kalman è dunque dimostrabilmente il minimizzatore globale unico della varianza d'errore di stima.